Scalability Benchmarking of a Promotional Loan System

Master's Thesis

David Benedikt Wetzel

July 31, 2022

Kiel University
Department of Computer Science
Software Engineering Group

Advised by: Prof. Dr. Wilhelm Hasselbring
Sören Henning, M.Sc.
Nils Christian Ehmke, M.Sc.

Eidesstattliche Erklärung

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Kiel, 31. Juli 2022

Abstract

A common trend is the decomposition of large software systems into multiple loosely coupled and independently deployable components in order to improve maintainability or scalability. Consequently, cloud applications' composition, orchestration, and operation become more complex, as do test and benchmark scenarios.

In this work, to address this increasing complexity, we provide a monitoring and logging approach to improve the observability of the distributed software Promotional Loan System (PLS) in Kubernetes. The main focus point of this thesis is a scalability evaluation of the observability-enhanced PLS using the cloud-native benchmarking framework Theodolite. We incorporate multiple tools into Theodolite, examine allowable service level indicators to create a service level objective (SLO), and model complex load profiles to provide a benchmark specification and an executable benchmark for the PLS. The PLS uses messages to assist processes in the promotional loan business, which may include attachments that increase the size of a message. Therefore, we perform a scalability evaluation of the PLS to investigate the degree to which the PLS is able to scale with an increasing rate of and size of messages in two analyses. The first analysis investigates different frequencies in combination with large message sizes and discovers that the maximum amount of manageable bytes is nearly identical for different frequency and size combinations. The second benchmark configuration examines high request rates and demonstrates that the PLS can handle high frequencies if message sizes are small. We discover that the PLS can scale vertically with the number of CPU cores and that the amount of provided memory significantly impacts performance. The number of CPU cores is more critical for high message frequencies than for large message sizes.

Contents

Glossary

b+m The b+m Informatik AG is an IT service provider in the financial sector and the developer of the Promotional Loan System (PLS).

message A message (german “Mitteilung”) can be send from a PI to a ZI to share information in a semi-structured way. Each message consists of a reference to an existing process, invoice, credit, or similiar. Main parts are a free text field and the possibility to add attachments.

OSM The Open Service Mesh is a service mesh designed for Kubernetes. It provides a infrastructure layer to control the network and provide network related metrics.

PI The primary institute (german “Primärinstitut”) is a financial actor, which interacts directly with customers and communicates with other institutes such as ZI, for example in processes related to the promotional loan business.

PLS The Promotional Loan System is a software solution to support credit applications.

request Requests refer to HTTP requests. Since messages are created and transmitted via HTTP requests the two terms are synonymous for certain situations.

SLI A service level indicator is a measure of a particular aspect of the service provided by a system.

SLO A service level objective specifies objectives that must be met by a system based on SLIs.

SUT In this work, the system under test (SUT) refers to the application which scalability is to be evaluated.

ZI The central institute (german “Zentralinstitut”) communicates with the PIs and other actors of the financial domain.

Chapter 1

Introduction

1.1 Motivation

Cloud computing offers various opportunities and is discussed more frequently in industrial and academic communities [Kratzke and Quint 2017; Armbrust et al. 2010]. For example, it provides opportunities for applications to scale up or down. Whereas not all applications can take advantage of these capabilities [Jamshidi et al. 2013], architectural cloud patterns [Pahl et al. 2018] offer a way to benefit from various aspects of cloud computing. Nevertheless, the distribution, orchestration, and operation of software in the cloud is becoming increasingly complex, which can be addressed by adding observability [Goniwada 2021].

For modern cloud-native applications scalability is a core requirement [Gannon et al. 2017; Henning and Hasselbring 2022a]. We understand scalability as "[...] the ability of [a] system to sustain increasing workloads by making use of additional resources" [Herbst et al. 2013]. The scalability benchmark framework Theodolite provides a methodology to evaluate the scalability of cloud-native applications [Henning and Hasselbring 2022a]. Benchmarks can be run and the results studied to gain a granular understanding of the system's behavior [Sim et al. 2003].

Financial systems are also becoming more complex and the requirements more stringent. For example, the German promotional loan business is a distributed network in which different financial institutes use potentially different infrastructures in social, economic, and technical manners. Financial institutes in the promotional loan business need to communicate and interact with each other to manage credit applications successfully. The different actors use software products such as the Promotional Loan System (PLS) [b+m Informatik AG 2021]. The PLS is a distributed and established software system in the financial sector. Those institutes must ensure that the underlying software can handle a high amount of credit applications. It becomes evident since a potential b+m Informatik AG (b+m) customer wants to know if the PLS can handle his use case. Consequently, benchmarking the software solution PLS is interesting for both the financial institutes that use the software and for b+m, the developer of PLS.

In this thesis, we enhance the observability of the financial software PLS and analyze the scalability of the PLS in Kubernetes using Theodolite. We consider the PLS as a gray-box [Ehmer and Khan 2012]. Thus, we combine network-level metrics and log messages with

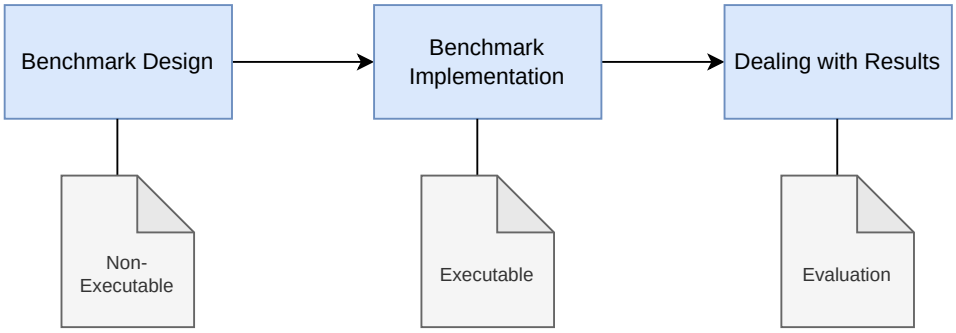


Figure 1.1. Three phases of benchmarking.

internal knowledge about the software products generated through expert interviews. Nevertheless, we do not have access to internal metrics or implementation details. Theodolite is used for the first time to investigate a distributed software system with HTTP-based communication. Therefore, a central part of this work is to identify the essential requirements to benchmark the PLS. For this purpose, we create a benchmark that includes a detailed description and an executable implementation [v. Kistowski et al. 2015]. Bermbach et al. [2017] describe benchmarking in three phases and provide a methodology. Figure 1.1 illustrates the three phases which include benchmark design, benchmark implementation, and dealing with benchmark results.

1.2 Goals

The research question for this thesis is *"How scalable is the distributed containerized application PLS?"*. We use the standardized benchmarking research method [Hasselbring 2021] and formulate the following goals (G) to answer our research question.

G1: Enhance Observability of the PLS

Our first goal is to address the challenge of the complexity of operating distributed systems in the cloud. Detailed insights in terms of monitoring, tracing or logging are required to create observability and discover the state of a system. Our objective is to enhance observability in the PLS. Specifically, we aim to combine multiple tools to enable centralized monitoring and logging of the PLS. This includes measurements of infrastructure components and the metrics on the network level.

G2: Design of a Benchmark for the PLS

We present an approach to evaluate the scalability of the PLS with Theodolite. Conceptually, the distributed PLS consists of two elements: The ZI and the PIs. The PIs submit messages to the ZI which processes those messages.

Our second objective is to provide a benchmark specification to evaluate the scalability of the PLS which describes each part of the *PLS benchmark* in detail. Core aspects are the quality, metrics, measurement method and identification of service level objectives (SLOs). The PLS benchmark should analyze how many messages the ZI can process and the maximal manageable size of messages.

G3: Implementation of the PLS Benchmark

In addition to a benchmark description resulting from the previous goal we aim to provide an executable benchmark implementation. The executable benchmark primarily includes a JMeter-based load profile, Theodolite benchmark specifications, and integration of the observability-enhanced version of the PLS resulting from G1. A technical part of this goal is the modification of Theodolite to allow the execution of this benchmark specifications.

G4: Scalability Evaluation

We state this goal to answer the research questions by a scalability analysis of the PLS. For that, we intend to use the implemented benchmark, the customized Theodolite version, and the discovered SLOs.

The objective is to benchmark the PLS system and subsequently make conclusions about the scalability of this system. For this, we simulate user interactions with the PIs and increase the number of PIs. That affects increasing messages with different sizes to the ZI. We aim to evaluate the HTTP connections between both components to address the scalability evaluation of the PLS — focusing on latency and the maximum number of or size of messages that can be processed.

1.3 Document Structure

This study is structured as follows: Chapter 2 presents all used technologies and a short introduction of the company b+m, an essential partner of this work. Motivated by G1, we explain the enhanced observability of the PLS in Chapter 3. The structure of the following main chapters are in accordance to the three phases of benchmarking. First, Chapter 4 exposes the benchmark design. Second, the benchmark implementation is addressed in Chapter 5. Third, Chapter 6 presents the experimental scalability evaluation. Related work is addressed in Chapter 7. We conclude this work in Chapter 8 and provide an outlook on future work.

Chapter 2

Foundations and Technologies

This chapter covers the foundations and technologies necessary for this thesis.

2.1 The b+m Informatik and Promotional Loan Business

The *b+m Informatik AG (b+m)* is an IT service provider with more than 25 years of experience in multiple areas of the financial sector. b+m is specialized in various financial economy topics. These include company pension plans, software engineering, promotional loan business, or artificial intelligence (AI) services.[1]

2.1.1 The Promotional Loan Business

For companies, it is possible to obtain subsidized credits with preferential conditions in terms of interest rate, grace period, or partial debt forgiveness. In Germany, companies do not directly request public loans but use intermediary financial institutions. PIs (house banks) are the gateway between the companies and the ZI. Often is the role of a ZI to mediate and coordinate between the PIs and the actual funders – so-called *Mittelgebern* – (e.g., Kreditanstalt für Wiederaufbau [KfW]) who provide the credits. In addition to the PIs, other actors such as guarantee banks are important for the promotional lending business [Pongratz and Vogelgesang 2019]. Figure 2.2 shows the structure of the German lending business and the role of the b+m software products.

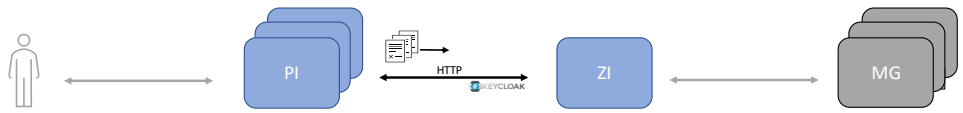


Figure 2.1. PIs send messages to the ZI. The communication is based on HTTP and secured by Keycloak. Section 2.7 describes the communication with Keycloak in detail. Schematically shown in gray are the actors communicating with the PLS like bank users and other financial institutes (credit founder [MG]) which can grant credit applications.

[1] https://www.bminformatik.de/

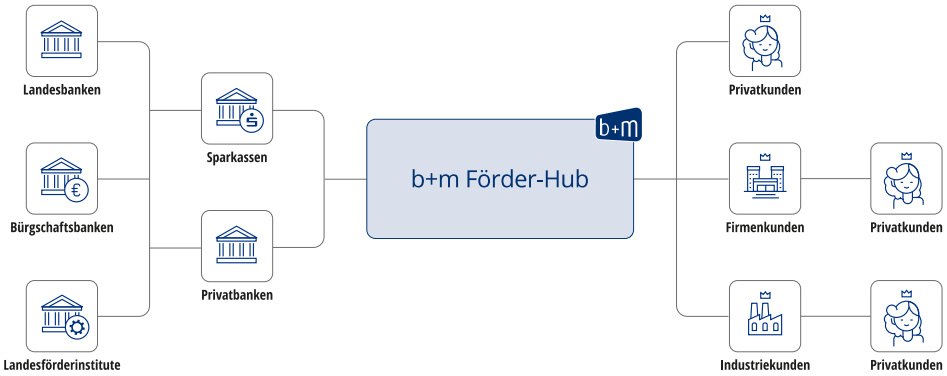


Figure 2.2. The b+m *Förder-Hub*. This figure shows the distributed and heterogeneous network of the German lending business, which is the economic environment of the PLS.[4]

The b+m software PLS assists the promotional loan business with case-closing processing of business processes between the PIs and a ZI [b+m Informatik AG 2021]. The PLS is the consequential evolution of the *FGCenter*[2] into current standardized modular architecture paradigms and consists of a component for the PIs and one for the ZI. The ZI is requested by a (potentially) large number of PIs to process loan requests. The FGCenter is on the market since 2004. The PLS supports different processes in the context of the promotional loan business such as credit applications or contract acceptance. The so-called message assist these processes by providing a semi-structured way to exchange information, as illustrated in Figure 2.1.

Technically, each individual component accesses its own *PostgreSQL*[3] database instance and can be executed as containerized application in *Kubernetes* (see Section 2.4). For security reasons is the authentication managed via the identity and access management provider *Keycloak* (see Section 2.7). Keycloak is requested by both components (PI and ZI) and provides an OpenID Connect authentication layer.

2.2 Research Method

The methodological design of this master thesis is based on *Empirical Standards*.

> **Empirical Standard**: A brief public document that communicates expectations for a specific kind of study (e.g., a questionnaire survey) [Ralph 2021].

[2] https://www.bminformatik.de/angebot/fgcenter/
[3] https://www.postgresql.org/
[4] https://www.bminformatik.de/themen/foerdergeschaeft/

In software engineering, there are several methods for conducting and evaluating research. However, different researchers understand the methods differently. The empirical standards define different research methods and provide a standard for those. This can be used to select the research method but also to determine whether all criteria of a method are met or not [Ralph 2021; Ralph et al. 2021]. For example, there are methods for *Action Research*, *Case Study*, *Engineering Research*, *Experiments with human participants*, *Grounded Theory*, *Qualitative Surveys*, *Questionnaire surveys*, *Systematic Reviews* [Ralph et al. 2021]. The related GitHub page[5] lists additional standards. Of particular interest for this work is the new standard *Benchmarking* [Hasselbring 2021].

2.3 Benchmarking Software Systems

Benchmarks allow the comparison of computer systems by comparing different configurations of one or differences between two or more systems, tools, methods, or techniques based on a standardized method [Bermbach et al. 2017; Henning and Hasselbring 2022a]. Therefore, a benchmark can be defined "[...] as a test or set of tests used to compare the performance of alternative tools or techniques" [Sim et al. 2003]. Also, v. Kistowski et al. [2015] defines "[...] a benchmark as a standard tool for the competitive evaluation and comparison of competing systems or components according to specific characteristics, such as performance, dependability, or security". Similarly, Kounev et al. [2021] describes a benchmark as "[...] a tool coupled with a methodology for evaluating and comparing systems or components concerning specific characteristics, such as performance, reliability, or security".

2.3.1 Terms

The following is a brief introduction of the terms used to define benchmarks.

Quality A quality describes a (non-) functional property of a system. A *quality level* describes the deviation between the current system state (more precisely the measured quality level) from an ideal state [Bermbach et al. 2017]. A quality could be, for example, *performance, availability*, or *scalability* [Bermbach et al. 2017; Henning and Hasselbring 2022a]. Complex qualities, like performance or scalability, can be split into sub-qualities, so-called *dimensions* (e.g., for performance this can be throughput or latency) [Bermbach et al. 2017].

Metric A metric is a function that quantifies the value of a specific quality dimension [Bermbach et al. 2017]. The assigned values are derived from (one or more) fundamental measurement. In benchmarks, metrics are used to characterize qualities of a system under test (SUT) [Kounev et al. 2021]. The literature defines multiple requirements for quality

[5] https://acmsigsoft.github.io/EmpiricalStandards/tools/

metrics. These requirements are *Correlation*, *Tracking*, *Monotonicity*, *Discriminative Power*, and *Reliability* [Bermbach et al. 2017].

Measurement methods The measuring method clarifies the usage of quality metrics to quantify the quality. It includes questions such as "what is the best (/worst) example of a quality?", or "how to stress the SUT?" [Bermbach et al. 2017]. In this work, we follow the measurement method of Theodolite as described in Section 2.6.

2.3.2 Parts of a Benchmark

A benchmark has to specify different aspects to make a benchmark executable and to fulfill all requirements (see Section 2.3.3). Bermbach et al. [2017], Kounev et al. [2021], and Sim et al. [2003] give approaches to define which parts are necessary for a benchmark specification.

Parts of a Benchmark based on Bermbach, Wittern, and Tai

Bermbach et al. [2017] formulate the *quality* of interest as a core part of a benchmark. The quality under investigation depends on the motivation of the benchmark and sets the focus for all other benchmark parts. Exemplary, a quality could be performance or scalability. Depending on the quality, a *quality-specific metric* is required. For the quality performance, this could be typically throughput or latency. Consequently, a benchmark specifies a *measurement method*. The measurement method describes how to quantify a particular quality using a particular quality metric. Also, a *workload specification* is part of the benchmark used to generate the load for a SUT.

Parts of a Benchmark based on Sim, Easterbrook, and Holt

In the approach of Sim et al. [2003] is comparison the central aspect of a benchmark. Therefore, the first aspect is *Motivation Comparison*. It means a benchmark has to define the comparison in detail including the motivation behind it to describe research relevance. Second, a benchmark must specify the considered tasks of a SUT. Since it is often not possible to consider the entire population of the task domain occurring in reality, a *Task Sample* must be selected. This sample should be as realistic as possible. Finally, the *Performance Measure* is part of a benchmark and describes the measurements and the evaluation.

Parts of a Benchmark based on Kounev, Lange, and von Kistowski

Following Kounev et al. [2021], a benchmark is characterized by the three attributes *metrics*, *workloads*, and *measurement methodology*. Here, the metrics define which values should be taken from the measurement. In a benchmark, the workload specifies the usage scenario.

While stressed with this workload, the SUT is measured during an experiment. The measurement methodology defines the overall benchmarking process (execution, collection measurements, evaluation).

2.3.3 Requirements of Benchmarks

The requirements for a benchmark specification are formulated and discussed by Huppler [2009]; v. Kistowski et al. [2015], and Bermbach et al. [2017] as followed:

1. **Relevance** The benchmark scenario should be as close as possible to the scenario of interest.
2. **Reproducibility** The benchmark must be reproducible to verify the result. The same input parameters should have the same benchmark result.
3. **Fairness** Fairness means that a benchmark should treat all potential SUTs equally by allowing different test configurations and different SUT without artificial restrictions.
4. **Verifiability** The result of a benchmark must be accurate.
5. **Usability** Different users should be able easily to run a benchmark in their environments.

2.3.4 Different Kinds of Benchmarks

Benchmarks can be run at multiple levels with each level providing a different view of the system under test. A distinction can be made between black-box, white-box, and gray-box benchmarking [Ehmer and Khan 2012]. White-box benchmarking can provide detailed investigation about internal logic and code details. In contrast, black-box monitoring and testing are symptom-based. It can detect problems and errors as they occur [Beyer 2016]. Classical black-box benchmarking uses only metrics which it can collect without access of or knowledge about the system. Benchmarking or testing with limited knowledge about internals is known as gray-box benchmarking [Ehmer and Khan 2012].

2.3.5 The (Work-) Load

Workload is generated to stress the SUT and is a cornerstone of any benchmark [Kounev et al. 2021; Bermbach et al. 2017]. Important concepts and differentiation for this work are:

Terminology The term *workload* is composed of the words *work* and *load* [Brataas et al. 2017; Brataas and Fægri 2017]. Load specifies how often an operation is called on the SUT. An exemplary load parameter could be the arrival rate. In contrast, work describes the amount of data to be processed by a service, for example, the document size could be related to work. Figure 2.3 illustrates these differences. In the literature these terms are often not differentiated. Therefore, we also use the terms workload, load, and work interchangeably.

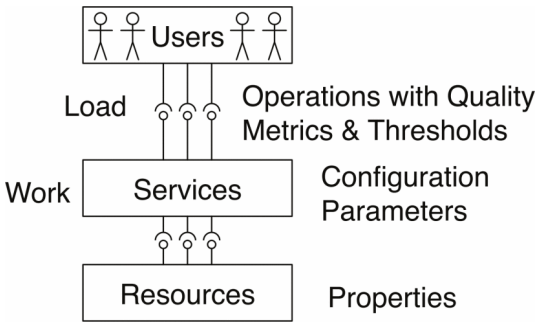


Figure 2.3. A service requires resources to run and process load. Also the service offers different operations which can be called by users [Brataas et al. 2017].

Workload categorization Workload can be categorized into the types *synthetic* and *trace-based*. Synthetic workload creates samples based on a mathematical function or random distribution. Trace-based workload generates load based on traces of real user interactions with the SUT. Depending on the scenario both methods have advantages and disadvantages. For example, trace-based load generation is more realistic than synthetic load. However, the synthetic load is not well predictable by the SUT which avoids load-specific optimization [Bermbach et al. 2017].

Further categorization is possible through the concepts of *Open-Workload* and *Closed-Workload* introduced by Schroeder et al. [2006]. In closed-workload models, new samples are generated only after processing previous ones. In contrast, open-workload models create new samples without considering old requests. A third category combines both approaches and is known as *Partly-open Workload*.

Executable and non-executable parts The generation of load can be divided into an *executable* and a *non-executable* part. The executable part (i.e., code or tools) generates the load on the target system, for example, by sending messages to the SUT. To guarantee repeatability and reproducibility, executing methods and rules must be well-defined. The *non-executable* part specifies when and how the executable parts should be executed to make the load well-defined and repeatable. For example, it defines the period and intervals of how the executable part has to be configured and executed [Kounev et al. 2021].

2.3.6 Service Level Indicators, Objectives and Agreements

Defining service level agreements (SLAs), service level objectives (SLOs), and service level indicators (SLIs) is helpfully to ensure that a specific *level of service* can be provided by a service system. It also assists in managing a software system correctly [Beyer 2016]. Therefore, the following terminology is used:

Terminology

1. **SLI** A SLI is a measurement of system behavior with respect to a well-defined aspect of the *level of service* provided by the system. For example, measuring of *request latency* or *error rate* could be considered by SLIs. Ideally an SLI measurement corresponds directly to the service level of interest [Beyer 2016].

2. **SLO** A SLO defines a target range of values for a set of SLIs that must be met for the SLO to be satisfied. There are many considerations involved in defining a SLO including target values and SLIs [Beyer 2016].

3. **SLA** A service level agreement is an agreement with users including a set of SLOs. The SLOs must be met to make the SLA satisfied [Beyer 2016].

2.4 The Container Orchestration Platform Kubernetes

Virtualization decouples hardware and software from each other. Abstracting hardware and execution environment make it possible to flexibly run software products on different systems regardless of the underlying hardware. Docker [Merkel 2014] is a modern virtualization technology. Docker facilitates the packaging of software in so-called containers. These encapsulated containers can now be executed, managed, and operated on different systems with the help of Docker without installing the application itself on the target system.

Kubernetes [Burns et al. 2016; Cloud Native Computing Foundation 2022a] is the de-facto standard for the orchestration of distributed containerized and cloud-native applications [Cloud Native Computing Foundation 2018]. For this, Kubernetes creates a cluster of physical nodes and manages the application on top of this. To make this possible, Kubernetes provides multiple objects (e.g., Kubernetes resource manifests) for declaring the properties and relations between containers, databases, or network components.

In Kubernetes there are different *kinds* of resources for different tasks. The main Kubernetes resource is the so-called *Pod*. *Deployments* and *ReplicaSets* mainly control these Pods. A Deployment manifest defines the process of deploying and executing an application. Kubernetes creates one or more Pods for a Deployment, mostly a Deployment represents an application (e.g., microservice [Dragoni et al. 2017]). Pods correspond to instances of an application. A Pod can contain one or more containers representing a single instance of a running process or application [Cloud Native Computing Foundation 2022a]. In addition to the main container other containers can be started in a Pod as so-called sidecar containers. These often provide smaller auxiliary functionalities [Cloud Native Computing Foundation 2022a]. Other resources such as *Services*, *StatefulSets*, or *ConfigMaps* allow the orchestration of complex applications.

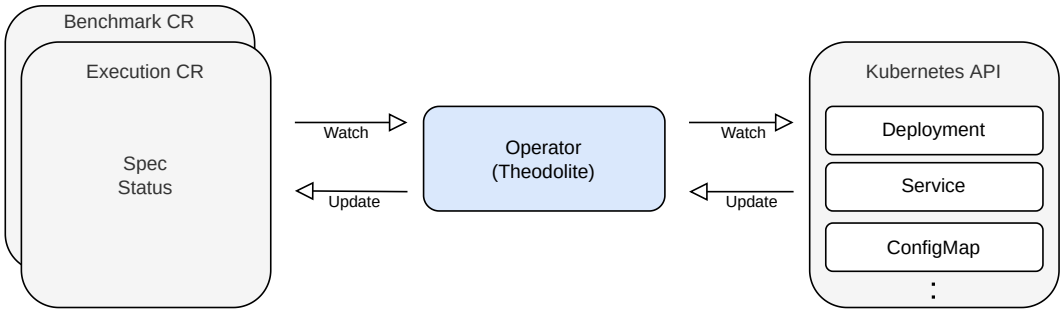


Figure 2.4. The role of Kubernetes operator (a specialized controller), exemplified by Theodolite. The operator watches and updates custom resources and modifies the cluster, to achieve the specified state.

Kubernetes controller There is one controller for each resource type. For all applied Kubernetes resources (e.g., `kubectl apply -f <resource file>`) of one kind the corresponding controller checks whether the state of the cluster meets the requirements of the resource manifest. If it does not the controller takes action to achieve the desired state, e.g., by creating additional Pods for a Deployment [Cloud Native Computing Foundation 2022a].

Custom Resource Definition In Kubernetes exist multiple kinds of resources. The *Custom Resource Definition (CRD)* is a special kind. Kubernetes can register a CRD to extend the Kubernetes API. Extending the Kubernetes API by additional kinds (i.e., CRDs) makes it possible that a user can create a resource of this kind. For example, Theodolite first registers the *Benchmark CRD* by Kubernetes. This enables a user to create a resource of the *Benchmark* kind. The benchmark CRD specifies the schema of this resource. Kubernetes validates the benchmark resource given by the user. Consequently, the benchmark resource can be managed via the Kubernetes API by standard commands like `create`, `update`, or `delete`.

Kubernetes operator For handling CRDs custom controllers are needed. These custom controllers manage custom resources. Analogous to standard controllers custom controllers look for a specific resource type (specified via a CRD) and act to achieve the desired state. On top of this, the operator pattern [Ibryam and Huß 2019] combines application knowledge and the functionality of a Kubernetes controller. Figure 2.4 visualizes the role of a Kubernetes operator/controller, exemplified by an abstract view of Theodolite.

2.5 The Quality Scalability

As formulated by Herbst et al. [2013] "scalability is the ability of [a] system to sustain increasing workloads by making use of additional resources". Three core attributes exist to describe and characterize the scalability of a system namely *load intensity*, *provisioned resources*, and *SLOs* [Weber et al. 2014; Henning and Hasselbring 2022a].

Scalability can be distinguished in *horizontal* and *vertical* scalability [Lehrig et al. 2015; Henning and Hasselbring 2022a]. Vertical scaling refers to increasing the computational power of a node; horizontal scaling refers to adding more nodes (or instances).

2.6 The Scalability Benchmarking Framework Theodolite

Theodolite [Henning and Hasselbring 2021b] is a framework for scalability benchmarking of distributed systems. Therefore, Henning and Hasselbring [2022a] specify the general benchmark requirements for the quality scalability. The current focus is on benchmarking the scalability of stream processing engines (SPEs) [Carbone et al. 2020; Henning and Hasselbring 2021b; a]. Theodolite is used in multiple studies to investigate and compare the scalability of different SPEs [Biernat 2020; Bensien 2021; Ehrenstein 2022; Vonheiden 2021; Henning and Hasselbring 2021b].

The framework consist of two parts: First, a benchmark method specifies the theoretical background for scalability benchmarking [Henning and Hasselbring 2022a]; Second, an implementation of this method as Kubernetes operator [Henning et al. 2021]. A benchmark consists of multiple components in Theodolite: Figure 2.5 depicts these components and their interactions. The benchmark tool Theodolite deploys the SUT and starts workload generators to generate the load. The configuration for this is given by a *benchmark description*. Part of the benchmark description are also *measurement methods* and the *metric*. These parameters are important to evaluate an *experiment*. An experiment is the execution of a single run in a benchmark; which tests the system under test for exactly one combination of *load-* and *resource dimension* and requires a benchmarking tool (e.g., Theodolite) and an executable load generator.

2.6.1 The Benchmark Method Theodolite

Theodolite's benchmark method quantifies scalability as a combination of resources and load values which a system can process under test [Henning and Hasselbring 2022a]. As mentioned scalability can be characterized by three core attributes [Weber et al. 2014; Henning and Hasselbring 2022a]. In Theodolite these attributes are defined as followed:

1. **Load intensity** A set L of possible load values for a specific load dimension.
2. **Provisioned resources** A set R of resources which can be provisioned for a specific resource dimension.

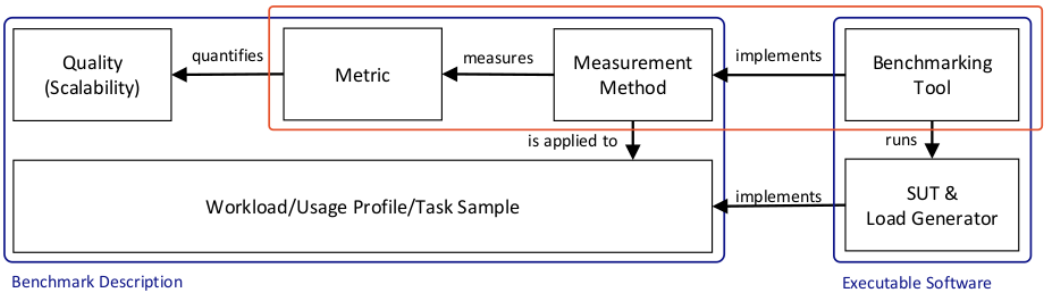


Figure 2.5. Components of a benchmark [Henning and Hasselbring 2022a]. The orange part is provided by Theodolite. Illustrated in blue are the non-executable benchmarking description (left) and the executable software (right).

3. **SLOs** A SLO describes a value or a range of a service level that should never be exceeded or fallen below [Beyer 2016]. In Theodolite S is the set of all SLOs. A SLO $s \in S$ is characterized as Boolean-value function $slo_s : L \times R \to \{true, false\}$. The value of $slo_s(l, r)$ with $l \in L$ and $r \in R$ is only true, if the SUT meets the SLO $s \in S$ for $l \in L$ load with $r \in R$ resources.

On top of this Theodolite defines two scalability metrics which are conceptually inverse of each other. With help of these metrics, scalability can be described as "how resource demands evolve with increasing load intensities" [Henning and Hasselbring 2021a].

1. **Resource Demand Metric** This metric relates load intensities to provisioned resources. This metric is denoted as $demand : L \to R$. Equation 2.1 defines this mathematically.

$$\forall l \in L : demand(l) = min\{r \in R | \forall s \in S : slo_s(l, r) = true\} \tag{2.1}$$

2. **Load Capacity Metric** This metric relates provisioned resources to load intensities. This metric is denoted as $capacity : R \to L$. Equation 2.2 defines this mathematically.

$$\forall r \in R : capacity(r) = max\{l \in L | \forall s \in S : slo_s(l, r) = true\} \tag{2.2}$$

Theodolite searches for the minimal required value $r \in R$ with that a SUT can process a load without violating the defined SLOs. After each single SLO experiment Theodolite evaluates the measured metrics with concerning the SLOs. Theodolite increases the value $r \in R$ without changing the load if a SLO is violated. In contrast, if the experiment meets all SLOs, Theodolite increases only the load value $l \in L$.

2.6.2 Theodolite's Reference Implementation

For the benchmark method Theodolite exists a reference implementation as Kubernetes operator [Henning et al. 2021]. This implementation is a cloud-native application for

Listing 2.1. Examplary Theodolite Benchmark specification as YAML file. For the resource dimension, two patchers are specified to set the requests and limits for the resource (enables vertical scaling). The load dimension need more patchers than presented.

```yaml
apiVersion: theodolite.com/v1
kind: benchmark
metadata:
  name: benchmark-pls
spec:
  infrastructure:
    resources:
       - ConfigMap:
          name: kc-resources
       -  ...
  sut:
    resources:
       - ...
  loadGenerator:
    resources:
      - ...
  resourceTypes:
    - typeName: "CPU"
      patchers:
        - type: ResourceLimitPatcher
          resource: "zi-deployment.yaml"
          properties:
            limitedResource: "cpu"
            container: pls-zi-bank
        - type: ResourceRequestPatcher
          resource: "zi-deployment.yaml"
          properties:
            requestedResource: "cpu"
            container: pls-zi-bank
  loadTypes:
    - typeName: "apm"
      patchers:
        - type: "EnvVarPatcher"
          resource: "jmeter-deployment-1.yaml"
          properties:
            variableName: "APM"
            container: "jmeter"
        - ...
    - typeName: "filePath"
      patcher:
        ...
```

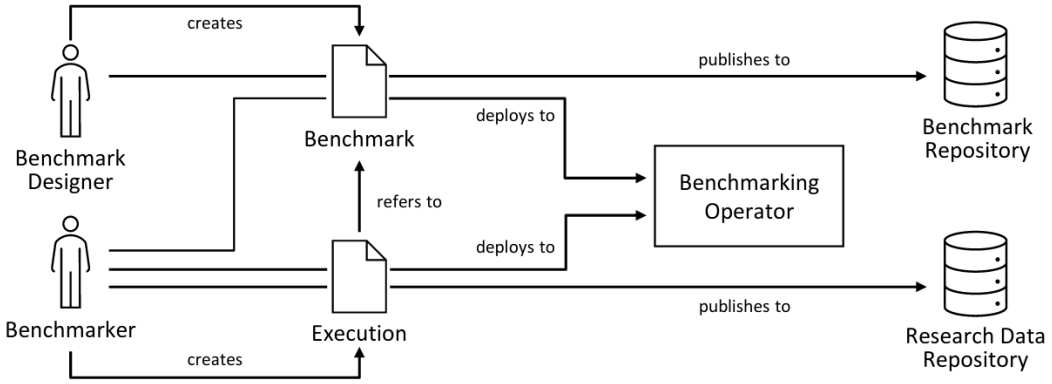


Figure 2.6. The role and data model inside the Theodolite operator [Henning et al. 2021].

Kubernetes and is able to control the execution of different benchmarks. The *Kotlin*[6]-based application is built on top of the *Quarkus*[7]-framework and can be easily installed in Kubernetes via *Helm*.[8]

Theodolite differs conceptually between a *benchmark* and an *execution*. The benchmark describes in particular the SUT, the load generation, available metrics, and possible SLOs. A benchmark is stateless and can be executed for different executions. An execution defines specific load and resource values, selects one or more SLOs, and sets duration and other attributes to configure a single execution of a benchmark. A so-called *Benchmark Designer*, who knows all SUT aspects, defines the benchmark. In contrast, the so-called *Benchmarker* is responsible for the execution. The Benchmarker can compare different benchmarks or explore the behavior of a SUT with different configurations. Figure 2.6 illustrates this concept.

Since the reference implementation is built as a Kubernetes operator (see Section 2.4), it interacts with the Kubernetes API to handle benchmarks and executions and run experiments. To make this possible Theodolite registers two custom resource definitions to Kubernetes: a *Benchmark CRD* and an *Execution CRD*. So the Benchmarker only needs to create a benchmark (illustrated in Listing 2.1) and executions (given in Listing 2.2) as custom resources in Kubernetes, and the Theodolite operator will execute the benchmark automatically.

Theodolite modifies provided Kubernetes manifest files to change the deployment of the load generator or the SUT. Therefore, the concept of so-called *patchers* [Henning et al. 2021] allows the modification of Kubernetes manifest files in a well-defined way. Each patcher can customize the YAML specification of a Kubernetes resource in a specific way.

[6] https://kotlinlang.org/
[7] https://quarkus.io/
[8] https://helm.sh/

Listing 2.2. Theodolite Execution file. Setting values for the load dimension *apm* and the resource dimension *CPU*. The number of PIs are specified through the configuration overrides.

```yaml
apiVersion: theodolite.com/v1
kind: execution
metadata:
  name: exec-rate-apm
spec:
  benchmark: benchmark-pls
  load:
    loadType: "apm" # scales the rate of created messages
    loadValues: [1,2,3,4,5,6,7,8,9,10,11,12,13,15,15]
  resources:
    resourceType: "CPU"
    resourceValues: [2,3,4,5,6,7,8,9,10,11,12,13,14,15,16]
  slos:
    - sloType: "generic"
      prometheusUrl: "http://logging-loki:3100/loki"
      offset: 0
      properties:
        externalSloUrl: "http://localhost:8082"
        promQLQuery:  "..." # see appendix
        warmup:  360
        queryAggregation: mean
        repetitionAggregation: mean
        operator: gte
        threshold: 90
    - ...
  execution:
    strategy: "LinearSearch"
    duration: 1080
    repetitions: 5
    loadGenerationDelay: 90 # in seconds
    restrictions:
      - "LowerBound"
  configOverrides:
    - patcher:
        type: PLSReplicaPatcher
        resource: "pi-deployment.yaml"
      value: "20"
    - patcher:
        type: PLSReplicaPatcher
        resource: "pi-application-service.yaml"
      value: "20"
    - ...
```

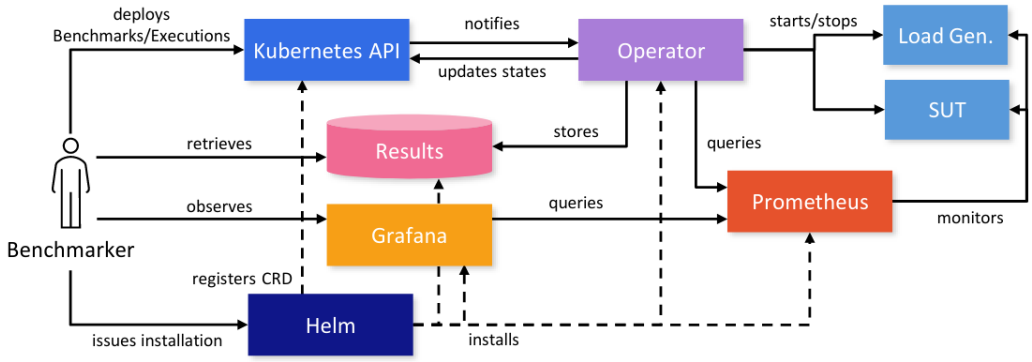


Figure 2.7. Components of the Theodolite operator including external components [Henning et al. 2021].

For example, the `ReplicaPatcher` makes it possible to alter the field *replicas* of a Deployment or StatefulSet. *Benchmarks* and *Executions* specify patchers to define a SLO experiment accurately. For example, when addressing horizontal scalability the resource dimension could be the number of SUT instances. The discussed ReplicaPatcher can be used to scale this resource dimension. Let be the load dimension be the number of requests a load generator sends per second. Assuming the load generator can be adjusted using environment variables, the *EnvVarPatcher* can be used to increase the amount of created loads by changing the value of particular environment variables.

2.6.3 External Components

The Theodolite operator needs a running *Prometheus*[9] monitoring system to collect required metrics. A *Grafana*[10] dashboard is integrated and can be helpful for the Benchmarker. Figure 2.7 shows the architecture of the Theodolite operator including external services.

2.7 The Identity and Access Management Solution Keycloak

Keycloak is an identity and access management solution for state-of-the-art applications and services. Keycloak can perform various tasks such as user management, authentication, authorization, or single-sign-on (SSO) via OpenID Connect and OAuth 2.0 [N. and Cholli 2020]. Single-sign-on [Nongbri et al. 2018] allows a user the authentication/authorization to multiple applications with only one single login. Since Keycloak handles the authentication/authorization process, the applications do not know the user credentials. A key advantage of using OpenID Connect is that a system does not need to manage user

[9] `https://prometheus.io/`
[10] `https://grafana.com/`

credentials, passwords, and user management but delegates those to an external service which authorizes the user.

In the following, we will focus only on the OpenID Connect protocol, and the associated authentication flows that are important for this work.

2.7.1 OAuth 2.0 and OpenID Connect

OAuth 2.0 and *OpenID Connect (OPIC)* [OpenID Foundation 2022] allow the federated authorization and authentication. In contrast to the first version of OAuth (Open Authorization), OAuth 2.0 [Hardt 2012] was not only designed to handle API based authorization. OAuth 2.0 allows authorization also for mobile, desktop, JavaScript applications, or browser extensions [Boyd 2012]. OAuth 2.0 allows easily authorization via an identity provider (common identity provider are Facebook or Google). OpenID Connect is built on top of OAuth 2.0 und allows easily federated authentication [Boyd 2012]. The usage of OpenID Connect in addition to OAuth 2.0 provides a higher level of security than using OAuth 2.0 alone [Pohlmann 2019].

OpenID Connect Terminology

For better understanding we introduce the most important terminology of the OpenID Connect protocol [OpenID Foundation 2022].

- **End User** The user of an application (*resource owner* in OAuth).
- **Relying Party (RP)** The application relying on the OpenID Connect provider (e.g., Keycloak) and used by the *end user*.
- **OpenID Provider (OP)** The identity system for authentication and authorization (Keycloak).
- **Claim** An information about the user.
- **Tokens** Tokens represent the identity of a user during a session and contain claims.
  - *ID Token* Provides information about the authenticated user to the application.
  - *Access Token* The access token is inserted into the authorization header when the application requests a protected API.
  - *Refresh Token* With the refresh token an application can refresh an ID or access token which expire after few minutes.
  - *Authorization Code* Special code that is sent from Keycloak to the application. This code can be exchanged for a token.
- **Endpoints** The OpenID provider provides different endpoints which can be used to obtain all the features of OIDC.

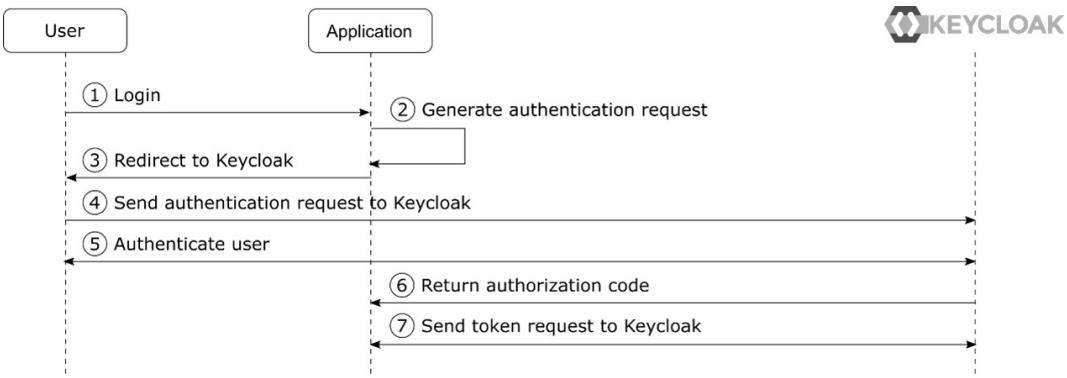


Figure 2.8. The OpenID Connect Auth flow [Stian Thorgersen 2021].

Authentication Flows

OAuth defines different *Flow Grant Types* which specify different ways for authorization. The OpenID Connect protocol refers to those as *flows* [Stian Thorgersen 2021]. Again, we focus on the flows with particular interest in this work.

- **Authorization Code Flow** The authorization code flow is the most common way for user authorization via OpenID Connect and Keycloak. This flow can be used for authentication of *end users* to an application and has 7 steps (shown in Figure 2.8) [Stian Thorgersen 2021].

  1. The user clicks on the login button.
  2. The application generates an authentication request (302[11] to the Keycloak authorization endpoint).
  3. The user-agents receive the redirection message.
  4. The user-agents redirect to Keycloak with the given parameter from the redirection message.
  5. The user enters their username and password to the Keycloak login page.
  6. After authorization Keycloak sends an authorization code to the application.
  7. The application exchanges the authorization code to an ID or refresh token.

- **Client Credential Grant** The client credential grant type can be used when an application calls a protected REST API. For this the application needs an access token from the identity provider. With the *client credential grant* the application (client) uses its credentials to authenticate themselves to the Keycloak server. In order to get the access token the application sends a POST request to Keycloak containing the credentials, and

[11] HTTP status code for redirection

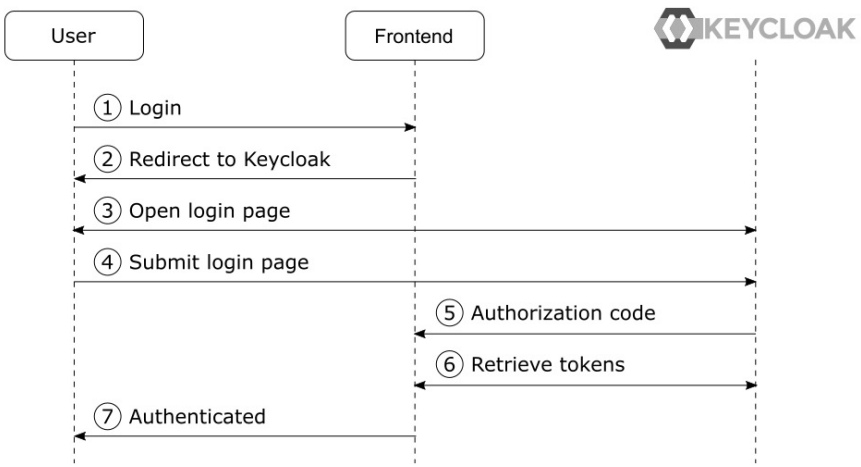


Figure 2.9. The usage of access tokens [Stian Thorgersen 2021].

Keycloak returns an access token for the application. Figure 2.9 shows this process with a frontend and a backend component. As seen below this process consists of four steps [Stian Thorgersen 2021].

1. The backend retrieves the public keys of the identity provider (Keycloak).
2. The frontend sends a request to the backend. The authorization header contains the access token.
3. The backend verifies the access token (using the public keys).
4. The backend returns.

2.8 The Load Generator Apache JMeter

For load generation there exist several tools and different approaches. One of the most popular tools is *JMeter* [Erinle 2017]. Murawski et al. [2014] offer a detailed performance evaluation and comparison to other load generator tools.

The highest configuration object in JMeter is the *test plan* a JMX (Java management extensions) file specifying all parameters and is executable directly by JMeter. In each test plan *thread groups* are the entry point for each test specification. Thread groups combine and configure *plugins* to control the exact behavior of JMeter. For that the thread group creates different threads, and each thread executes the configured plugins. Threads are the way for JMeter to emulate user behavior so each thread can be called an *emulated user*. The users can differ from each user by the number of the individual thread. Many plugins exist for this plugin-based architecture making it possible to benchmark and test various scenarios. JMeter can test the performance of web applications [Erinle 2017] but

plugins also exist to test messaging systems like Kafka or other systems. JMeter provides a graphical user interface to generate this JMX file; a CLI exists for interacting with JMeter to run the benchmark [Apache Software Foundation 2021; Matam and Jain 2017].

Distributed testing is possible for large test cases. Meaning that a minimum of three instances of JMeter run together via the master/slave concept [Matam and Jain 2017] to generate a high number of queries [Apache Software Foundation 2021]. There are tools for performing the distributed tests in Kubernetes, for example, a dockerized version,[12] or a Helm chart to quickly deploy it to Kubernetes.[13]

2.9 The Open Service Mesh

A *service mesh* is a special network layer that provides additional policy-based services to network-related components in Kubernetes. Typically it is implemented by adding a network proxy to a single software component without affecting the application code [Li et al. 2019]. A service mesh is typically deployed within a *sidecar container* to control a Pod's network behavior [Cloud Native Computing Foundation 2022a]. The network can manage multiple aspects of a distributed system such as security, configuring the network, or providing monitoring data [Li et al. 2019]. The *Open Service Mesh* is an open-source implementation and based on the network proxy *Envoy* [Lyft 2022; Cloud Native Computing Foundation 2021b].

[12] https://github.com/pedrocesar-ti/distributed-jmeter-docker

[13] https://github.com/benediktwetzel/charts/tree/master/stable/distributed-jmeter

Chapter 3

Enhanced Observability of the PLS

In this chapter, we address the first goal and explain the enhanced observability of the PLS. The observability of a system is a measure of how effectively its internal state can be inferred from its external outputs [Goniwada 2021]. In contrast, monitoring collects, analyzes, and visualizes system metrics, telemetry data or logs, and can provide information whether a system can meet its SLOs and is able to enhance the observability of a system [Goniwada 2021]. In the first part we describe the tooling and in the second part we focus on the integration of components to build a comprehensive view of the PLS.

3.1 Monitoring Tools

In this section, we explain the tools to increase the observability of the PLS in Kubernetes.

1. **Open Service Mesh** The *Open Service Mesh* is designed for Kubernetes and is used to inject a proxy (*Envoy*) as a sidecar container to each individual Pod. In addition, OSM configures each Pod to route all traffic through these Envoy proxies. We activate the *permissive traffic policy mode* to allow all traffic between components in the mesh. Making it possible to collect network-related metrics such as the number of or duration of HTTP requests.

   Limitations are that Open Service Mesh (OSM) can only monitor HTTP-based network communication. Since the PostgreSQL connection is TCP-based, OSM does not provide metrics about the communication with PostgreSQL. However, in contrast to OSM the underlying Envoy proxy provides rudimentary TCP-based metrics. Additionally, it is not possible to filter recorded traffic by the HTTP method or the requested path. Therefore, the latency between PIs and ZI is a measurement based on all requests and not only on requests required to create a message.

2. **Grafana Dashboard** Dashboards visualize monitoring and support multiple objectives [Few 2006]. We use a Grafana dashboard for getting an overview and potentially detecting bottlenecks. Figure 3.2 shows a part of the created dashboard.

3. **Prometheus** Prometheus is used to store collected numeric metrics as time series. Prometheus offers with *PromQL*[1] a query language to select and aggregate metrics.

[1] https://prometheus.io/docs/prometheus/latest/querying/basics/

4. **Grafana Loki** Loki [Veeramachaneni 2018] is a storage for log messages. Loki queries the Kubernetes API to figure out the matching metadata of stored logs. Based on this metadata, Loki adds labels to the stored logs in the same way as Prometheus. The advantage is that Prometheus and Loki metrics can be correlated using identical labels. Similar to Prometheus, Loki provides *LogQL*[2], a query language that allows aggregation and calculations on top of logs. There exist two types of queries: *Log queries* return or filter log line content. *Metric queries* calculate values based on log lines, for instance, the rate per minute of logs containing a particular string.

5. **Grafana Promtail** Promtail [Veeramachaneni 2018] runs as an agent on each node. Node-level agents are a common approach for cluster-level logging architectures. This agent has access to a directory containing log files from all applications, running on the individual node [Cloud Native Computing Foundation 2022a]. Promtail queries the Kubernetes API to receive metadata to relabel logs similarly to Prometheus. The advantage is that time series metrics and logs can be correlated based on their labels. Finally, Promtail transfers all logs to Loki. These logs are used to determine how many messages are registered by the ZI.

6. **cAdvisor (Container Advisor)** Kubernetes provides metrics about the resource consumption of different resources via *cAdvisor*.[3] We use cAdvisor to identify bottlenecks and resource requirements of different components.

7. **Node Exporter** The Node Exporter[4] is an official Prometheus exporter developed to monitor the host system by collecting all hardware and operating-system-level metrics.

8. **PostgreSQL Exporter** The Prometheus *PostgreSQL Exporter*[5] is used to provide simple statistics about the performance of PostgreSQL.

3.2 Integration of Monitoring Tools for Observability

In this section, we explain the integration of the described monitoring components. Therefore, we focus on the communication between a PI and a ZI running in Kubernetes.

Figure 3.1 illustrates a Kubernetes node on which a PI and ZI are running. Both Pods interact with a PostgreSQL database and communicate with each other. For simplification, the communication with Keycloak or user interactions are not displayed but also monitored. The green dashed part in Figure 3.1 shows the scope of the service mesh. The service mesh adds an Envoy-based proxy (green) as a sidecar container to the running Pods. The network communication between Pods in this scope is redirected through these proxies. Prometheus scrapes metrics from cAdvisor, Node Exporter, Envoy, and the PostgreSQL

[2] https://grafana.com/docs/loki/latest/logql/

[3] https://github.com/google/cadvisor

[4] https://github.com/prometheus/node_exporter

[5] https://github.com/prometheus-community/postgres_exporter

Exporter. The dashed black lines show monitoring-related communication. Usually, Pods write logs to standard out (stdout) and standard error (stderr). Promtail collects and sends these logs to Loki. cAdvisor and the Node Exporter provide information about resource usage of all Pods running on the individual nodes. The Node Exporter and Promtail run as Pods controlled by DaemonSets [Cloud Native Computing Foundation 2022a]. In contrast, Kubelet integrates cAdvisor natively [Cloud Native Computing Foundation 2022a]. Prometheus and Loki provide an HTTP-API to query stored data. The dashboard solution Grafana uses both API to visualize the measurements (as illustrated in Figure 3.2).

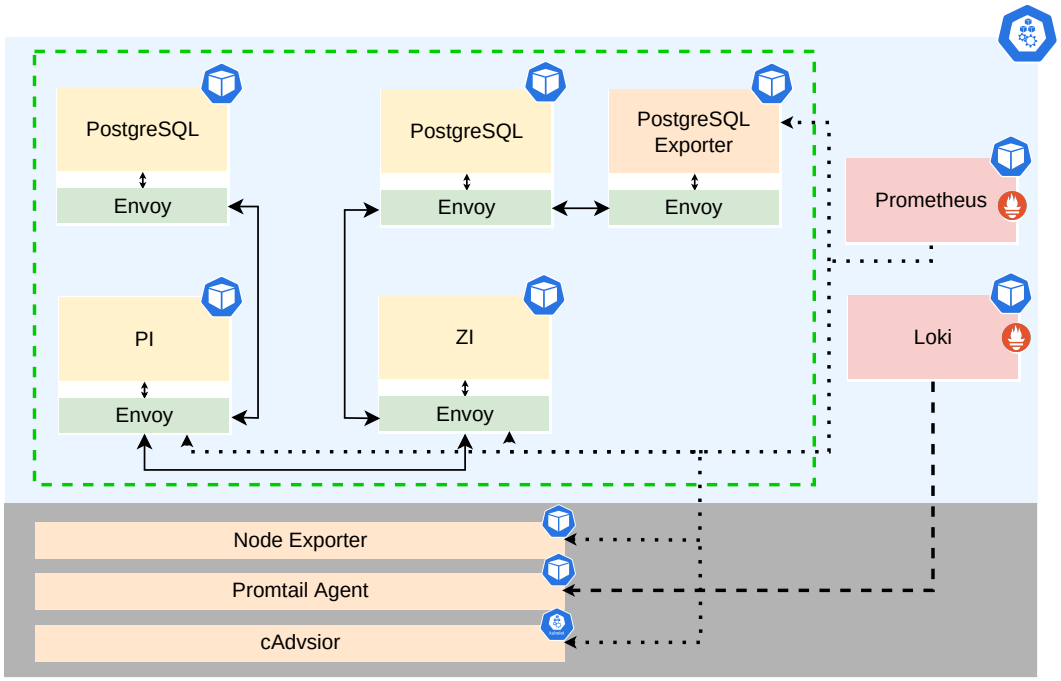


Figure 3.1. Integration of different monitoring tools to enhance the observability. The blue part shows the Pods running on a node, the gray part shows components that interact directly with the node.

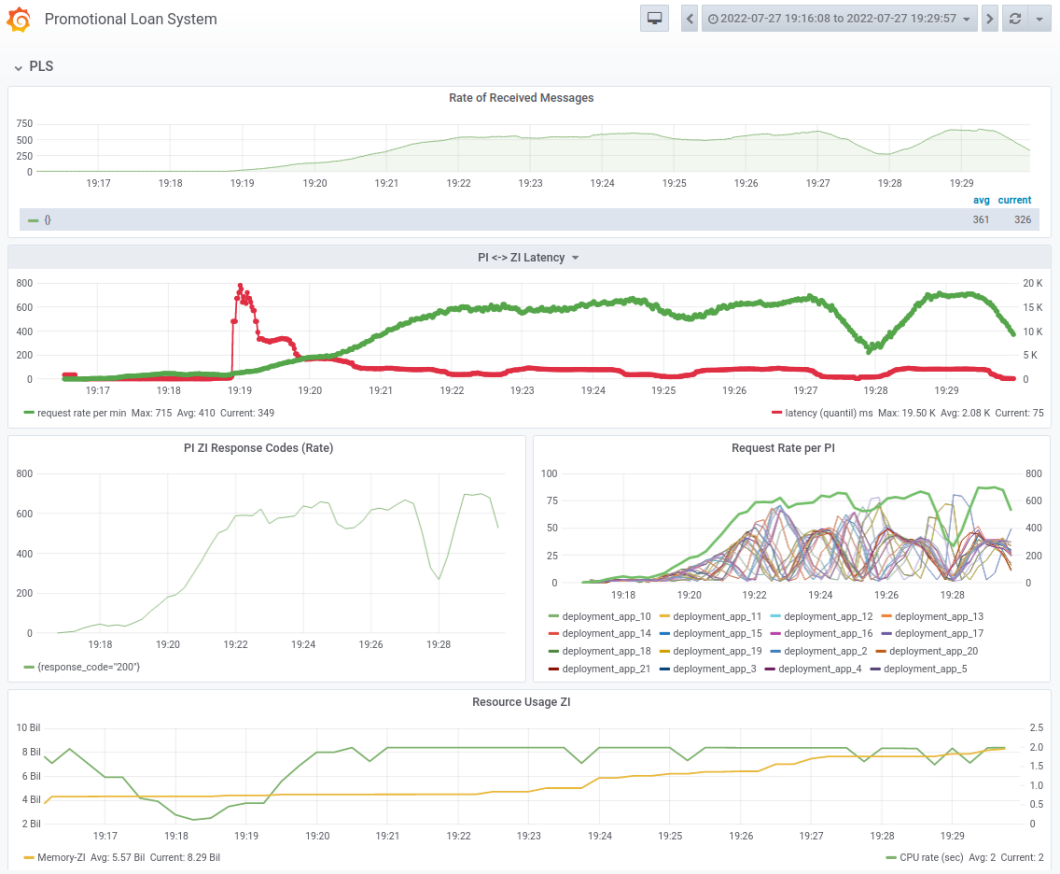


Figure 3.2. A view of the Grafana dashboard. It shows the rate of received messages, the rate of and the latency of HTTP requests between PIs and ZI. The bottom graph depicts the resource consumption of the ZI in terms of CPU and memory.

Chapter 4

The PLS Benchmark Design

This chapter presents the specification of the PLS benchmark to address the second goal. The specification is used to implement and run scalability benchmarks on the PLS as described in Chapters 5 and 6 and corresponds to the first benchmarking phase (illustrated in Figure 1.1). This definition is necessary to make the results traceable and ensure technical reproducibility [Papadopoulos et al. 2021]. Also, the research method *benchmarking* requires comprehensive specifications and precisely defined procedures when conducting a benchmark study [Ralph 2021; Hasselbring 2021]. We give an overview of the requirements for benchmarks in Section 2.3.

The described benchmark examines the core functionality of different versions of the same software provided via the HTTP-API. Therefore, we assume that the generated load is comparable (identical) on all target systems. Accordingly, the benchmark is *fair* [v. Kistowski et al. 2015; Bermbach et al. 2017]. As illustrated in Figure 2.1 a single ZI communicates with a potentially large number of PIs which send messages to exchange information. According to reality, the source of the messages sent to the ZI are the PIs in this benchmark, so we assume that the scenario is realistic. Therefore, we conclude the benchmark as *relevant* [v. Kistowski et al. 2015; Bermbach et al. 2017].

In the following, we use the components of a benchmark introduced by Henning and Hasselbring [2022a] to explain the PLS benchmark. Figure 4.1 highlights the left part (*Benchmark Description*) of Figure 2.5, which represents the interaction of the individual components.

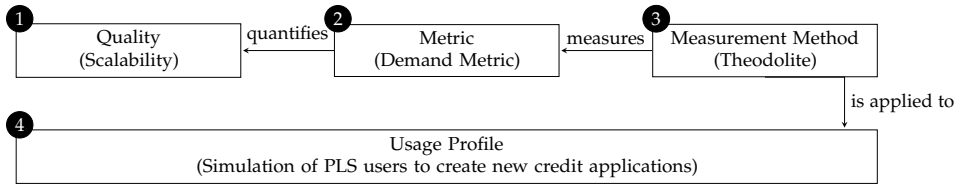


Figure 4.1. Main components of a benchmark specification based on Henning and Hasselbring [2022a].

4.1 The Quality to be Benchmarked

❶ In this study, the working hypothesis is that the performance of the PLS depends on two conditions: The configuration of the infrastructure (e.g., the databases) and the resource provisioning for the ZI. Here, performance addresses the ability to handle different frequencies or sizes of messages.

Consequently, the vertical scalability of the ZI is of interest and therefore scalability is the quality under investigation. In this context, scalability is the ability to increase the number or size of successful centrally managed messages when more resources become available (see Section 2.5 for detailed information about scalability). That is, the number of messages sent (decentral) from PIs to a (central) ZI, acknowledged with an HTTP status code *200*.

4.2 The Load Profile

❹ We consider messages to benchmark the PLS. Messages are an essential part of semi-structured communication around the credit process. A message contains various information about the promotional loan such as contact information about the borrower, or a contact person. A free text area for adding unique texts and file attachment possibility are the main elements.

The load profile describes the interaction of bank clerks with the PLS to create messages. Therefore, we simulate office users in primary finance institutes. These users interact with the web dashboard of the PIs and send messages to the ZI. Two users are required to create, control, approve, and release a message on a PI, before the PI sends it to the ZI. This combination of two users follows the *four-eyes principle* [Bilzhause et al. 2016]. Figure 4.2 shows all required steps to create and send a message from the user's point of view. We divide the workload logically into two parts: The first part is the interaction between a user and a PI, and the second part is the communication between a PI and the ZI. We only focus on the first part. The PIs automatically create the second step by sending the created messages to the ZI.

This workload is based on user interactions which is called trace-based workload [Bermbach et al. 2017]. A trace consists of a sequence of recorded instructions that specify requests in detail [Bermbach et al. 2017]. Repeating a trace as a workload is a method for loading a system in a manner similar to production mode, making this benchmark specification more realistic [Gregg 2020].

(1) **Login** During this step, the first user (*bank clerk*) has to authenticate and authorize at the PI. In the background this involves interactions with Keycloak (see Section 2.7) to obtain the required access token.

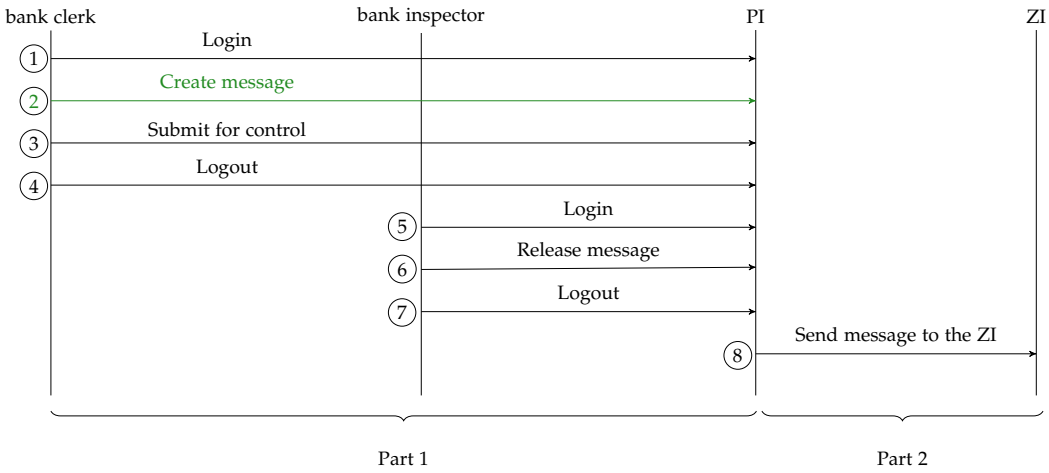


Figure 4.2. Sequence of generated requests to trigger the PIs to send messages to the ZI. The communication between the *bank clerk*, *bank inspector*, and PI is the first part of load generation. The second part is illustrated by the communication between the PI and the ZI. The green arrow shows the interchangeable part to create different types of load.

(2) **Creation of a message** In this step, a new message is created on the PI. We assume that the exact content of a message is not crucial. Therefore, all created messages contain identical values for all fields such as borrower or contact person.

The exact behavior in this step is interchangeable and depends on the examined load dimension. For example, in this step a user can create multiple messages, or add one or more attachments to a message. Theoretically, *credit applications*[1] could also be created instead of messages. The usage of credit applications would also affect the implementation of step (6).[2]

(3) **Submit for control** Before a message can be sent to the ZI, a second user needs to approve the message. In this step, the message is marked for control so that another user can approve and release the message. This is the last step for the first user (*bank clerk*) as seen in Figure 4.2.

(4) **Logout** The final step for user 1 (*bank clerk*) is to log out.

(5) **Login** In this step, the second user (*bank inspector*) logs in as described in step (1).

[1]Credit applications can be used to request credit and requires more calculation steps than a message.
[2]We also provide a prototype test plan for this scenario, but since executing a benchmark takes much time, corresponding benchmarks are not considered in this work.

(6) **Release the message** In this step, the second user approves and releases the message. This is the essential step for user 2 (*bank inspector*) of Figure 4.2.

(7) **Sending to the ZI** As a consequence of the previous step the PI will send the message to the ZI. This step requires additional authorization via Keycloak to allow sending requests from a PI to the ZI.

(8) **Logout** The last step of user 2 (*bank inspector*) is to log out. This is the final step in this load profile.

4.3 The Metric to Quantify the Quality

Henning and Hasselbring [2022a] proposed a scalability metric that quantifies the scalability of a system as described in Section 2.6. This metric relates load intensities to provisioned resources concerning a specific set of SLOs. This metric is denoted as $demand : L \to R$. L is the set of load values, R is the set of possible resources, and S is the set of SLOs which must meet by the SUT. It means that for each considered combination of load value $l \in L$ and resource value $r \in R$, the metric gives the information whether a system can or can not meet a set of SLOs. Equation 2.1 defines this mathematically. This benchmark defines the set of load values (L), resource values (R), and the SLO as follows:

1. *Set of load values L* The described load shows how emulated user interact with the PLS. As described in (2), a user can perform different actions. Different load scenarios are possible by varying performed actions and minor adjustments to the load implementation. We map each scenario to an individual load dimension. The load profile remains unchanged during the experiment in all scenarios. The set of load values L depends on the load dimension under investigation. We define the following load dimensions:
   (a) *Rate of created Messages* Adjustment of the rate at which messages are transmitted to the ZI. This load dimension can be divided into two sub-dimensions.
      i. *Number of PIs* Adjustment of the number of PIs that send messages to the ZI at the same fixed rate.
      ii. *Message Rate* Adjustment of the transmit rate used by the PIs to send messages to the ZI. This can be done by increasing the number of simultaneous accessing users, or by increasing the frequency of each user or both.
   (b) *Size of Messages* By adding attachments the size of messages are changeable. This also changes the load on the target system. This load dimension can be divided into two sub-dimensions.
      i. *Attachments Size* We adjust the size of the messages by adding attachments. Considered attachments are *PDF* files of different sizes.

ii. *Number of Attachments* Each message can contain multiple numbers of attachments. Increasing the number of attached *PDF* files increases the size of messages.

2. *Set of resource values R* The set of resource values R is the number of provisioned CPU cores for the ZI. Multicore CPUs consist of several cores, which contain different hardware threads. A hardware thread is visible as a logical CPU [Gregg 2020]. In Kubernetes provided cores are either logical or physical, depending on the underlying hardware [Cloud Native Computing Foundation 2022a]. Only provisioning of whole CPU cores is considered; provisioning of fractions of them (e.g., half of the execution time of a core) is not part of the resource set R.

3. *Set of SLOs S* The fundamental question is how many messages a ZI can handle. We consider a message as processed successfully if the PI receives an HTTP response that does not indicate an error. The SLO for this benchmark is based on the *arrival ratio* between created messages and received messages by the ZI. The *SLO requirement* is that the ZI is able to receive more than a specified percentage of the total number of generated messages. This should be done in a reasonable duration of time and without errors. The following sub-SLOs are defined to assess whether this SLO is satisfied.

   (a) The arrival ratio of received messages per minute (msgs/min) in contrast to the rate of created messages must be at least as high as a defined threshold value (if the ZI is not able to receive a certain percentage value of the created messages, it is overloaded).

   (b) The latency is not allowed to exceed a defined threshold value (since Jake 2009 reports that high latency can reduce the number of users in Google search, we conclude that high latency can reduce the user experience of the PLS web interface).

   (c) The measurable error must be smaller than a specific threshold (if too many network errors are detected between PIs and ZI, we consider an experiment as unsuccessful. Alternativly, if too many errors between JMeter and the PIs can be measured, we assume that the load can not be created and no conclusion of this experiment can be drawn).

   The exact thresholds for the SLOs depend on the benchmark implementation and configuration. The standard values of this benchmark are given in Table 4.1. The arrival ratio is set to a minimum of 90 %. A rough approximation causes this low threshold to filter out the variability and fluctuations in this metric. The latency between the PIs and the ZI should be in a reasonable amount of time. Since the transfer of large files takes a considerable amount of time, we set the maximum accepted latency to 20 seconds. We accept a maximum rate of 20 error indicating responses on average between the PIs and the ZI or between JMeter and the PIs. We assume, that higher HTTP error rates indicate that the PLS runs faulty.

Table 4.1. Default thresholds and configurations of the PLS benchmark (used in Chapter 6). We describe the implementation of the measurements for the corresponding SLIs in Section 5.7.

SLO	Aggregation	SLI	Comparator	Threshold
The ZI should receive most of created msgs/min	mean	arrival ratio metric	$\geqslant$	90 %
The latency between PIs and the ZI should be bounded	median	p90 latency in ms	$\leqslant$	20 ms
The measurable errors on HTTP level should be low	mean	send/create errors per minute	$\leqslant$	20 responses

4.4 The Measurement Method

3 In this work, we use the measurement method of Theodolite. As described in Section 2.6 this method is based on the concepts of *SLO experiments* and *search strategies*. Details about the implementation and execution of individual experiments are explained in Chapter 5. The configuration is presented in Chapter 6. While this benchmark examines SLO experiments with different points of view, the selected search strategy for all experiments is the linear search. The linear search starts with the lowest resource value and successively increases it if necessary [Henning and Hasselbring 2022a].

Chapter 5

The PLS Benchmark Implementation

Based on the benchmark specification from Chapter 4, we address the third goal and develop a benchmark implementation to perform the scalability evalution of the PLS. This chapter corresponds to the second phase of the benchmarking process described from Bermbach et al. [2017]. According to Figure 5.1, we have organized this chapter and the description of this implementation by addressing each component in a separate part.

The main focus points are on the load generation as well as the adaption of Theodolite to be able to execute this benchmark. Henning and Hasselbring [2022a] describe this as the executable part of a benchmark as shown in the right part of Figure 2.5. Finally, we present a systematic description of defined SLIs and details regarding underlying measurements.

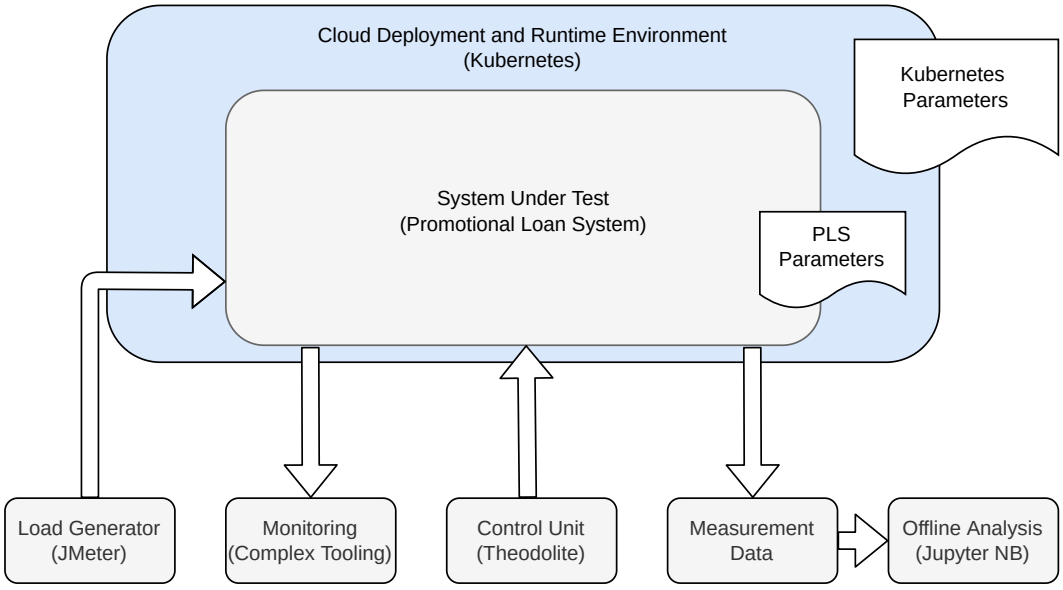


Figure 5.1. Core components of a benchmark implementation based on Bermbach et al. [2017].

5.1 Runtime Environment

b+m operates the PLS in Kubernetes. Thus, Kubernetes is a realistic execution environment and the selected benchmarking environment. The implementation of Theodolite is designed for the cloud and allows benchmarking of distributed systems in Kubernetes [Henning and Hasselbring 2022b].

5.2 The System Under Test

The SUT is the observability-enhanced version of the PLS as introduced in Chapter 3. The PLS consists of the PIs and the ZI. The ZI needs to handle different kinds of incoming requests for a potentially large number of PIs. Therefore, we consider only the ZI as the SUT and the PIs as part of the load generation (PIs are used to stress the ZI).

The PLS is considered as a *gray-box* (see Section 2.3). Therefore, the only parameterization is the configuration of resource provisioning through Kubernetes by setting resource *requests* and *limits*. While a container can use more resources as requested it is not allowed to use more than the specified limits. The Kubernetes scheduler uses resource requests to decide which node to deploy a Pod (e.g., a PI or ZI) [Cloud Native Computing Foundation 2022a]. In production, b+m sets both the resource requests and limits to 24 GB memory and 4 CPU cores. For testing, the memory is limited to 2 GB. We set the default values based on the results of our first exploratory pre-study (as shown in Table 6.2). The CPU limits of the PIs are nearly identical to b+m defaults. The PIs are part of the load generation and the provisioned resources may affect the performance of a PI and consequently the load generation. Therefore, these limits can affect the results and are part of the configuration of a benchmark execution.

The main focus lies on the system behavior of the application level, meaning we are looking at the system behavior from a user's point of view. In consequence, this benchmark implementation uses the OCI[1] images provided by b+m without knowledge of the implementation details.

Keycloak and PostgreSQL The PLS uses Keycloak as an identity provider and requires a specific configuration to enable user management and security features. b+m provides a *Keycloak tool* which is able to perform this configuration for the PIs and the ZI. In addition, each Pod of the PLS requires a PostgreSQL database, the applications are able to perform the configuration of PostgreSQL. Section 6.1.3 describes resource requests, limits, and used software versions.

[1] Open Container Initiative

5.3 The Load Generation

This section consists of three parts. The first part (*Workload Description*) explains the load based on HTTP requests. The second part (*Workload Implementation*) addresses the implementation. The third part (*Load Dimensions*) describes how to scale the workload in different scenarios.

5.3.1 The Load Description

In this section, we focus on the workload itself. Section 4.2 describes the load profile. The load generation is separated into two parts (as illustrated in Figure 4.2). In the first part, JMeter interacts with the PIs to create messages. Consequently, in the second part the PIs send messages to the ZI.

The workload generation with JMeter is based on the concept of emulated users [Apache Software Foundation 2021]. An emulated user requests the target system in a specified way. The first part of the load generation contains four steps, in each step, an emulated user sends requests via JMeter to a PI. We configure JMeter to be able to perform browser-like actions such as caching or cookie management. The workload contains multiple requests with *inter-request dependencies* [Hashemian et al. 2012]. To model this behavior, a user interacts with the requested system in *sessions* [Hashemian et al. 2012]. Each session consists of a sequence of HTTP requests and each request depends on the previously send requests. For example, a user can mark a message as *controlled* only if the message was processed before. In this implementation JMeter waits for the response of the previous request before sending another one. This concept is known as *closed-workload* [Hashemian et al. 2012]. JMeter can simulate the concurrency of multiple emulated users via multithreading [Apache Software Foundation 2021; Matam and Jain 2017].

A detailed description of the workload implementation follows below. It includes details about HTTP requests between JMeter and Keycloak to obtain required tokens. Figure 4.2 shows the sequence in each session to create a message and to send it to the ZI from a user's perspective. The following refers also to Figure 4.2 but describes the technical view. Figure 5.2 shows each part of this profile by a sequence diagram.

(1) **Login (user 1)** The required HTTP requests to perform the login process for a user on a PI:

Step	Destination	Description	Correlation with previous requests
1	PI	get state token	
2	Keycloak	get submit URL	state token
3	Keycloak	login and get authorization code	state token
4	Keycloak	get access token	authorization code, state token

(2) **Creation of a message (user 1)** The required HTTP requests to create a message. The order of these steps corresponds to the dashboard user guidance. Step 4 is optional and only enabled if a user should add attachments (depends on the scenario under consideration):[2]

Step	Destination	Description	Correlation with previous requests
1	PI	get metadata*	access token
2	PI	post schema	access token
3	PI	post, create message, extract message ID	access token
4	PI	put, add attachments	access token, message ID

(3) **Submit for control (user 1)** Necessary steps to set the state of a message to "To Control". When using the dashboard the validation step is performed automatically to ensure that the message meets all requirements before control:

Step	Destination	Description	Correlation with previous requests
1	PI	validation	access token, message ID
2	PI	set state of ready for control	access token, message ID

(4) **Logout (user 1)** HTTP requests and correlation to perform the logout process:

Step	Destination	Description	Correlation with previous requests
1	Keycloak	send logout request	access token

(5) **Login (user 2)** The steps for the login process are shown above.

(6) **Release the message (user 2)** HTTP requests to approve and release a message. Initially the user must define themselves as the editor of the message. Afterwards the message can be controlled and released. The implementation with JMeter contains of additional queries to obtain the required information (message ID) to assign a message to user 2:[2]

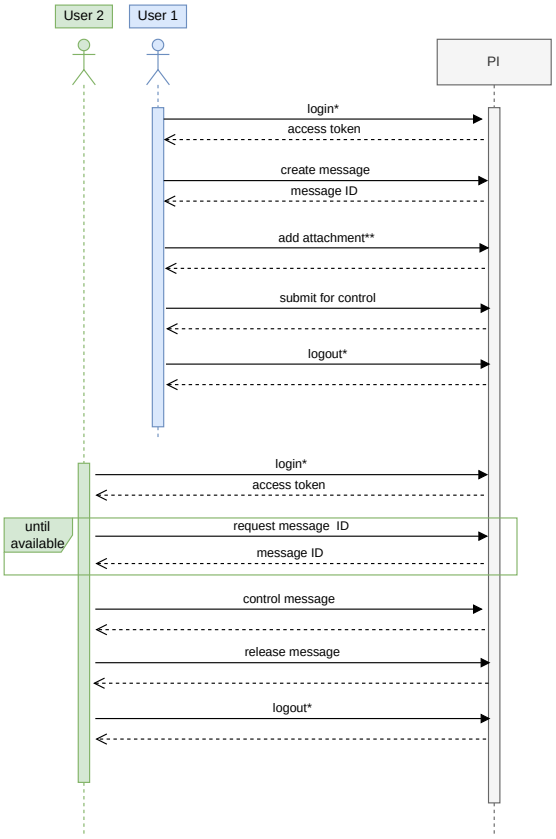
Step	Destination	Description	Correlation with previous requests
1	PI	show messages, get message ID	access token
2	PI	take message	access token, message ID
3	PI	show metadata*	access token, message ID
4	PI	control message	access token, message ID
5	PI	validation	access token, message ID
6	PI	release	access token, message ID

[2] "*" marked requests based on recorded traffic, i.e. to make the load more realistic

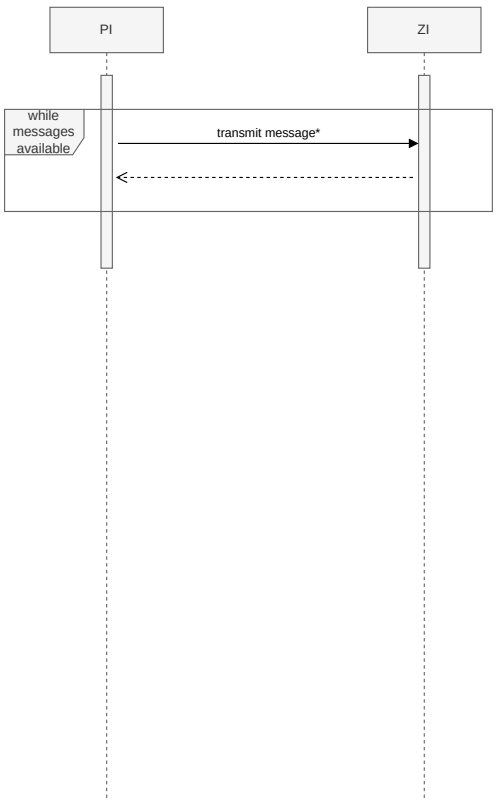
(7) **Logout (user 2)** The required requests for the logout process are explained in (4).

5.3.2 The Load Implementation with JMeter

The previous section presents the required HTTP requests. In this section, we explain how we create these requests. Technically, load generation with JMeter is based on combining several plugins. Each plugin provides specific functionality. Integrating different plugins allows the creation of complex test scenarios. For that reason, we list used plugins and describe the configuration to create a specific load.



(a) Part 1: User interactions to create, control, and release messages.



(b) Part 2: A PI synchronously sends created messages to the ZI.

Figure 5.2. Simplified diagram to illustrate the load profile. User 1 and 2 create messages and mark them as ready for transmitting to the ZI. PIs transfer messages synchronously and sequentially to the ZI (* requires additional requests to Keycloak, ** optional)

Used JMeter Plugins

The exact configuration and combination of these plugins[3] define the JMeter test plan. Figure 5.3 shows a test plan to create high rates of messages. The used plugins are:

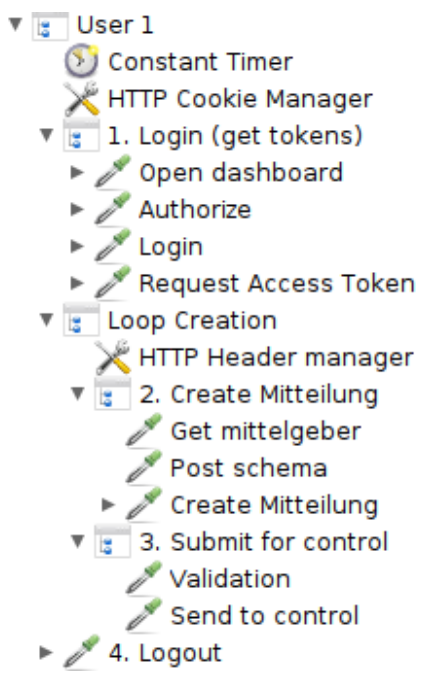
1. **HTTP Request** allows sending HTTP requests and handles the response.
2. **HTTP Header Manager** allows adding or override the HTTP request header.
3. **HTTP Cache Manager** allows adding browser-like cache functionality for each individual virtual user.
4. **HTTP Cookie Manager** allows handling cookies. The cookie manager extracts all cookies from the HTTP responses and uses them for further HTTP requests. Additionally, the cookie manager allows to add cookies manually.
5. **Regular Expression Extractor** allows extracting elements from HTTP responses that match the specified regular expression. This plugin can be used as a *post-processor* after each request. This plugin used to extract the access token or the ID of created messages from the header or body of a response.
6. **JSR223 Sampler** supports calculation, updating variables, and creating samples with JSR223 script code [Zukowski 2006; Apache Software Foundation 2021]. This sampler is mainly used for internal test logic.
7. **Constant Throughput Timer** allows the specification of JMeter to create samples (e.g., HTTP requests) with a fixed rate per minute.
8. **Constant Timer** specifies a timeout to wait between two consecutive requests.

JMeter Test Plans

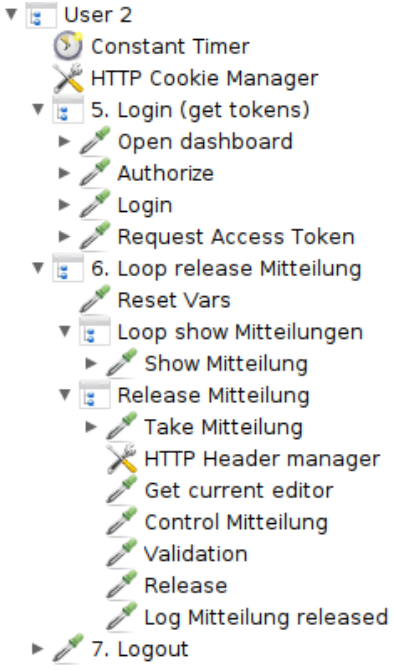
We provide two test plans: One to increase the size of attached files and another to increase the rate of created messages. The parameterization of JMeter to generate a specific amount of messages is not trivial and depends on the frequency in which JMeter sends HTTP requests. We cannot predict the exact number of required requests accurately at all times. This is caused by asynchronous communication between the dashboard and the backend of the PIs. It can take a considerable amount of time before a message is displayed and details queryable. The two provided test plans use different approaches to address this problem.

1. **Test plan 1: Increase the size of attached files** This test plan specifies the timeouts between two consecutive requests explicitly in milliseconds. This is possible for two reasons: First, the test plan to increase the size of attached files produces exactly one

[3] https://jmeter.apache.org/usermanual/component_reference.html



(a) Test plan for user 1 to create messages on the PIs.



(b) Test plan for user 2 to send created messages to the ZI.

Figure 5.3. Parts of the JMeter test plan. This example shows the test plan to create high request rates. In each iteration user 1 creates multiple messages (*Loop Creation*). After this, user 2 releases all created messages and sends them to the ZI (*Loop release Mitteilung*). To send each message to the ZI, user 2 queries the PI multiple times to obtain required information (*Loop show Mitteilungen*). On top of these two parts a constant throughput timer ensures that the HTTP requests are created with a constant frequency. The numbering matches graphic Figure 4.2.

message per user per PI per minute. Second, user 2 does not query the dashboard to get the message ID but uses the returned message ID from the creation process of user 1. It means user 2 skipped the loop in step 2 of Figure 5.2a and waits 25 seconds before assigning themselves as an editor. For technical reasons using the message ID returned to user 1 is possible in this test plan for user 2 since only one message (ID) is handled in each iteration. Therefore, no *constant throughput timer* is used to control the request rate of the whole test plan but multiple *constant timers* are used to set timeouts explicit between two requests.

By multiplying the number of identical thread groups in each test plan it is possible to increase the number of parallel acting users per JMeter instance and consequently, the frequency of generating new messages. Therefore, it is uncomplicated to create

new versions of this test plan with different frequencies, but each test plan has a fixed frequency.

A *limitation* of this test plan is that the simulated user behavior of user 2 differs from a realistic user interaction since the message ID is not taken from the dashboard. However, we evaluate the ability of the ZI to handle incoming messages and not primarily the process of the creation of messages, therefore we assume that this does not affect the scalability evaluation presented in Chapter 6.

2. **Test plan 2: Increase the rate of creations** Each login and logout process needs communication with Keycloak which can slow down the PLS by thousands of logins and logouts per minute. To reduce this communication user 1 produces $w \in \mathbf{N}$ messages in each iteration and adds a unique *reference ID* to each message to make them identifiable. User 1 creates these references through a pattern that can be reproduced by user 2. To control and release these messages user 2 needs the ID of the messages.

   To get the IDs of created messages user 2 calculates the *reference IDs* and queries the asynchronous dashboard of a PI to get the corresponding message ID. Due to the asynchronous nature of the dashboard, user 2 requests the PI periodically until the message matching the *reference ID* is available (illustrated in Figure 5.2a). User 2 aborts if the message ID is not available after 10 requests. The exact required waiting time depends on the utilization of the PI and cannot be predicted which leads to a varying number of necessary requests.

   To increase the rate of new messages the time between two requests must be decreased. JMeter allows only to change the request rate per minute. Therefore, if this number of requests per minute is increase to create more messages, JMeter sets the timeouts between requests accordingly. Since the exact number of required requests varies it is difficult to determine the required number of requests per minute in general.

   We set the number of requests per minute to the value in which we expect that a JMeter instance creates exactly w messages per minute (msgs/min) per PI, assuming that three requests are required to obtain the message ID. The process of generating w messages is called an *application* of this test plan. The parameter apm $\in \mathbf{N}$ (applications per minute) specifies the number of applications of this test plan to be performed per minute. Therefore, increasing the parameter apm by one, increases the number of msgs/min which creates a JMeter instance on a single PI by w messages.

   A *limitation* is that high request rates may require more than three requests because of potentially high utilization of a PI and the decreasing timeouts with increasing message rates. Therefore, JMeter may not create the desired number of messages. However, the arrival ratio metric does not consider the expected rate of generated messages but the actual generated rate. Therefore, we assume that this limitation does not affect our results.

5.3.3 Load Dimensions

To increase the load, we need to increase either the message rate or the size of messages. In this section, we present the implementation of the load dimensions described in Section 4.3.

Notation We introduce the notation shown in Equation 5.1 to describe the orchestration and involved components. Figure 5.4 illustrates the deployment graphically.

$$x = \text{number of PIs} \tag{5.1a}$$
$$y = \text{number of PIs per dedicated PostgreSQL instance} \tag{5.1b}$$
$$z = \lceil \frac{x}{y} \rceil \text{ is the number of required PostgreSQL instances} \tag{5.1c}$$
$$k = \text{maximum number of PIs on which JMeter create load simultaneously} \tag{5.1d}$$
$$c = \lceil \frac{x}{k} \rceil \text{ number of required JMeter components} \tag{5.1e}$$
$$m = \text{number of JMeter replicas per JMeter component} \tag{5.1f}$$

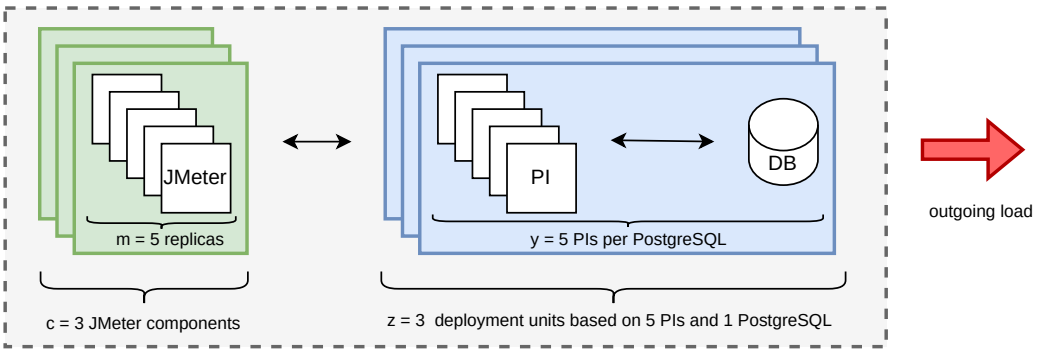


Figure 5.4. This example shows the configuration of $x = 15$ PIs, $y = 5$ PIs per PostgreSQL instance, the number of PIs which can be loaded parallel by JMeter is $k = 5$, the number of JMeter replicas is set to $m = 5$. The green boxes show $c = \frac{5}{15} = 3$ JMeter components. A JMeter deployment consist of $m = 5$ replicas represented by the white boxes inside the green boxes in each JMeter component. Each blue box shows a combination of PIs accessing a single PostgreSQL instance. In this case, $z = \frac{5}{15} = 3$ PostgreSQL instances are required. A blue box shows a deployment unit containing y PIs and a single PostgreSQL. Increasing the number of PIs can be done by multiplying the deployment units (illustrated by the blue boxes). For $x = 15$ PIs, $z = 3$ dedicated databases and consequently three deployment units needed. The overall composition determines the total generated load on the target system. The red arrow symbolizes the outgoing load.

Load Dimension 1a: Increasing the Number of PIs

We focus on the replication of PIs which communicate simultaneously with the ZI. This can be used either to increase the number of PIs on which JMeter create load in parallel or to investigate the number of PIs which can be handled by a ZI.

The deployment of a PI instance consists of multiple parts such as network components, a database, an individual Keycloak configuration, and the application itself (described in Section 5.5.2). The creation of a new PI instance needs the modification, configuration and replication of all these parts. Therefore it is not sufficient to increase the number of replicas in Kubernetes. Simple replication through the number of replicas of the PI deployment would affect errors since identical databases and Keycloak configurations are used.

In previous versions of Theodolite the term *scaling* referred to modify one or more resource manifests to change the deployment of the SUT. In order to scale PIs, it is necessary to multiply and modify the individual resources to create completly independent instances. This means cloning all manifests and changing all labels, names, and configurations to an instance-specific value. Therefore we replace all labels `label-value` with `label-value-`x_i, where x_i represents an individual instance with $i \in \{1, ..., x\}$ (x is the total number of PIs, as defined above). Since each PI is connected to its own database, we need to ensure that the database instance can handle all incoming queries from the PIs. For this purpose, a separate PostgreSQL database is created for each y PIs. Additionally, the configuration of a PI instance needs to be customized to request the correct database and use the new Services for network interactions. We develop new patchers to create new instances as described below. Also, Keycloak needs to be configured individually for each instance. For this, Theodolite uses a *Keycloak configurator* that configures Keycloak to manage the required number of PIs.

Dimension 1b: Increasing the Message Rate

We explain how the request rate can be increased for a fixed number of PIs. Therefore, we increase either the frequencies in which simulated user interact with the PIs, the number of users, or the number of JMeter instances which run the test plan in parallel.

Theoretically, it is possible to increase the generated load by a single JMeter instance to an arbitrarily high value by increasing the number of parallel acting thread groups or by increasing the frequencies. Nevertheless, JMeter cannot yield a very high workload for a long time. A reason for this is the memory consumption of JMeter. In our experiments, one instance requires between 70 and 90 GB memory at high load rates. The high memory usage is based on the fact that nearly all HTTP requests have inter-request dependencies. As a result, JMeter must correlate these queries by extracting the values from the different queries and inserting them into others. Providing less memory increases the likelihood of memory-related problems such as *out of memory* [Maxwell et al. 2010] errors.

For that reason, we decided to use multiple JMeter instances to generate the required amount of load. The typical way to increase the number of instances with JMeter is to

run JMeter in master/slave mode [Apache Software Foundation 2021; Matam and Jain 2017]. To make the individual JMeter instances more configurable (e.g., by setting different environment variables on each slave) we decide to adjust the number of JMeter instances (e.g., replicas of the Deployment in Kubernetes) via Theodolite. The master instance would provide basic metrics about the test running on the slaves. However, the introduced monitoring stack provides all metrics to evaluate an experiment. Therefore, the aggregated metrics by JMeter are not necessary. Required metrics are introduced in Section 5.7.

As described above, we provide a test plan to increase the frequencies in which users interact with JMeter. The number of PIs on which JMeter can parallelly create load is limited (e.g., too much memory is required). Accordingly, we develop additional patchers to replicate and configure the JMeter deployment so that each JMeter instance only interacts with a certain number of PIs (specified via the variable k). The replications of JMeter instances increase the number of parallel acting users.

Dimension 2: Increasing the Size of Messages

Increasing the size of attached files is based on the test plan described in Section 5.3.2. Technically, the deployment of JMeter needs to mount an additional volume in Kubernetes to access the test files which should be added as attachments. We take an image of the b+m cooperate logo to create the test data. This image is replicated multiple times and transformed into a PDF file until the file size reaches a specific value (sizes are between 1 MB and 20 MB in steps of 0.5 MB). This work does not investigate multiple attachments since b+m expects similar behavior of the PLS for multiple small and one large file.

5.4 Monitoring for the PLS Benchmark

Detailed monitoring helps to identify bottlenecks in software systems as well as collects and provides the necessary metrics for benchmarking [Bermbach et al. 2017]. This benchmark uses the observability-enhanced PLS resulting from G1 (introduced in Chapter 3). The control unit Theodolite supports natively the integration of Prometheus to collect measurements. Since Loki's HTTP API and data format are consistent with Prometheus no additional implementations are required to use Loki metric queries. Therefore, Theodolite uses PromQL and LoqQL to query both APIs and perform calculations and aggregations on the collected data to obtain metrics.

5.5 Execution and Experiment Control

This section addresses the execution of the described implementation. For that, we explain the required setup in Kubernetes and the experiment control unit, ensuring technical reproducibility [Papadopoulos et al. 2021] and simplifying usability.

5.5.1 The Benchmarking Tool Theodolite

The configuration of Theodolite's operator is based on a benchmark description and a specification to execute the benchmark. Listing 2.1 and Listing 2.2 show simplified specifications of the PLS benchmark. Motivated by goal 3 several modifications are needed on the code base of Theodolite to enable the execution of the PLS benchmark.

Implementation of addional patchers for Theodolite The modular architecture of Theodolite allows the integration of new patchers. The concept of patchers allows to modify Kubernetes manifests (see Section 2.6). These patchers are needed to allow the replications of the PI which is required to increase the workload (described in Section 5.3) and to modify the JMeter deployment in Kubernetes. Since each patcher provides a specific technical function, we list all developed patchers. The first three patchers are specific for this benchmark to handle the PLS and JMeter. The other patchers can be used in generic contexts.

1. **JMeterReplicaPatcher** The number of PIs that a JMeter instance can request simultaneously is limited. This patcher can create new JMeter instances and configure them in a way that the maximum number of PIs requested per JMeter instance do not exceed the limit but all PIs are loaded identically.

2. **PLSPiPostgresPatcher** This patcher uses several other patchers and creates a PostgreSQL configuration for each individual PI. A separate PostgreSQL instance is created for each y PIs, i.e. each PostgreSQL instance provides y databases.

3. **PLSReplicaPatcher** This patcher uses several other patchers and can create a new PI instance that is decoupled from all other PI instances. Technically, this patcher duplicates the resources and changes all values to avoid associations with resources used by other PIs. Depending on the *Kind* (e.g., type) of the given resource, this patcher modifies different values. This patcher requires additional implementation since, so far, Theodolite only allows changing a manifest without replicating it.

4. **AnnotationPatcher** This patcher adds annotations to the `spec.template` level. This can be used to exclude specific resources from the scope of the OSM.

5. **ConfigMapPropertiesFilePatcher** This patcher modifies the values of an `application.properties` file within a ConfigMap. This patcher is used by the *PLSReplicaPatcher*.

6. **ImagePullPolicyPatcher** This patcher changes the *ImagePullPolicy* of a container.

7. **MatchLabelPatcher** This patcher adds match labels on the `spec.template` level. This sets match labels on the Pod template.

8. **NodeAffinityPatcher** This patcher changes the (weighted) *NodeAffinity*. Setting node affinities increase the likelihood of Pods deployed on a specific node.

9. **PodAffinityPatcher** This patcher changes the *PodAffinity*. It increases the probability to schedule multiple Pods on the same nodes. It makes it possible to increase reproducibility of scheduling and deploy multiple Pods together.
10. **ServiceSelectorPatcher** This patcher changes the labels which a Service uses as selectors. This changes the associated resource for which the Service is responsible.
11. **TemplateLabelPatcher** This patcher adds labels on the `spec.template` level. This sets labels on the Pod template.
12. **VolumesConfigMapPatcher** In Kubernetes, Pods can mount different volumes to read data and configurations. For example, ConfigMaps provide configurations for a container inside a Pod. This patcher is used by the *PLSReplicaPatcher*. This patcher can change the name of a ConfigMap to be mounted.

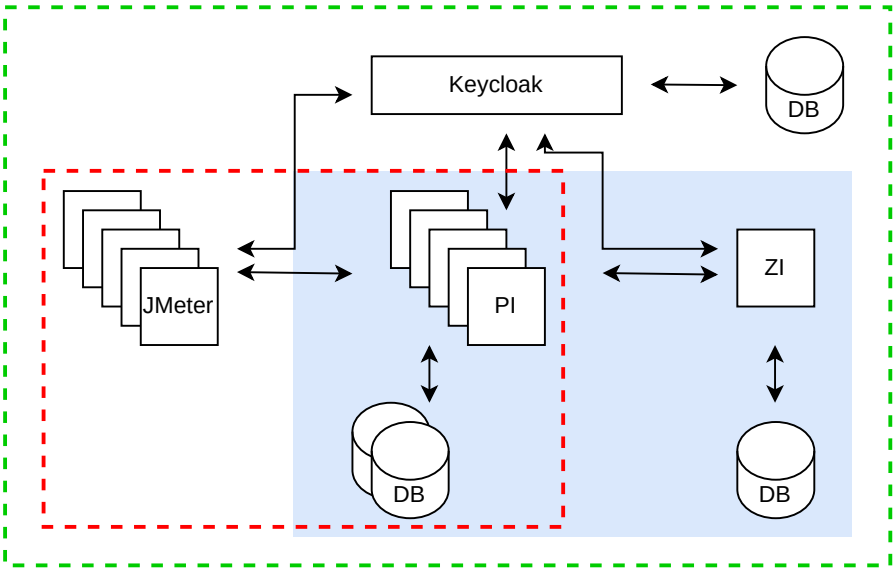


Figure 5.5. High-level overview of components of the SUT and the load generator without components to configure, control, analyze, or monitor an experiment. The blue part shows all components of the PLS itself. The red part shows the components required to generate load on the ZI. On top, the required infrastructure component Keycloak provides security features between all components. The green part shows that all components are in the scope of the service mesh.

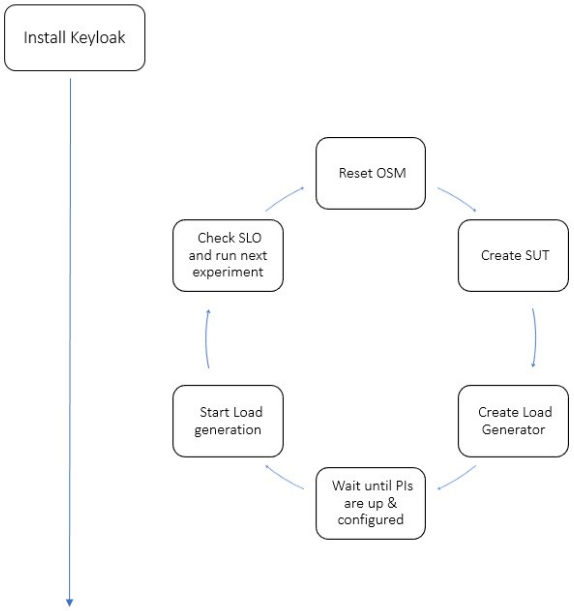


Figure 5.6. The cycle illustrates the steps that are performed for each SLO experiment. To avoid network problems associated with OSM the mesh is restarted for each SLO experiment.

Integration The combination of these patchers allow the replication of PI instances. For example, let the number of PIs be $x = 5$, deploying maximally $y = 4$ PIs per PostgreSQL instance, results in $z = \lceil \frac{5}{4} \rceil = 2$ required PostgreSQL instances. For this, the *PLSPiPostgresPatcher* creates two dedicated database instances including individual ConfigMaps and Services. Each PostgreSQL instance provides exactly $y = 4$ databases. The *PLSReplicaPatcher* creates five absolutely decoupled versions of the PI. To provide an individual configuration for each PI, each PI mounts a different ConfigMap.[4] Inside these ConfigMaps the values are adjusted to match exactly one specific database. The correct Service and database must be selected to avoid conflicts. This example shows only some aspects of duplicating PI instances. Further, an individual Keycloak configuration is required. The configuration of Keycloak is processed in multiple steps and needs to match the configuration of different components. Another aspect is the scheduling in Kubernetes. Both affinity patchers are used to decrease scheduling variability between associated PostgreSQL and PI instances. Setting affinities also requires the correlation between different values on each Deployment.

[4]The Kubernetes *replica* mechanism allows only to increase the number of instances but without changing the configuration of different instances.

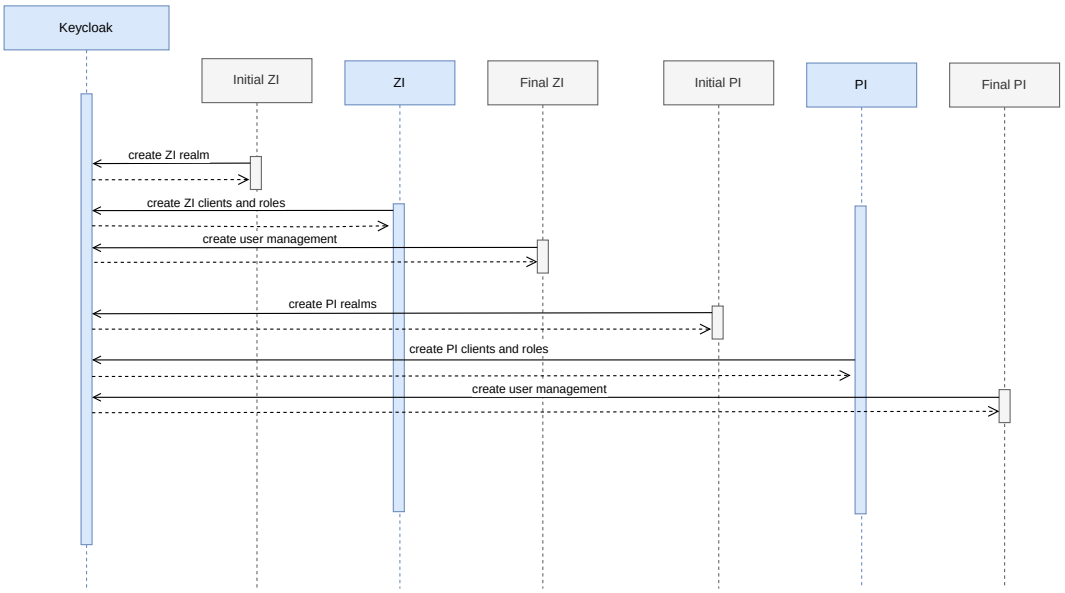


Figure 5.7. Startup sequence of the PLS focusing on the configuration of Keycloak. In blue the components of the PLS, in gray the parts that are performed by configurator Pods (*Initial ZI/PI* means creating the initial configuration of Keycloak for the ZI/PI and *Final ZI/PI* means finalize the configuration).

Deployment and execution process The components of the PLS depend on each other and require a specific start sequence. We add an init container to the individual Pods to control the start sequence. Each init container checks the state of the Pod which must be up and running before the own Pod can start. Therefore, the init container periodically requests a specific address of the dependent container and blocks until the response has the status code 200. For example, we use the address of the *Readiness-Probes* [Cloud Native Computing Foundation 2022a] to identify whether Keycloak, the PIs, or the ZI are ready. We create configurator Pods based on the Keycloak tool (described in Section 5.2) to perform the configuration. Figure 5.7 shows this configuration and startup process. The load generator and the experiment timer start only if the whole SUT runs correctly. To reduce the configuration and start-up time we configure Keycloak only once per execution. Afterwards, an additional state-checker Pod indicates whether all components are up and running before the SLO experiments start (illustrated in Figure 5.6).

5.5.2 The Kubernetes Setup

In this subsection, we explain the setup of the different parts in Kubernetes. Figure 5.5 shows all involved components and their relationships.

Setup the benchmarking control unit The execution, evaluation, and sharing of benchmarks is an important and complex task [Boden et al. 2018]. For that, we use the Theodolite operator, including the modification discussed above. The installation of Theodolite also includes Prometheus and Grafana. Section 2.6 describes the Theodolite operator in detail.

Setup the required infrastructure components On top of the provided infrastructure components by Theodolite, the monitoring tooling introduced in Chapter 3 is required.

Setup the SUT and the load generator The following components are required for the SUT and the load generation.

1. **PI and ZI** The Kubernetes deployments of the PI and the ZI consist of a Deployment manifest for PI or ZI, respectively. Multiple Services, Secrets, and ConfigMaps exist for configuration and networking.
2. **Keycloak** The Kubernetes deployment of the identity provider Keycloak consists of the Keycloak Deployment, a PostgreSQL database as storage, and Services for inter-cluster communication. Additionally, there exist four *Jobs* of a Keycloak configurator tool to configure Keycloak for each instance of the ZI or the PI. For this, the configurator creates realms, groups, and users and manages the permissions.
3. **JMeter** The JMeter deployment consists of the Deployment manifest and a ConfigMap for the test plan. Further, test files need to be provided through a volume. We use five manifest files, each with a different initial delay.

5.6 Measurement of Data and Offline Analysis

Theodolite measures the experiments based on our previous discussed observability-enhancements of the PLS. Theodolite analyzes each experiment within an online analysis to determine the following experiment. A (manual) offline analysis can be used to explore certain aspects after the execution. Therefore, Theodolite stores the required measurements for offline analysis. Saving fine-grained results could be essential to analyze and compare new aspects after executing the benchmark [Bermbach et al. 2017]. Theodolite also provides tools to support the offline analysis. Based on existing components for offline analysis we provide a customized *Jupyter Notebooks*[5] to evaluate and visualize different aspects of executed experiments.

[5] https://jupyter.org/

Table 5.1. Standardized definitions of specified SLIs (Aggr)

SLI	Aggr. intervals	Aggr. regions	Scrape freq.	Included data	Data acquisition
Ratio between received and created	1min rate	all JMeter instances and the ZI	3sec	logs, which indicates successfully created message of JMeter and received messages by the ZI	Loki, Promtail
Latency (90th percentile)	1min rate	between PIs and ZI	10sec	HTTP requests between PIs and ZI	OSM, Prometheus
Send errors	1min rate	between PIs and ZI	10sec	HTTP requests between PIs and ZI, with an error indicating response	OSM, Prometheus
Create errors	1min rate	between JMeter and PIs	10sec	HTTP requests between JMeter and PI, with an error indicating response	OSM, Prometheus

5.7 The SLO and Required Metrics

As described in the previous chapters the SLO is based on multiple sub-SLOs. Each SLO specifies a range of target values of a SLI to evaluate whether the SLO is met. As suggested by Beyer [2016], Table 5.1 defines all SLIs in a standardized way to increase the reproducibility. The SLO and the target values for these SLIs are described in Table 4.1. Therefore, the SLIs are constructed using the metrics described below. Appendix A shows the corresponding PromQL and LogQL queries.

1. **The arrival ratio metric** We configure JMeter to create a log message for each created message. This log message contains information about the message ID and whether the creation was successful. The response code of the HTTP request by which user 2 released the message, is evaluated to determine whether the creation of the message was successful. Listing 5.1 shows exemplary a log message. Logs that indicate errors are ignored. In addition, the ZI writes a log message for each received message. We use Loki and LogQL to calculate the per minute rate of the ratio between created and received messages.

2. **Latency (90th percentile)** The latency of all HTTP requests between the PIs and the ZI is considered. OSM provides this metric. Prometheus provides a function to calculate quantiles of measurements.[6] This function is used to calculate the 90th percentile of the latency.

3. **Error (sending, creation)** Every HTTP request that receives an error response code is taken into account. I.e., all requests whose response code is not prefixed by 2 or 3 (i.e., *2XX* or *3XX*). The per minute rate of such HTTP requests is calculated and considered

[6] `https://prometheus.io/docs/practices/histograms/#quantiles`

as an error metric. This metric is calculated twice: Once for the communication between JMeter and the PIs (create errors) and once for the communication between the PIs and ZI (send errors).

Listing 5.1. JMeter log indicating successfully created message

```
1 Mitteilung freigegeben: true, id: d44e52cc-14fc-41c1-b36e-e113e750dd2b
```

Chapter 6

Experimental Evaluation

In this chapter, we mainly address the third phase of benchmarking and present the execution of the PLS benchmark and the scalability evaluation of the PLS, as depicted in Figure 1.1. Our research question is *"How scalable is the PLS?"*. We divide this research question into three evaluation questions to aim for goal 4.

- **EQ 1** *How does the PLS behave under load, and which parameterizations are appropriate for this evaluation?*

  We use an exploratory data analysis (EDA) to address this question. Tukey [1977] introduces this method and describes different procedures to analyze data. These procedures are used for the first data investigation from many points of view to detect unknown patterns and characteristics or to test assumptions [Tukey 1977; Morgenthaler 2009]. This data analysis can be done graphically. This includes various techniques and computational methods such as box plots or the visualization of meaningful values such as the average, median, or quantiles [Morgenthaler 2009]. The use of quantiles is more resistant against outliers and anomalies in relation to average values [Kounev et al. 2021; Bermbach et al. 2017].

  In the domain of (cloud) benchmarking, exploratory data analysis can be used to lead to a better understanding of how applications react to new workloads and what (potential) problems may arise [Bermbach et al. 2017]. In this context, the repeatability of the EDA is of particular importance. After each iteration and new findings an adjusted benchmark configuration is executed, focusing on other aspects [Bermbach et al. 2017].

  We perform an exploratory pre-study to improve our understanding of the behavior of the PLS under high load.

- **EQ 2** *To which degree does the ZI scale with increasing attachment's file size?*

  Based on the results of the pre-study we design experiments to examine the ability of the ZI to scale with the size of messages. Therefore, we attach files to messages. Section 6.3 shows this with systematic and statistically grounded experiments.

- **EQ 3** *To which degree does the ZI scale with increasing rates of created messages?*

  We address benchmarking of high request rates in Section 6.4 and perform systematic and statistically grounded experiments. Such experiments are required on top of an EDA to obtain recoverable results [Bermbach et al. 2017].

The remainder of this chapter is structured as follows: We describe the used methodology in the first section. Afterwards, we present in Section 6.2, Section 6.3, and Section 6.4 the scalability analysis. We address the threats to validity in Section 6.5, and finally, we conclude this evaluation in Section 6.6.

6.1 Methodology

6.1.1 Notation and Structure

We divide this evaluation into two benchmark configurations, each configuration is executed in multiple scenarios. We denote that as $B_{\text{configuration}}E_{\text{scenario}}$. The configuration describes the load dimensions and parts which are identical in all executions. The execution describes details about the examined scenario. The concrete benchmark configurations are B_{size} (load dimension is the increasing size of attachments) and B_{rate} (load dimension is the increasing frequency in which messages are created). This is motivated by the technical fact that Theodolite requires two different Benchmark configurations to execute different load dimensions of the PLS benchmark. Table 6.1 shows all configurations and scenarios.

6.1.2 Execution Environment

We execute our experiments on a five-node Kubernetes (v.1.18.6) cluster.[1] Each node has 348 GB RAM and 2 $\times$ 16 CPU cores. The nodes are connected with 10 Gbit/s Ethernet.

6.1.3 Benchmark Setup

The execution of the PLS benchmark implementation requires the selection of a load dimension and a fitting parameterization. We use identical resource limits and requests for all experiments of B_{size} and B_{rate}, as shown in Table 6.2. Only the number of CPU cores of the ZI will be changed since it is the resource dimension to be benchmarked.[2] We use the following software versions: Keycloak v15.1.1 (JBoss), PostgreSQL v12.4, OSM v0.9.2, JMeter v5.4.3, Theodolite v0.7.0, and PLS v2.0.

6.1.4 Handling Multidimensional Load

As discussed in Chapter 4 and Chapter 5 we can specify the load by the combination of the number of PIs, the frequency in which a user interacts with a PI, and by the size of attached files. The focus of this evaluation lies on benchmarking with exact one load dimension since Theodolite does not support multidimensional loads. For that reason, we set within each benchmark configuration exactly one dimension as variable and all others as fixed.

[1] www.se.informatik.uni-kiel.de/en/research/software-performance-engineering-lab-spel/

[2] We reduce memory in two executions to study the impact on scalability.

Table 6.1. Benchmark configurations and scenarios examined in this evaluation; default values are used unless otherwise noted.

Benchmark configuration	Execution scenario
B_{size} (attached files between 1 MB and 20 MB)	100 msgs/min 200 msgs/min 400 msgs/min 100 msgs/min (different resources for the same load) 100 msgs/min (decreased memory)
B_{rate} (500 msgs/min - 5000 msgs/min)	increasing rates increasing rates (decreased memory)

6.1.5 Coordination of Distributed Load Generation

As indicated by Bermbach et al. [2017], distributed load generation requires coordination. Otherwise, the combined load may consist of multiple peaks that drop and do not have a constant frequency. In this work, the load generation is distributed when multiple JMeter instances create load parallelly on the same PI. This is the case for $m > 1$ JMeter instances (notation is introduced in Section 5.3.3). We use $m = 5$ JMeter instances in all experiments to generate constant and high load. The five instances start successively[3] within one minute with a delay of 12 seconds. The objective of consecutive starts is to obtain a constant request rate per minute.

6.2 Explorative Pre-Study

The goal of this pre-study is to provide initial insights for planning further experiments. This section provides high-level information about which load configurations cause which behavior of the ZI. This pre-study identifies and reveals different issues and explains how we address these issues in the further evaluation.

Table 6.2. Used resource requests and limits in Kubernetes.

Resource Name	CPU Request/Limit (Cores)	Memory Request/Limit (GB)
ZI Deployment	scaling factor	10
PI Deployment	3.5	4
Keycloak	8	15
PostgreSQL (ZI)	6	15
PostgreSQL (PI)	4.5	10
PostgreSQL (Keycloak)	2	10
JMeter	0.5	60

[3] Implemented through five deployments and init containers.

6.2.1 Experiment Duration

We combine short- and long-term experiments in this pre-study. We use short-term experiments to find relevant scenarios. Due to the shortness a risk is created that certain interesting system behavior will not be recognized. For example, it is possible that a SUT can handle a certain amount of messages for a short-time but has problems processing this amount over a longer period. In contrast, extremely long executions are unpractical. However, the PLS should run for a long time (months, years), therefore it is necessary to consider periods of more than 10 hours.

Performing long experiments has two objectives. The first objective is to verify that JMeter can generate a high load over a long period of time. The second objective is to examine the system for anomalies while it is stressed for an extended period of time.

6.2.2 Generating Load for a Long Period

We identify that constantly creating high loads for long periods of time requires a large memory but low CPU resources. The use of small memory limits leads to varying load curves. Therefore, the memory limits are set to 60 GB per instance. We use in this evaluation $c = 1$ JMeter components with $m = 5$ replicas, therefore 300 GB memory are required for JMeter in total (see Section 5.3.3 for the notation).

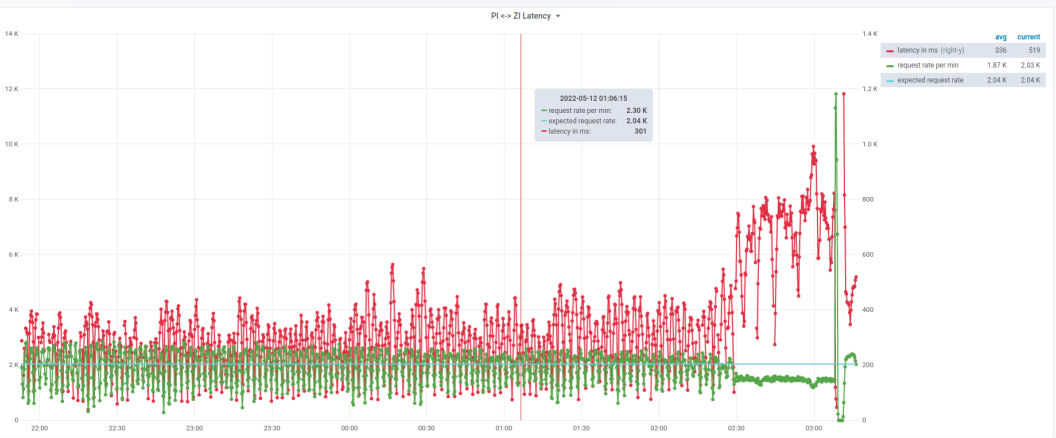


Figure 6.1. A screenshot of the Grafana dashboard illustrating the rate of HTTP requests between PIs and ZI (green) in contrast to the average[a] latency in ms (red). The expected rate of HTTP requests is shown in blue. The figure shows the measurements before an out of memory caused restart of the ZI.

[a] Since this experiment was conducted at a very early stage of this work, the average and not the 90th percentile is shown.

6.2.3 Memory Leak in PI and ZI

In the first benchmarks, we figured out that the ZI and the PIs restart automatically after five to ten hours under a load of thousands of messages per minute (msgs/min). The time the system is running *correctly*, i.e., without periodic restarts, depends on the generated load. The behavior of the ZI was very similar before each restart: The latency between PIs and ZI suddenly raised, and the number of HTTP requests between both components dropped (as illustrated in Figure 6.1). Based on this finding, we analyzed the memory consumption in the following experiments and found that the allocated memory consumption always increases (as shown in Figure 6.2). To identify the underlying cause, we periodically created *heap dumps* [Oracle 2022]. The analysis of the heap dumps by b+m revealed a memory leak and that the connections between the components established too many sessions on the ZI. We assume that this was not recognized at an early stage since such high loads were examined for the first time.

Memory leaks are programming errors causing programs to not release unused memory. Memory leaks can significantly slow down a system. In the worst case, a memory leak can cause a system to crash because no more memory is available [Maxwell et al. 2010]. Memory leaks are one of the most common problems in server systems which run for a long time. Also, they are difficult to detect and to identify the cause of the bug [Maxwell et al. 2010].

Since both findings can significantly affect the performance of the software – especially in terms of availability – we use a patched version for the following experiments on the PLS. We could not detect the increasing memory problem (memory leak) or periodic restarts on the patched version, as in the previously benchmarked version. Additionally, we see that the PLS benchmark is able to investigate different versions of the same software.

6.2.4 Identification of Keycloak as a Bottleneck

We identify that Keycloak can slow down the whole PLS. The reason is that Keycloak secures the communication between JMeter, PI, and the ZI, which causes many HTTP requests. Therefore, a high workload and associated high latency in communication with Keycloak can slow down the system. We found out about this when creating high loads

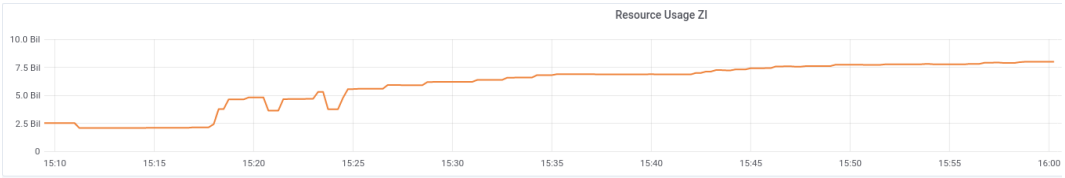


Figure 6.2. Memory consumption of the ZI for 8 GB memory limits in Kubernetes (Grafana screenshot).

with 10 JMeter instances for short durations. Two users have to log in and out to create each individual message, therefore high loads and thousands of messages result in thousands of logins and logouts per minute. Also, the PIs and the ZI communicate with Keycloak which leads to additional requests. Section 2.7.1 describes the communication protocols with Keycloak in detail. We assume that this high load has not been investigated on the PLS so far and therefore did not cause a problem previously.

To address this issue, we modified the test plan to create several (e.g., 5) messages per user session, reducing communication with Keycloak significantly. We also set the Keycloak limits in Kubernetes to a high value. Table 6.2 shows the resource requests and limits. Additionally, the patched versions of PLS reduces communication with Keycloak.

6.2.5 Reproducibility of the Kubernetes Deployment

From the first experiments we conclude different results for identical configurations. As reported by Papadopoulos et al. [2021], identifying the source of variability is important, and reproducibility is hard to achieve in cloud computing.

One source of variability is scheduling. We use the concept of *affinities* in Kubernetes [Cloud Native Computing Foundation 2022a] to address this issue and make scheduling as reproducible as possible. We realize this with two additional patchers, namely the *PodAffinityPatcher* and the *NodeAffinityPatcher* (see Section 5.5.1). The PodAffinityPatcher increases the likelihood that related Pods will be deployed on the same nodes and the NodeAffinityPatcher that Pods will be scheduled on nodes with particular characteristics. Nonetheless, a second source of variation is that Kubernetes was not able to provide the Pods with the resources specified as *limits*. This can occur if the requests are less than the specified limits and the individual node can only provide the specified requests but contains not enough resources for the limits. We solved this is problem by setting limits=requests. Explicitly specifying this resource provisioning avoids overusing node resources, as Kubernetes schedules Pods only on nodes with enough resources to provide the requests.

From this, we derive the following configuration for the entire evaluation to solve both problems. Explicitly specifying requests=limits restricts us to the following configuration in our execution environment. Table 6.2 shows the requests and limits for all resources to avoid the over-provisioning of nodes. We use $x = 20$ PIs, placing $y = 4$ on each dedicated database, so $z = 5$ PostgreSQL instances are required (see Section 5.3.3 for the notation). The execution environment consists of five nodes. We integrate both patchers to schedule each database on a different node and the PIs on the same nodes as the associated database. Further, all JMeter instances are deployed on the same node in all experiments. Also, the ZI is scheduled for all experiments on the same node. Only Keycloak and the configurator Pods are deployed without preferences.

6.2.6 Repetitions

The number of repetitions is a crucial factor for the statistical significance of results but also for the practicability of experiments. If the number is too low, the experiments loose their significance. However, too many repetitions reduce practicability as the execution takes too much time. A result of our pre-study is that the variance in five repetitions is low in respect to the arrival ratio metric in cases in which the ZI is over-provisioned. This can be seen in Figure 6.3a for low load values. For that reason, we perform each experiment five times.

6.3 Scaling the Size of Messages

In the following, we analyze the scalability of the ZI regarding the size of generated messages. To adjust the size of a message file attachments can be added. Therefore, exactly one PDF file of variable size is attached to each message. Specific file sizes allow setting the size of a message to a concrete value. We denote this analysis as benchmark configuration B_{size} since the load dimension is the *size* of the attached PDF files. The resource dimension is the number of provisioned CPU cores to the ZI. The number of PIs are fixed in this benchmark configuration, the request rate is fixed in each execution.

6.3.1 Setup

The overall experiment setup is identical for all scenarios (represented by executions). We describe the configurable parameters and corresponding notation to specify precisely the experiment setup (as described in Section 5.3.3). In benchmark configuration B_{size}, there are $x = 20$ PIs and $y = 4$ PIs per PostgreSQL instance. Consequently, $z = \lceil \frac{20}{4} \rceil = 5$ PostgreSQL instances are required. $k = 20$ is the number of PIs on which a JMeter instance is able to parallelly generate load. Therefore, c = $\lceil \frac{20}{20} \rceil = 1$ JMeter components are required. The number of JMeter replicas are set to $m = 5$.

In addition to these parameters, we introduce $n \in \mathbf{Z}$. n is the number of parallel acting thread groups running in each JMeter instance. Increasing the number of thread groups per JMeter instance increases the number of simultaneously requesting users per JMeter instance for each individual PI. Each thread group creates exactly one message per PI per minute. Consequently, for one thread group ($n = 1$) 100 messages per minute (msgs/min) are created in total. Let $l \in L$ be the size of attached files in megabyte.

The load generation is based on the first test plan introduced in Section 5.3.2. Equation 6.1 gives the total amount of send messages per minute. Equation 6.2 describes the amount of transmitted megabytes per minute. The ideal case is that the rate of the ZI receiving messages is equal to the rate of generated messages. In this case, Equation 6.1 and Equation 6.2 hold for both the communication between all JMeter instances and the PIs as well as for the communication between the PIs and the ZI.

$$\text{Total msgs/min} = m \times n \times x \tag{6.1}$$

$$\text{Total MB/min} = m \times n \times x \times l, \text{ for } l \in L_1 \tag{6.2}$$

The scaling parameter of the load dimension is the size of the attached files. We assume not only the size but also the frequency is an important parameter. Therefore, different frequencies are considered from a minimum of 100 msgs/min up to a maximum of 400 msgs/min. This is implemented by changing the number of users, simultaneously creating new messages (e.g., the number of thread groups). Different executions use different steps by increasing the size of the attached files since analyzing all sizes in each execution would lead to extremely long execution times.[4] Therefore, we select the sample sizes such that the probability of experiments yielding relevant results is assumed to be high.

6.3.2 Experiment Configuration

Each experiment is executed for 18 minutes with five repetitions. We ignore the measurements of the first 6 minutes and start the analysis after this warm-up period.

6.3.3 Execution of $B_{\text{size}}E_{\text{100 msgs/min}}$

In this execution we investigate the situation of 100 created msgs/min in total. Therefore, the number of thread groups is $n = 1$. As load, the size of added attachments is scaled with $l \in L_1 = \{1\,\text{MB}, 2\,\text{MB}, \ldots, 20\,\text{MB}\}$. Equation 6.3 and Equation 6.4 describe this scenario mathematically.

$$\text{Total msgs/min} = 5 \times 1 \times 20 = 100 \tag{6.3}$$

$$\text{Total MB/minute} = 5 \times 1 \times 20 \times l = 100 \times l, \text{ for } l \in L_1 \tag{6.4}$$

Results For the considered resource values, up to 16 cores, the ZI is able to handle a file size of maximally 18 MB by 100 created msgs/min (as shown in Figure 6.3a). For a file size of 18 MB, $5 \times 1 \times 20 \times l = 100 \times 18 = 1800$ MB in total are transmitted per minute and 16 cores are required. In contrast, for 17 MB only 2 cores are sufficient. Figure 6.3b presents that the latency increases strongly for file sizes larger than 13 MB. It shows also that for 18 MB the latency is very volatile. The percentage of the CPU utilization decreases while the number of provided CPU cores increase, which indicates that the CPU performance is not the limiting factor (as shown in Figure 6.3c).

In these figures, box plots[5] are used to display the arrival ratio metric, latency, and average CPU utilization for each experiment. It is important to note that the average CPU usage is calculated related to the specified CPU limits and that the 90th percentile of

[4] For example, $B_{\text{size}}E_{\text{100 msgs/min}}$ requires more than 47 hours execution time.

[5] The box spans from the first to the third quartiles of the data, with a median line. The whiskers extend the box's interquartile range by 1.5 times (IQR).

the latency is measured in milliseconds. The gray columns represent the outcomes of experiments conducted for every resource value (CPU cores). The white circles represent the mean of all performed repetitions of an experiment and the black points depict the mean arrival ratio for each repetition. The box plots are calculated using the mean values for each repetition.

6.3.4 Execution of $B_{\text{size}}E_{\text{200 msgs/min}}$

In this execution is $n = 2$, therefore in total we investigate the creation of 200 msgs/min. In order to reduce execution time we have decided to consider only a subset of possible MB sizes in this sub-study and investigate only attachments with even sizes (2 MB, 4 MB,...). $B_{\text{size}}E_{100\,\text{msgs/min}}$ has shown that up to 1800 MB can be handled successfully by the ZI. When doubling the frequency, we expect to be able to send only half the size of the file without violating the SLO. Hence, we decrease the steps between file sizes in the near of 9 MB. Therefore, the following load values are taken into account $L_2 = \{2\,\text{MB}, 4\,\text{MB}, \ldots, 20\,\text{MB}\} \cup \{8.5\,\text{MB}, 9\,\text{MB}, 9.5\,\text{MB}\}$. Equation 6.5 and Equation 6.6 describe this scenario mathematically.

$$\text{Total msgs/min} = 5 \times 2 \times 20 = 200 \tag{6.5}$$

$$\text{Total MB/min} = 5 \times 2 \times 20 \times l = 200 \times l, \text{ for } l \in L_2 \tag{6.6}$$

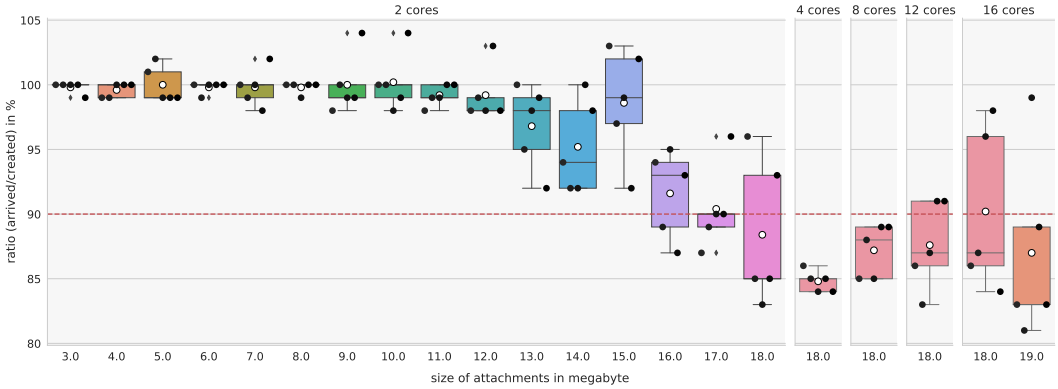
Results In this scenario, the ZI can process attachments up to 8.5 MB file size based on our SLO. As a result, we obtain $5 \times 2 \times 20 \times l = 200 \times 8.5 = 1700$ MB that can be received per minute. The latency decreases while adding more CPU cores but the average CPU utilization tends to be low. Figure 6.4 illustrates this by fundamental statistical analysis. It shows unexpected measurements for the SLO experiment characterized by $r = 16$ cores and $l = 9.0$ MB file sizes. The performance seems to be lower than for $r = 12$ cores, but we expect at least a stable performance.

6.3.5 Execution of $B_{\text{size}}E_{\text{400 msgs/min}}$

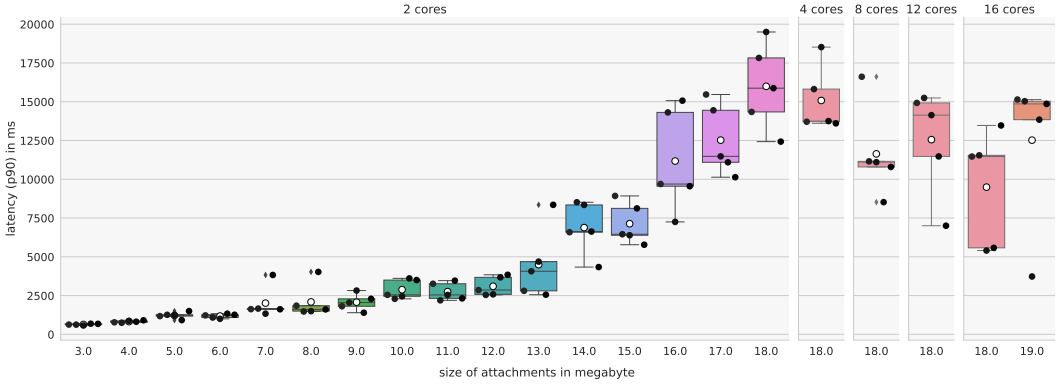
We investigate in this execution 400 msgs/min ($n = 4$). This is the highest request rate in this evaluation while considering the size of an attached file as load dimension (e.g., in B_{size}). Analoge to the previous execution, the set of load values L_3 considers potentially interesting file sizes. We set $L_3 = \{1\,\text{MB}, 2\,\text{MB}, \ldots, 20\,\text{MB}\} \cup \{3.5\,\text{MB}, 4.5\,\text{MB}, 5.5\,\text{MB}\}$ to increase the size of files in steps of 0.5 MB. Equation 6.7 and Equation 6.8 describe this scenario mathematically.

$$\text{Total messages/min} = 5 \times 4 \times 20 = 400 \tag{6.7}$$

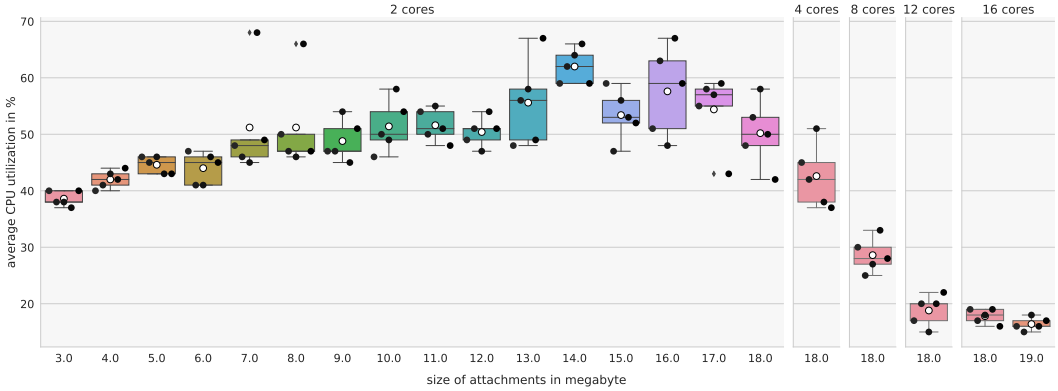
$$\text{Total MB/min} = 5 \times 4 \times 20 \times l = 400 \times l, \text{ for } l \in L_3 \tag{6.8}$$



(a) Arrival ratio metric. The red line displays the threshold of 90 %. The variance increases strongly for files greater than 13 MB.

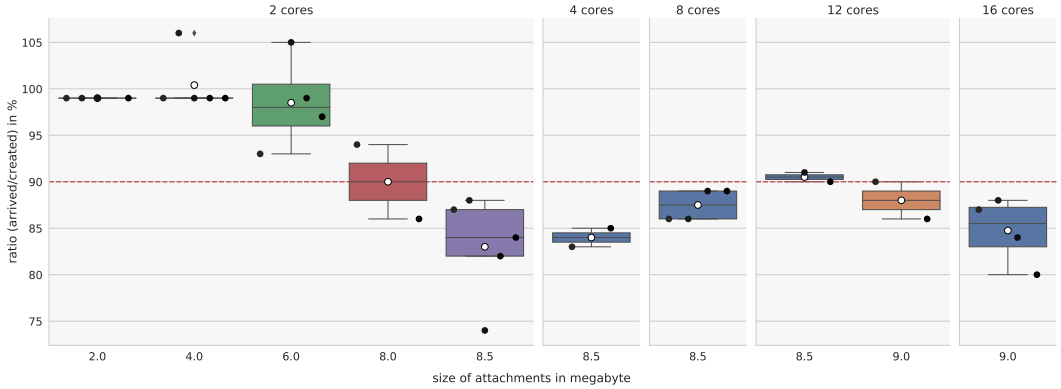


(b) Latency between PIs and ZI in ms (90th percentile).

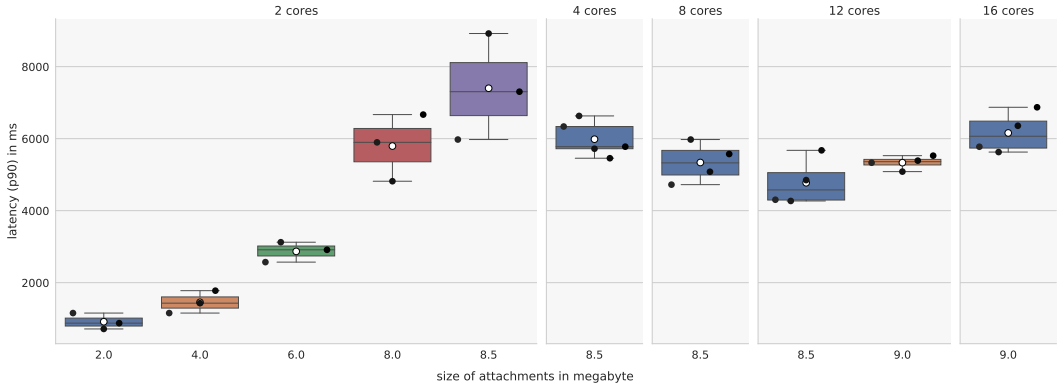


(c) The average percentage CPU utilization does not increase significantly by higher load values but decreases for higher provisioned CPU resources.

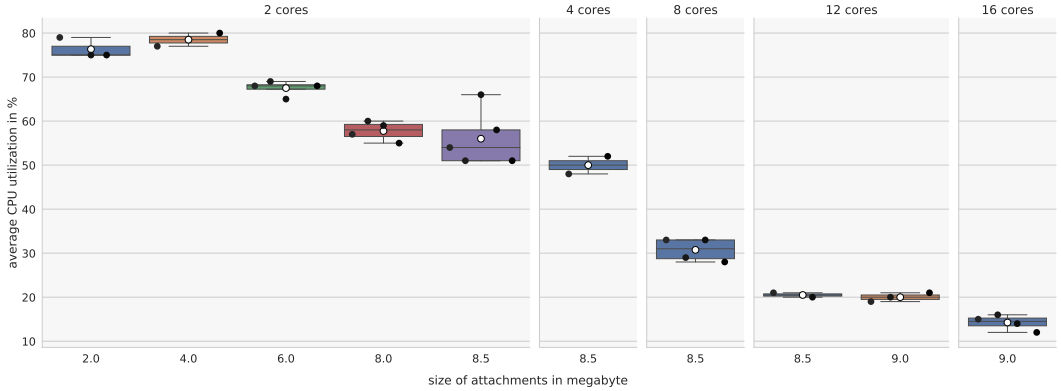
Figure 6.3. Illustration of $B_{\text{size}} E_{100\,\text{msgs/min}}$ it considers 100 msgs/min ($n = 1$). The figures show that the maximum manageable file size is 18 MB while the utilization of provisioned CPU cores is low.



(a) Arrival ratio metric. The red line displays the threshold of 90 %.



(b) Latency between PIs and ZI.



(c) The average percentage CPU utilization concerning the defined CPU limits decreases.

Figure 6.4. $B_{\text{size}}E_{200\,\text{msgs/min}}$ considers 200 msgs/min ($n = 2$). First, the figures show the expected behavior: Increasing performance by providing more CPU cores. However, the last column shows an unexpected behavior since the performance decreases for larger resource values (see Section 6.3.3 for an explanation of this figure).

Results With a request rate of 400 msgs/min the ZI is able to handle a maximum file size of 4.5 MB with the provisioned resources of maximally 16 cores. In total, these are, $5 \times 4 \times 20 \times l = 400 \times 4.5 = 1800$ MB. Figure 6.5a shows that the ZI is able to handle more messages by adding more resources, but 12 additional CPU cores are necessary to increase the processable file size from 3.5 MB (in total 1400 MB) up to 4.5 MB (in total 1800 MB). Figure 6.5 shows these relations with fundamental statistical analysis of the results.

6.3.6 Results of B_{size} for 100, 200, and 400 msgs/min

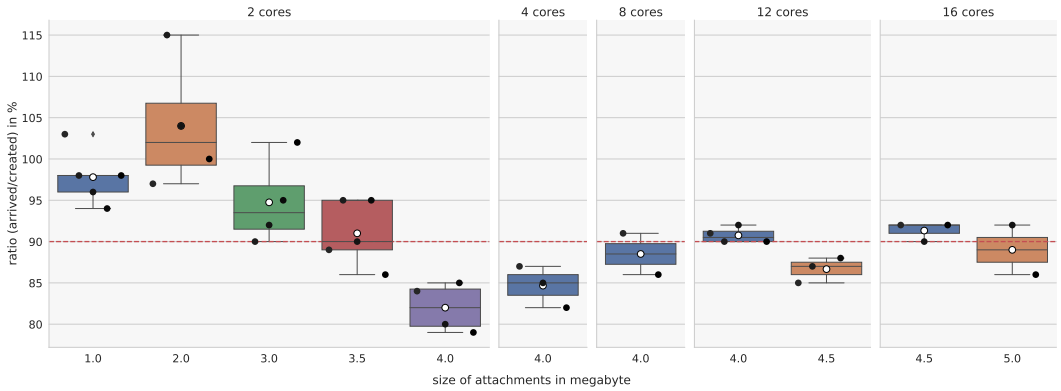
The previous experiments show the behavior of the PLS in respect to an increasing size of a single attachment which is included in each message. Figure 6.3, Figure 6.4, and Figure 6.5 visualize the latency, CPU utilization and the arrival ratio metric. Each figure shows a different frequency of generated messages.

Figure 6.6a shows that the ZI can handle a certain size of attached PDF files for each frequency. Adding more resources will not change the ratio substantially for files larger than this specific value. The total amount of transmitted/received bytes per minute depends on the frequency of send bytes and the file size. For the frequencies of 100, 200, and 400 msgs/min the amount of processable bytes is between 1700 MB and 1800 MB in total per minute (as shown in Figure 6.6b). Since these values are close to each other we assume that the total amount of processable bytes in this setup is independent of CPU performance. This becomes also evident since the ZI only used a small percentage of provided CPU cores (see Figure 6.3c, Figure 6.4c, Figure 6.5c). The relationship between the number of CPU cores and the arrival ratio metric is illustrated in Figure 6.7. It shows that the influence of the number of CPU cores is small. The graphic shows all load values for which Theodolite runs experiments with different resource values.

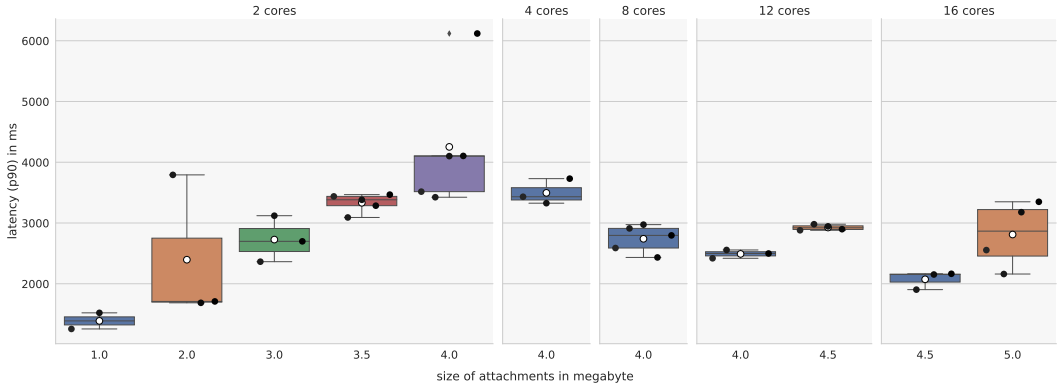
This analysis also discovers an increasing variance when the file size reaches a specific value. In particular, $B_{\text{size}}E_{100\,\text{msgs/min}}$ shows that the variance increases very suddenly for file sizes larger than 13 MB. Based on this, we investigate in the following execution the influence of provided CPU cores for a load, which can be successfully handled by a ZI.

6.3.7 Investigation of the Dependency of Variance and Resources

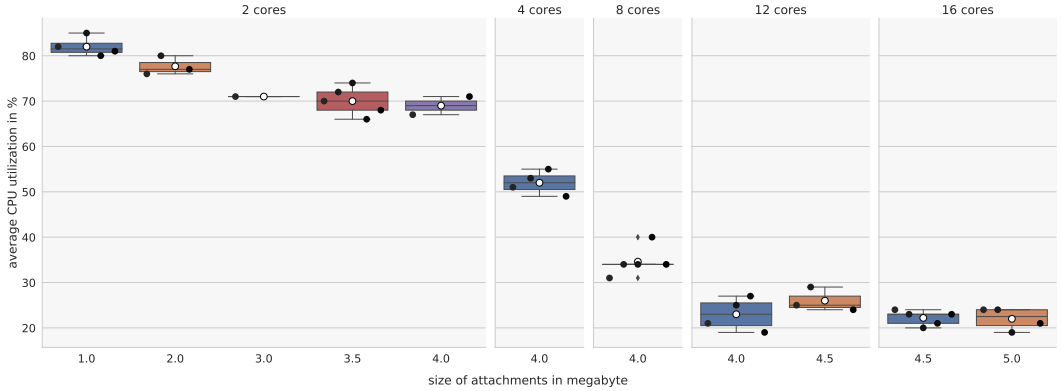
Derived from previous results, we design this execution to investigate the degree on which the number of CPU cores influence the stability of the results (e.g., variance) and denote it as $B_{\text{size}}E_{100\text{ msgs/min (diff. res., same load)}}$. In other words, we investigate whether the behavior of the ZI changes when more CPU resources are provided than required. In this context, handling the load means that the defined thresholds are not violated. For this, we consider 100 msgs/min and a file size of 13 MB. $B_{\text{size}}E_{100\,\text{msgs/min}}$ already presents the results for this constellation (100 msgs/min, $l = 13$ MB, $r = 2$ cores). Based on this execution, we re-run the same experiment and perform one additional experiment where only the resource values are change. Therefore, this experiment runs for the resources of 2 cores and 12 cores.



(a) Arrival ratio metric. The red line displays the threshold of 90 %.

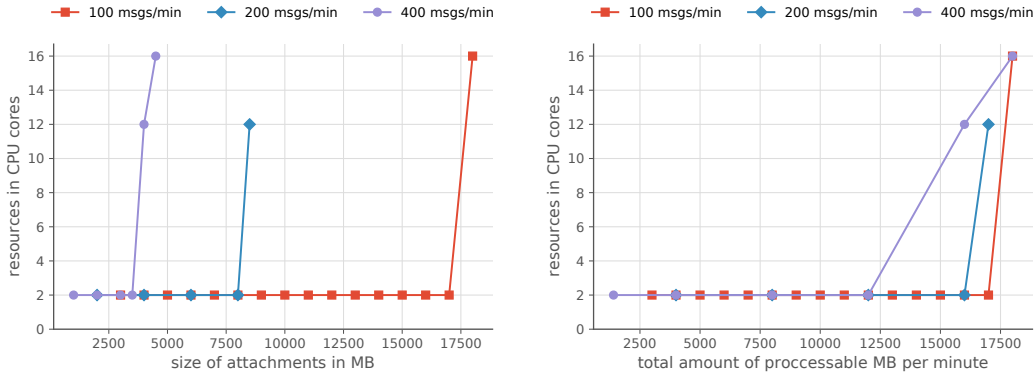


(b) Latency between PIs and ZI.



(c) Average percentage CPU utilization in respect to the defined CPU limits.

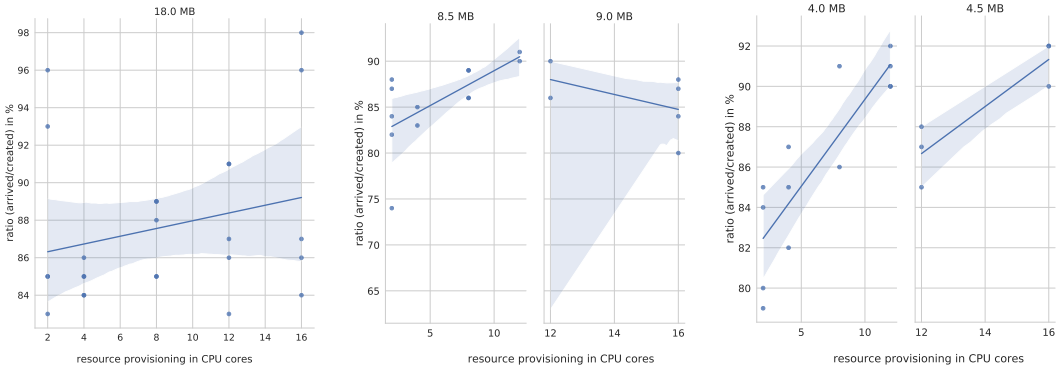
Figure 6.5. $B_{\text{size}} E_{400\,\text{msgs/min}}$ creates in total 400 msgs/min ($n = 4$). The results show that the ZI needs a large number of additional CPU cores to perform more load (see Section 6.3.3 for an explanation of this figure).



(a) The x-axis shows the size of the attached file.

(b) The x-axis shows the processable amount of megabytes in total. Therefore, the attachments' size is multiplied by the frequency.

Figure 6.6. Scalability results of the ZI regarding different sizes of attached files. The plots show the minimum required resources to process the messages with a specific size for a specific frequency of incoming messages. The lines end either at 16 cores or at the point where increasing the CPU to 16 cores does not increase the processable file size.

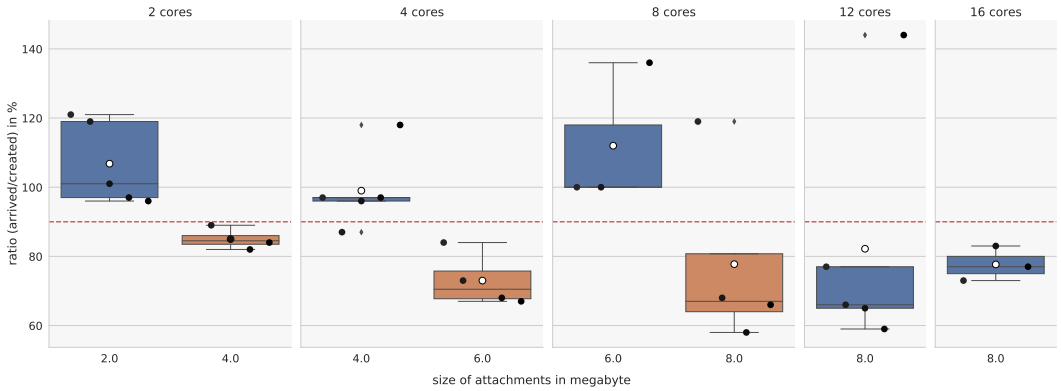


(a) Based on $B_{\text{size}}E_{100\,\text{msgs/min}}$. For a file size of 18 MB and 100 msgs/min, multiple resource values are considered.

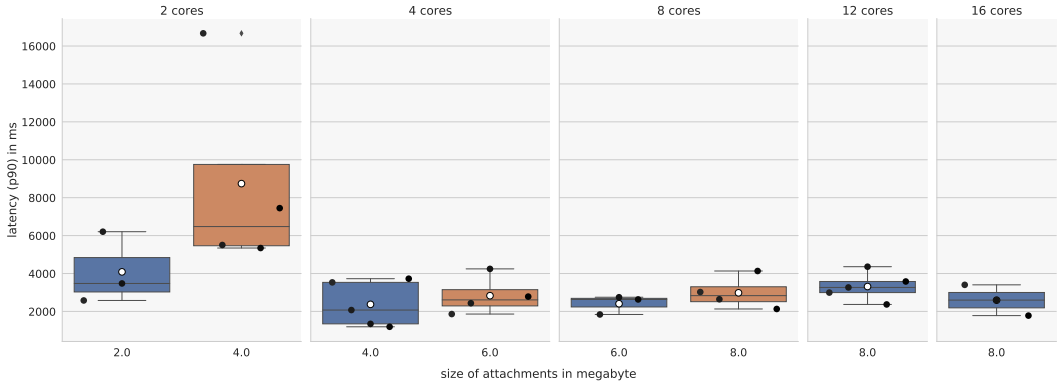
(b) Based $B_{\text{size}}E_{200\,\text{msgs/min}}$. For a file size of 10 MB and 200 msgs/min, multiple resource values are considered.

(c) Based on $B_{\text{size}}E_{400\,\text{msgs/min}}$. For file sizes of 4 MB and 5 MB and 400 msgs/min, multiple resource values are considered.

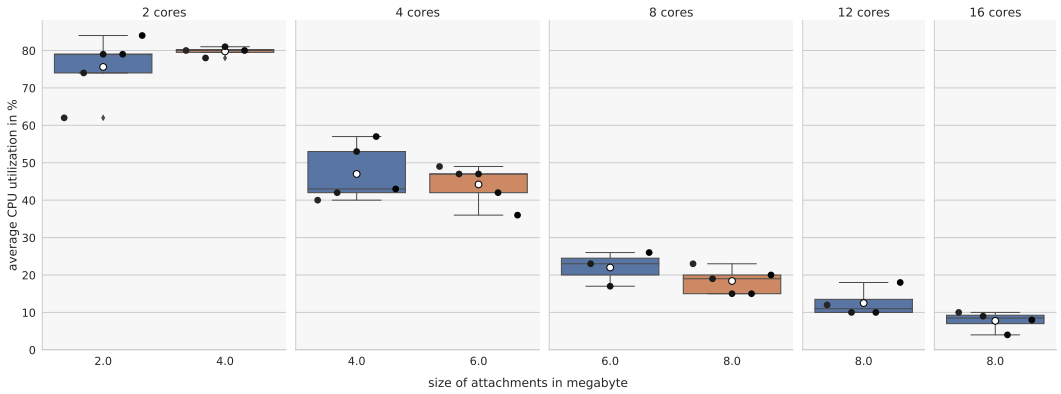
Figure 6.7. Relationship between the number of cores and sizes of received messages by linear regression for different frequencies. The x-axis shows the resource dimension and each column shows a load value. The y-axis shows the ratio metric. The second column of Figure 6.7b shows the anomaly that increasing resources do not lead to increasing performance.



(a) Ratio metric. The red line displays the threshold of 90 %.



(b) Latency between PIs and ZI in ms.



(c) Average percentage CPU utilization in respect to the defined CPU limits.

Figure 6.8. $B_{\text{size}}E_{100\,\text{msgs/min (decreased memory)}}$ investigates low memory limits by in total 100 msgs/min ($n = 1$). The performance is reduced in contrast to large memory limits (see Section 6.3.3 for an explanation of this figure).

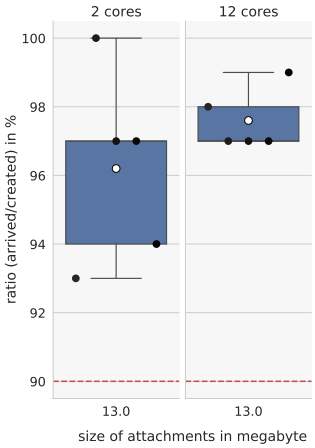


Figure 6.9. Relationship between the number of CPU cores and the arrival ratio metric for identical load (i.e, $l = 13\,\text{MB}$ and $r \in \{2\,\text{cores}, 12\,\text{cores}\}$). The metric increases and the variance decreases while providing more CPU resources.

Results The experiment shows that a higher number of CPU cores ensures more consistent performance of the ZI in terms of the arrival ratio metric. This becomes evident by a lower variance. High variance uncertainty indicates lower reproducibility of measurements [Kounev et al. 2021]. Figure 6.9 shows the arrival ratio metric of both experiments as box plots. The figure illustrates that the arrival ratio increases and the variance decreases while using more computational performance. Therefore, we infer the existence of a measurable impact of the number of provided CPU cores, regardless of the defined SLO.

6.3.8 Examining the Impact of Memory

The influence of the number of CPU cores tends to be low regarding the arrival ratio metric, as shown in previous experiments. Nevertheless, the experiments do not examine the impact of memory. Therefore, this experiment investigates the scenario of 2 GB memory limits (20 % of previous limits) for the ZI. We create 100 msgs/min in this experiment and refer to it as $B_{\text{size}}E_{100\,\text{msgs/min (decreased memory)}}$.

Results Figure 6.8 illustrates the measurements of this execution. The ZI restarts frequently in this experiment caused by out of memory errors. Despite these restarts, the results show that the influence of the CPU cores seems to be greater than with larger memory limits. Therefore, by adding more CPU cores, the manageable file size increases from 2.0 MB (in total 200 MB) up to 4.5 MB (in total 450 MB). Also, the performance is significantly lower than measured in $B_{\text{size}}E_{100\,\text{msgs/min}}$ (processable sizes up to 18 MB, in total 1800 MB) and the latency is higher. Figure 6.10 shows this graphically.

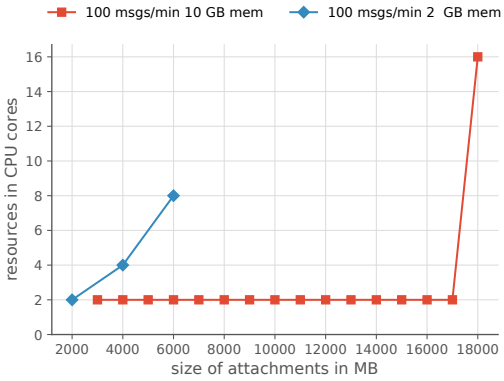


Figure 6.10. Contrasting the scalability of the ZI for 100 msgs/min with 10 GB and 2 GB memory provisioning by the demand metric. The lines end either at 16 cores or at the point where increasing the CPU to 16 cores does not increase the processable file size.

6.3.9 Results and Discussion of B_{size}

So far the experiments show barely scaling effects with large memory limits. While 2 cores were sufficient to manage 1700 MB in total, 16 cores are needed to process 1800 MB of the attached files. As discussed, the results indicate a limit of manageable bytes per minute of the ZI independent from the frequency or size in this setup. Execution $B_{\text{size}}E_{\text{100 msgs/min (diff. res., same load)}}$ shows that higher CPU performance increases the results' stability and the value of the arrival ratio metric. Therefore, a scalability effect result from increasing the resource dimension is present but tends to be low. In contrast, $B_{\text{size}}E_{\text{100 msgs/min (decreased memory)}}$ shows the influence of more assigned CPU cores for smaller memory limits. The execution indicates that the impact of higher resource values increases rapidly. Figure 6.6 and Figure 6.10 illustrate these scalability results based on the demand metric provided by Theodolite.

6.4 Scaling the Message Rate

In this section, we analyze the scalability of the ZI considering high rates of messages without attachments. This means sent messages are relatively small. Consequently, fewer bytes must be sent, making it possible to investigate higher frequencies. Therefore, the number of created messages per minute is used as load values. The resource dimension is the number of assigned cores of the ZI. As the second part of this scalability evaluation this is denoted as B_{rate}.

6.4.1 Setup

In B_{rate}, the parameters (introduced in Section 5.3.3) are defined as followed. We use $x = 20$ PIs and $y = 4$ PIs per PostgreSQL instance. Consequently, $z = \lceil \frac{20}{4} \rceil = 5$ PostgreSQL instances are required. $k = 20$ is the number of PI, on which a JMeter instance parallelly creates load. Therefore, one JMeter component is required. The number of JMeter replicas is $m = 5$.

We use the second test plan of Section 5.3.2 to create high rates of messages per minute (msgs/min). The test plan is configured to create five msgs/min per PI and JMeter instance. The process of creating these five messages is called an application of this test plan. We increase the rate of created msgs/min by increasing the number of applications of this test plan per minute. With the described configuration, we expect that one application per minute produces in total 500 msgs/min. Therefore, we aim to increase the load by increasing the number of applications in steps of 500 msgs/min. Equation 6.9 gives the total amount of messages expected to be send per minute. We specify the request rate by the changing the number of applications per minute.

$$\text{Total msgs/min} = m \times 5 \times \text{apm} \times x = 5 \times 5 \times \text{apm} \times 20, \text{ for apm } \in L \tag{6.9}$$

6.4.2 Experiment Configuration

Each experiment is executed for 18 minutes. The analysis starts after 6 minutes warm up time. We repeat each experiment five times.

6.4.3 Examine Increasing Request Rates

In this section, we focus on the scalability of the ZI for increasing rates of msgs/min and denote it as $B_{\text{rate}}E_{\text{increasing rates}}$. Our objective is to analyze the scalability for rates between 500 msgs/min and 5000 msgs/min in steps of 500 msgs/min. This is configured through the number of applications of the underlying test plan per minute (as described in Section 6.4.1 and Section 5.3.2). This specifies the rate of generated msgs/min as described above. However, apart from the technical configuration, the set of load values is $L_4 = \{500\,\text{msgs/min}, 1000\,\text{msgs/min}, \ldots\}$.

Results The analysis of this experiment shows on the one hand, that the test plan cannot yield the expected rate of messages (as illustrated in Figure 6.14a). Therefore, our analysis does not use the expected but the actual rate of generated messages.

On the other hand, the analysis shows that there is a strong influence on the number of provisioned CPU cores concerning the value of the arrival ratio metric (as illustrated in Figure 6.11a). Figure 6.11 presents that the CPU utilization decreases while the latency tends to remain stable. Similar to $B_{\text{size}}E_{200\,\text{msgs/min}}$ the performance decreases slightly by using the maximum number of 16 CPU cores. This could be caused by Kubernetes' scheduling and is discussed in the limitations of this work in Section 6.5.

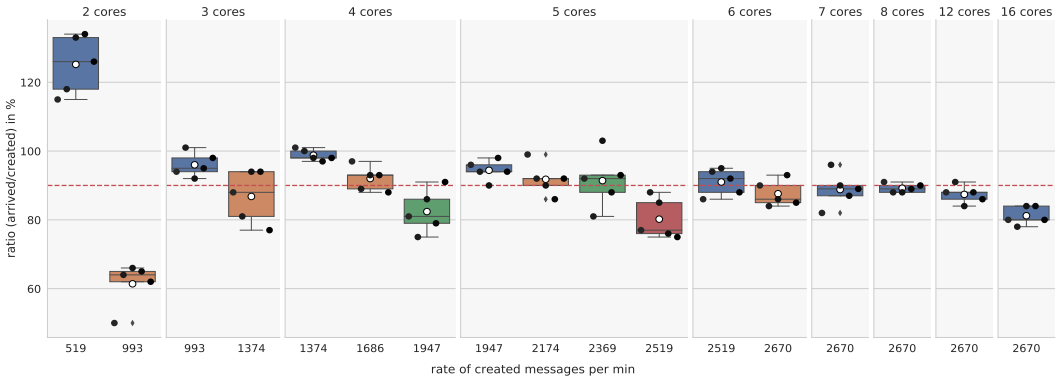
6.4.4 Examine Increasing Request Rates with Low Memory

In this section, we investigate the behavior of the ZI for high request rates and low memory provisioning. $B_{\text{size}}E_{100\,\text{msgs/min (decreased memory)}}$ shows that the memory size has a strong impact on the performance of the ZI. Also, the performance at low memory limits depends rather on the number of CPU cores than at large memory allocations. In the previous execution, we discovered that the number of CPU cores is crucial for the number of manageable msgs/min for high request rates. Both experiments are combined in this execution: high rates and small memory limits. We denote it as $B_{\text{rate}}E_{\text{increasing rates (decreased memory)}}$. To reduce the execution time we aim to increase the load by 1000 msgs/min instead of 500 msgs/min as above.

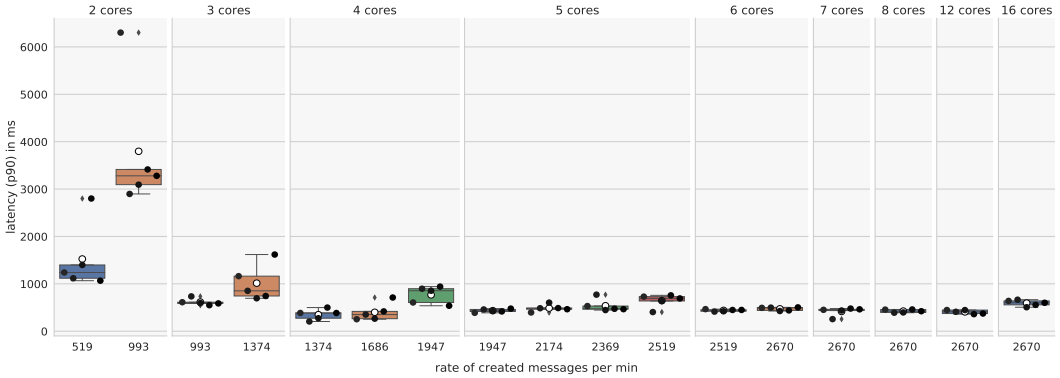
Results Figure 6.12 illustrates the result that the ZI is not able to handle the lowest created load in respect to the defined SLO. Unlike all previous experiments, the SLO violation is not affected by the SLO arrival ratio but rather by the SLO send errors. The defined threshold of on average a maximum of 20 HTTP requests/minute with an error indicating response is exceeded significantly (mainly with a 503[6] HTTP status code). Since so far this sub-SLO was not critical to the evaluation of the SLO, the corresponding metric is not discussed in this evaluation previously. This means previous experiments have not reached the threshold of 20 requests/minute with an error indicating response.

Figure 6.13a shows box plots illustrating the mean error rate between all PIs and the ZI. The mean shows extremely high rates. In contrast, Figure 6.13b shows the median value of the same measurements. The median value shows low rates (zero). Thus, we conclude that the ZI is unreachable for short periods of time. A reason for this could be that the garbage collection (freezes the JVM), possibly is more important for smaller memory sizes. However, it seems that the ZI can process a minimum of *500* requests per minute when considering larger periods of time and only the arrival ratio SLO. For further investigation, median values could be considered instead of average values for the error metrics.

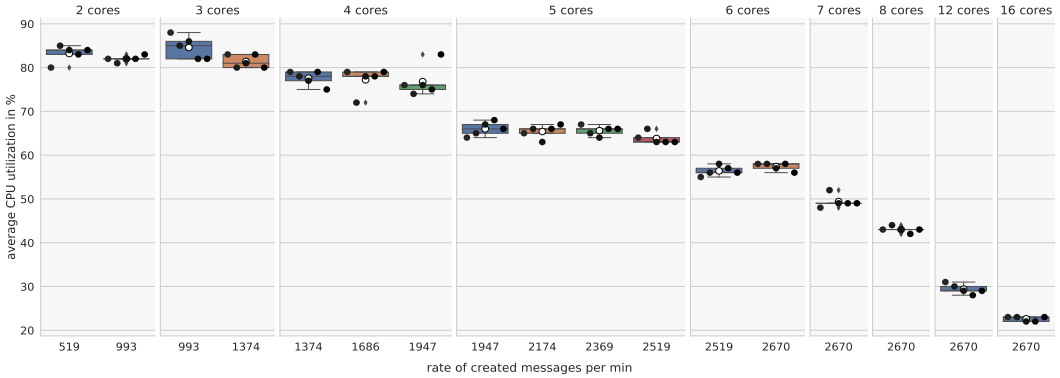
[6]Service Unavailable



(a) Arrival ratio metric. The red line displays the threshold of 90 %. The first experiment ($l = 519\,\text{msgs/min}, r = 2\,\text{cores}$) shows a fluctuation of the ratio metric by more than 100 %. This can occur when the rate of generated load is not constant. We discuss this in the limitations below.

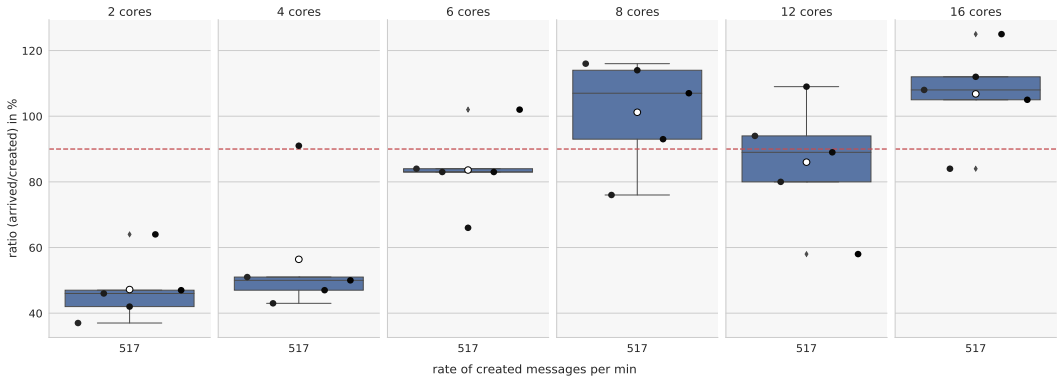


(b) Latency between PIs and ZI in ms (90th percentile) as box plot.

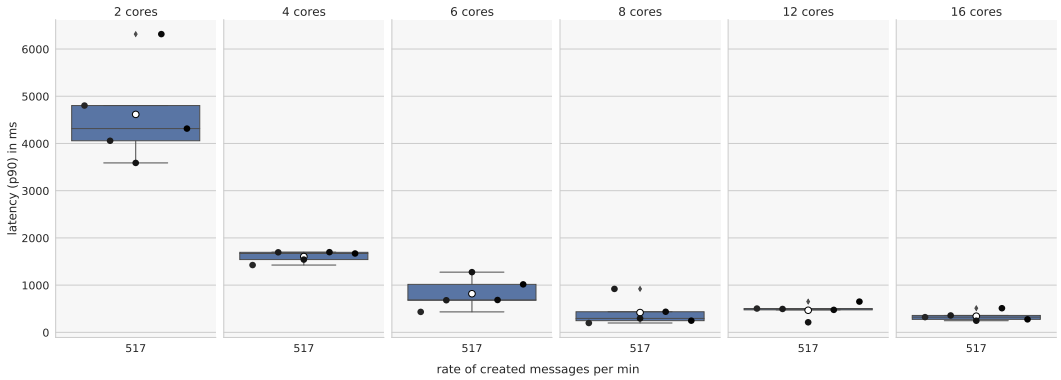


(c) Average percentage of CPU utilization in respect to the defined CPU limits.

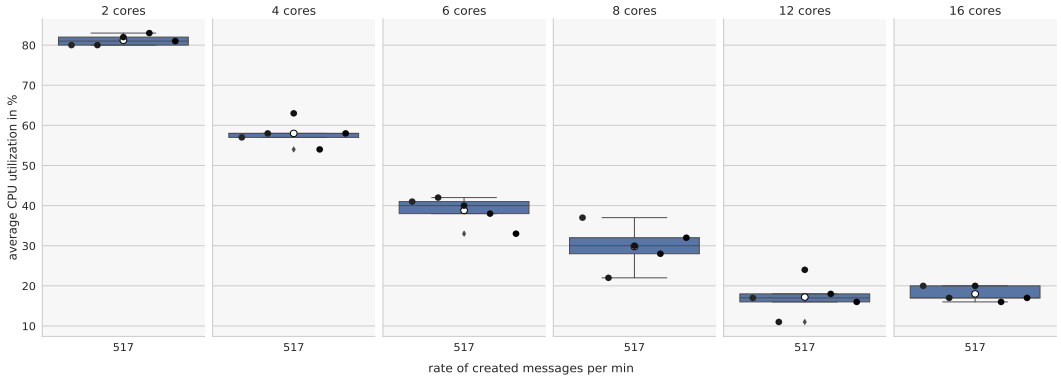
Figure 6.11. $B_{\text{rate}} E_{\text{increasing rates}}$, showing the behavior of the ZI for high request rates (see Section 6.3.3 for explanation of this figure).



(a) Arrival ratio metric. The red line displays the threshold of 90 %.

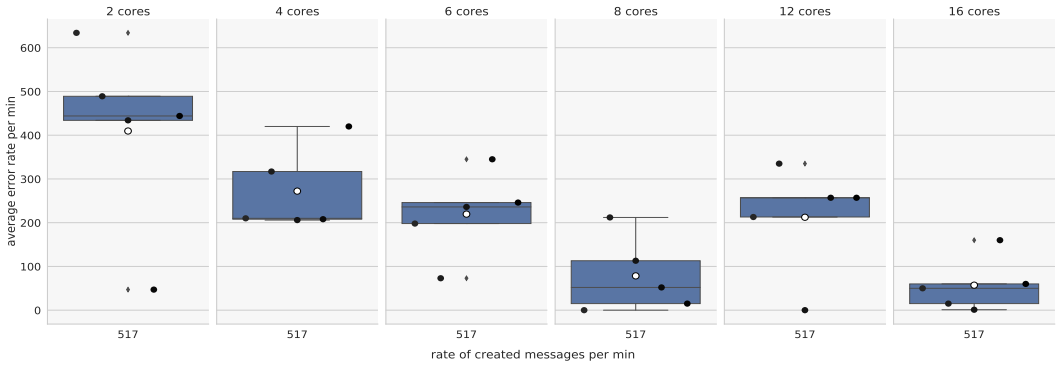


(b) Latency between PIs and ZI in ms.

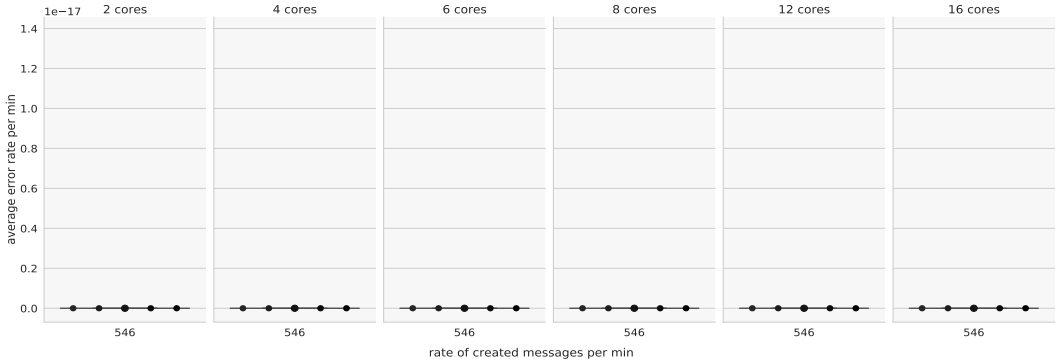


(c) Average percentage CPU utilization in respect to the defined CPU limits.

Figure 6.12. $B_{\text{rate}}E_{\text{increasing rates (decreased memory)}}$ investigates high request rates but low memory limits. The figure shows high fluctuations in the arrival ratio metric. Although the arrival ratio metric is high enough but the SLO is not fulfilled, as a cause of the error metric. This is visualized in Figure 6.13 (see Section 6.3.3 for an explanation of this figure).



(a) The boxes show the *mean* value of the error rate between all PIs and ZI (send error). The x-axis shows the *mean* value of the creation rate of messages per minute.

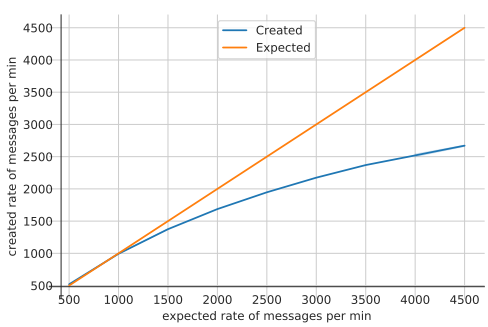


(b) The boxes illustrate the *median* value for the error rate between all PIs and ZI (send error). The x-axis shows the *median* value of the creation rate of messages per minute.

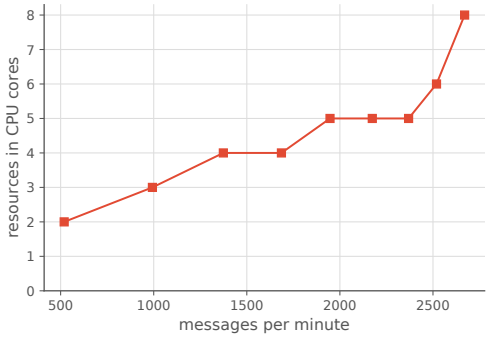
Figure 6.13. Comparison of the mean and the median of the amount of HTTP requests which are not acknowledged with an HTTP response code of *200* or *302*, as rate per minute resulting from $B_{\text{rate}} E_{\text{increasing rates (decreased memory)}}$. The considered requests are between the PIs and the ZI. To fulfill the SLO a maximum of 20 error indicating responses per minute are allowed. While the creation rate of msgs/min is not very different between the mean and median, the error rate is extremely different.

6.4.5 Results and Discussion of B_{rate}

The key observation of B_{rate} is that the ZI can process more messages by increasing the number of allocated CPU cores. This can be observed up to a number of eight cores. For more than eight and up to 16 allocated CPU cores, we can not detect any performance improvement in the ZI concerning the SLO. Figure 6.14b illustrates this using Theodolites' demand metric. Additionally, $B_{\text{rate}} E_{\text{increasing rates (decreased memory)}}$ addresses small memory sizes. In these cases, it can be shown that the error metric increases due to (short) unavailability times of ZI. This leads to the fact that the SLO is never satisfied.



(a) Rate of generated msgs/min in comparison to the expected rate of msgs/min.



(b) Scalability results of B_{rate}. The x-axis shows the observed rate of generated msgs/min.

Figure 6.14. Results of B_{rate}. This figure illustrates the ability of ZI to handle more load when more resources are available. It also shows the limitations of scaling load generation. The ZI can not handle more load by increasing the number of CPU cores up to 16.

Load Generation It can be seen that the rate of generated messages per minute is not as high as expected. Section 5.3.2 explain in details the underlying test plan. The expected behavior is that increasing the number of applications per minute by one, in turn, increases the creation rate by 500 msgs/min. In our experiments, the load scaled only linear by low intensities. In detail, the load increases by 500 msgs/min only for one or two applications per minute (apm) of this test plan. For a higher number of applications the increase decreases to a value of almost 200 msgs/min, for each increment of the apm. Figure 6.14a shows the discrepancy between expected and generated load. The time between two requests decreases as the number of requests per minute increases, which is necessary when the load is high. In addition to the increased utilization of the PIs this potentially makes more requests necessary. We assume that a higher distribution with more JMeter instances reduces this problem.

6.5 Threats to Validity and Limitations

This chapter outlines threats to validity and limitations possibly impacting this study.

6.5.1 Monitoring Overhead

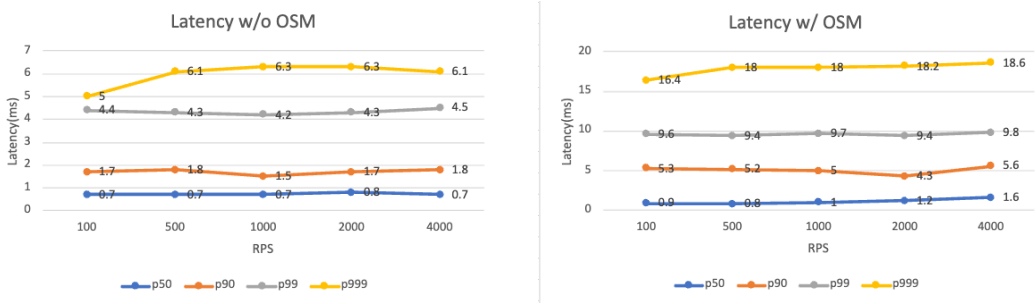
The impact of monitoring on the SUT and infrastructure should be as low as possible. Since instrumenting applications with Open Service Mesh (OSM) redirect all network traffic by adding Envoy proxies as sidecars, performance overhead is possible. The Cloud Native Computing Foundation [2021a] provides a benchmark comparing the service meshs

Linkerd[7] and Istio[8]. The analysis shows that both projects incur network overhead. The overhead is by 500 RPS, considering the 99th percentile by 3.6ms (Linkerd) and more than 643ms (Istio) [Morgan 2019]. However, OSM also provides benchmarks, analyzing the performance impact [Cloud Native Computing Foundation 2022b]. It should be noted that these benchmarks are executed on OSM v1.1, and we use v0.9 in our experiments but these are the best available data. It shows that OSM adds 4.9ms by 500RPS, considering the 99th percentile latency. Figure 6.15 shows this impact. The added latency is slightly larger than by Linkerd but much smaller than by Istio.

Our benchmarks show latencies of about 1000 ms for the 90th percentile. Therefore, we infer that the influence of the service mesh has little effect on the system and our measured latency findings.

6.5.2 Fluctuations in the Arrival Ratio Metric

Listing A.5 shows the Prometheus query to obtain the arrival ratio metric. This metric calculates the arrival ratio of received in relation to created messages per minute. Mathematically expressed the one-minute rate of received messages is divided by the one-minute rate of created messages for the same minute. Nevertheless, there is a time delay between the different components, particularly when the ZI is overloaded. Therefore, the minute ratio can theoretically differ from the whole experiment duration ratio. Also, it may cause the arrival ratio to take on a higher value than 100 %. A higher value than 100 % can occur, when the number of messages received by the ZI is not constant, which has two possible reasons. First, when the ZI is overloaded the latency increases to a high value through which the number of messages send to the ZI decreases (PIs send messages sequentially



(a) Network latencies between applications in Kubernetes without OSM [Cloud Native Computing Foundation 2022b].

(b) Network latencies between applications in Kubernetes with activated OSM [Cloud Native Computing Foundation 2022b].

Figure 6.15. Network overhead in terms of *Latency* caused by OSM.

[7] https://linkerd.io/
[8] https://istio.io/

and synchronously). The decreased send rate leads to decreasing latency and consequently the PIs are able to send messages at a higher rate which leads to a waveform of received messages. Consequently, there are time periods in which the ZI receives significantly more messages as created, which leads to a high ratio (received/created). In this case, the average ratio could be higher than 100 %. The second reason (only relevant for the second test plan) could be a non-constant creation rate, in which the ZI receives many messages for a moment, in which only a few messages are created, which leads to a high ratio and therefore the average also could be higher than 100 %. We assume, that both reasons cause the values of more than 100 % in our experiments. In most cases, the *arrival ratio per min* is approximately 1 % higher than the *arrival ratio per experiment*. The results of this evaluation are based on the *ratio per min*.

6.5.3 Latency Metric

OSM is not able to filter metrics by the host path or HTTP methods. Therefore, the latency metrics are calculated based on all requests between the PIs and the ZI. Since the quantitative amount of requests between the PIs and the ZI that are not related to messages (e.g., regular requests to exchange status information) is (i) low and (ii) constant in all cases, we assume that this does not affect our results.

6.5.4 Threshold of 90 Percent of the Arrival Ratio Metric

The threshold for all experiments performed is 90 %. We choose this low threshold because we assume that this only slightly shifts the relationship between load and resources with SLO being satisfied. The number of messages that can be processed is slightly higher with the resources provided, as with harder SLO limits. However, the discernible trend seems to be the same. In terms of the arrival ratio metric, this assumption should hold. Another reason is that the arrival ratio metric may have inaccuracies in individual cases, therefore a rough estimate leads to filtering out such effects. However, Figure 6.3a shows a huge increasing variance in the measured arrival ratio metric by falling below a value of 96 % of the arrival ratio metric. Probably the increasing variance is an indicator that the load is too high relative to the resources provided. A high number of repetitions could reduce the high variances but this would imply high execution times.

6.5.5 Performance Anomaly at 16 CPU Cores

$B_{\text{size}}E_{100\,\text{msgs/min}}$ and $B_{\text{rate}}E_{\text{increasing rates}}$ have slightly lowered performance when using 16 CPU cores for the ZI. Theoretically, this could be caused by the fact that Kubernetes cannot deploy all Pods corresponding to their *weighted* node affinities on the same node as the ZI since the ZI uses half of the existing CPU resources. A changed deployment can affect the performance of the whole system and would be might be able to explain the measured anomalies.

6.6 Summary

Respecting the discussed limitations, we highlight the most important results in this section.

The pre-study addresses *EQ 1* and discovers two bugs: A memory leak and too many established sessions in the communication between PIs, ZI, and Keycloak. It also found Keycloak as a potential bottleneck and a high variance in experimental results. Based on these findings, we use a patched version of the PLS and introduce two patchers to set Pod and node affinities in combination with specific resource requests and limits to make the schedule more reproducible.

In contrast, systematic investigations are executed in B_{size} and B_{rate}. These benchmark configurations address *EQ 1* and *EQ 2*, analyzing two different scalability aspects of the PLS. The main findings are listed below.

1. While considering a message with attachment, our experiments indicate that the ZI can process a certain amount of bytes in the setup under consideration. The load in bytes depends on the combination of frequency and size of the messages. This is in particular shown in B_{size} for 100, 200, and 400 msgs/min (Section 6.3.6).

2. Providing numerous CPU cores can stabilize the system's performance when dealing with messages with attachments, as analyzed in $B_{\text{size}}E_{\text{100 msgs/min (diff. res., same load)}}$ (Section 6.3.7).

3. While considering messages with attachments, the impact of the number of CPU cores increases while the available memory resources are reduced, as discussed in $B_{\text{size}}E_{\text{100 msgs/min (decreased memory)}}$ (Section 6.3.8).

4. For high rates of messages, it can be shown in $B_{\text{rate}}E_{\text{increasing rates}}$ that the number of CPU cores is crucial. This becomes evident because the ZI can process more messages with more CPU cores (Section 6.4.3).

5. In situations characterized by high message rates and low memory limits, the ZI appears to be partially unavailable. This becomes evident by the high percentage of queries that result in a 503 status code, as explained in $B_{\text{rate}}E_{\text{increasing rates (decreased memory)}}$ (Section 6.4.4).

These results show that the PLS, especially the ZI, can handle more load when more resources are available in the form of CPU cores and provisioned memory. This effect is negligible at low frequencies and large sizes. In such scenarios, we assume that the bottleneck is not the CPU performance but the available memory, I/O, network, or database performance. On the contrary, CPU performance seems critical when dealing with high rates of generated messages with small sizes.

Chapter 7

Related Work

In this thesis, we enhanced the observability of the PLS. Therefore, we built a monitoring and logging solution based on a service mesh to obtain network-level metrics, Grafana Loki, and Prometheus. The used service mesh is OSM. Frykholm and Mara [2021] introduce a similar monitoring solution based on *Istio*, which is probably the most common Envoy-based service mesh. Similar to our work, a distributed system in Kubernetes is considered. However, the focus of our work is slightly different. We use the service mesh to infer the state about applications running in the mesh. Frykholm and Mara [2021], on the other hand, focus on the capabilities and performance impact of a service mesh while monitoring a distributed application. Therefore, three research questions were addressed. The first research question, *"What data can a service mesh provide to aid in network monitoring?"* is implicitly also considered in this work. Both approaches consider the 90th percentile latency, calculated by Prometheus. However, Frykholm and Mara [2021] consider also *packet loss* and *bandwidth*, whereas our approach focuses additionally on the response code of transmitted HTTP requests. Apart from this, Frykholm and Mara [2021] also evaluate the research questions *"How can the data provided by a service mesh aid in identifying and locating problems with low bandwidth, high latency, and packet loss?"* and *"How does applying a proxy in the form of a service mesh affect application performance?"*. As a result, they report that changed latency and bandwidth behavior can be identified by an Envoy-based service network. However, increased packet loss rates cannot be detected. Experiments for the third research question indicate that the total transmission time increases with a service network. In our work, we discuss the impact of additional latency due to OSM as a limitation of the evaluation.

A vital aspect of this thesis was the description and implementation of the PLS benchmark to evaluate the performance and scalability of the financial software PLS. A popular benchmark for performance evaluation of a web server is the *SpecWeb 2009*, provided by the *Standard Performance Evaluation Corporation* [Standard Performance Evaluation Corporation 2009]. This benchmark provides different workloads, especially for benchmarking of web services in contexts of online banking. In contrast to our work, the main focus is considering the relation between the number of *simultaneous user sessions (sus)* and the power consumption. Furthermore, *SpecWeb 2009* does not focus on the scalability of distributed cloud systems but rather on the classic performance tests of web services.

In line with our approach, Vasar et al. [2012] introduced a framework for scalability benchmarking of web services in the cloud. The proposed framework helps performance engineers to set up and configure the environment, execute the SUT and create a load

based on *Tsung*[1]. Further, the framework can change the provided resources by adding or removing server instances, for example, *AWS EC2*[2] resources. The arrival rate of the last hour is considered to calculate whether server instances should be added or removed. If the desired number of instances differ from the currently allocated number, the framework will add or terminate server instances. This process is done every hour. The described experiment duration is one day [Vasar et al. 2012].

While Vasar et al. [2012] focus on testing the scalability of web services, the framework *PEEL*[3] [Boden et al. 2018] addresses benchmarking of distributed systems. Therefore *Hadoop*, *Flink*, and *Spark* are considered. Since benchmarking of distributed systems can be challenging, PEEL supports the processes of executing, analyzing, and sharing experiments. Benchmarking with PEEL is based on a four-phase model: *1. Define*, *2. Execute*, *3. Analyze*, *4. Share*. Experiments can be defined using *Spring*[4] injections container as a set of beans [Boden et al. 2018]. A CLI is provided for executing and interacting with PEEL. For analysis, PEEL stores measurements offline. Also, PEEL provides a mechanism for adding further SUTs. The central data structure is the *bundle*, which encapsulates all information to configure the environment, execute, and share experiments. PEEL focuses on a simple benchmark process distributed systems in a cloud context. Zhang et al. [2011] introduce the cloud-based performance testing system for web services (short CPTS) framework, which focuses on using cloud computing for load generation to resolve challenges by benchmarking distributed web services. CPTS is also portable, extensible, and "easy to use" [Zhang et al. 2011].

However, the *FRISBEE* [Nikolaidis et al. 2021] project focuses on testing cloud-native applications in Kubernetes. FRISBEE is a Kubernetes-native cloud platform for automating verifications, validations, and tests. This includes orchestrating distributed components (e.g., Microservices [Dragoni et al. 2017]), executing multiple phases like setup, steady-state, shutdown, clean-up, and analyzing the results. Similar to the Theodolite operator, the implementation of FRISBEE follows the operator pattern. Custom resources (CRs) exist to define benchmarks and interact with FRISBEE. Multiple CRs allow editing the desired Kubernetes deployment. For example, specifying resource provisioning or influencing the Kubernetes scheduling is possible. A central CR is the *Workflow*, the end-to-end description of an experiment. For example, modeling dependencies between components is possible by a directed-acyclic graph (DAG). This allows load generation to start if other components are up and running. In contrast to our approach, these are specified explicitly in the benchmark description. Our implementation based on Theodolite allows an order in the startup process by adding additional Pods and init containers indicating the state of specific system components (as described in Chapter 5). Also, FRISBEE does not focus explicitly on the scalability of SUTs but both projects aims to decrease the complexity of benchmarking.

While Nikolaidis et al. [2021] solved a similar problem to us, Vonheiden [2021] and

[1] `http://tsung.erlang-projects.org/`

[2] `https://aws.amazon.com/de/ec2/`

[3] `https://github.com/peelframework/peel`

[4] `https://spring.io/`

Ehrenstein [2022] also use Theodolite to address different research questions. Vonheiden [2021] uses Theodolite to evaluate the scalability of hopping window aggregations of different stream processing engines. Therefore, two different benchmarks are evaluated. One of these benchmarks is the *OSPBench* [van Dongen and Van den Poel 2020], which is not natively provided by Theodolite. The OSPBench uses different stream processing engines as SUTs. While this is the first integration of a non-Theodolite benchmark, our approach addresses the first integration of (i) an industrial system, (ii) which is not implemented as a streaming application, and (iii) is secured through an identity provider. In contrast to our work, Ehrenstein [2022] extends Theodolite to an additional evaluation method based on Universal Scalability Law [Gunther 1993]. This modification enables more efficient benchmarking. Additionally, Ehrenstein [2022] evaluates the scalability of *Software Visualization and Comprehension Tool ExplorViz* [Fittkau et al. 2017; Hasselbring et al. 2020]. ExplorViz is also not provided by Theodolite but the main parts are based on stream processing instead of HTTP communication. HTTP-based communication requires not only a different way of load generation but also additional monitoring possibilities as well as other SLOs.

Chapter 8

Conclusion and Future Work

This chapter summarizes the results and findings of this thesis, followed by an outlook on future work.

8.1 Conclusion

In this work, we enhanced the observability of the PLS, provided a benchmark to evaluate the scalability of the PLS, and consequently, we performed a scalability analysis. A particular focus of this thesis was to identify whether the ZI acts correctly or not. We improved the observability of the PLS to gain knowledge about the system state. In this context, we integrated the service mesh OSM, the logging solution Grafana Loki and different Prometheus exporters to the deployment. We identified multiple SLOs which must be met to ensure that the ZI is not overloaded. The most critical SLO indicates whether the ZI can receive all created messages within a specific time.

In addition, we designed a benchmark that specifies the number of CPU cores as resource dimension and the load dimensions either as the size of or frequency of messages which must be handled by the ZI. We realized this concept by an executable benchmark implementation based on JMeter as the load generator and Theodolite as the control unit. Therefore, we have shown the integration of the load generator JMeter into the Theodolite framework. Including JMeter allows simulating complex user behavior based on HTTP traffic. Multiple patchers were developed to enable Theodolite to deploy and configure the PLS as SUT and the load generator JMeter. The patchers allow the replication of the PIs which requires replication, configuration, and correlation of multiple components.

Finally, we addressed the research questions and performed scalability benchmarks on the PLS. Therefore, two scenarios were considered. The first benchmark configuration enlarged the messages by including several attachments. We increased the rate of created messages per minute in the second benchmark configuration. Multiple sub-experiments allowed the evaluation from several perspectives. So multiple frequencies and provided memory sizes were considered to built a comprehensive understanding of scalability of the ZI. We found out that the ZI's ability to scale with the number of CPU cores is low for large memory sizes and messages with big attachments. The scalability, in terms of provided CPU cores, increases while decreasing the memory size. In contrast, considering high frequencies showed that the ZI can handle more messages as more CPU cores are

provided (up to eight cores). In addition, these benchmarks identified a memory leak and a programming error that led to excessive sessions during communication with the identity provider Keycloak.

8.2 Future Work

Future studies should aim to replicate the results of this work in a larger cloud environment. More computational resources allow deploying a high number of PI instances, making it possible to use a low frequency but many PIs to create another benchmark scenario. Additionally, the next benchmarks could address *credit applications* instead of *messages*, which requires only the modification of existing test plans. As previously discussed, the 90 % threshold for the ratio metric may be too low; further experiments could be conducted using a 95 % threshold. Increasing the threshold to 95 % in the offline analysis is not possible due to the chosen search strategy. Further, considering the median instead of the mean could be helpful for the error metric to be more resistant against short periods with many errors (described in Section 6.4). Since in our experiments there are executions with high variance in the results, in further investigations the variance could be added as an indicator, whether an experiment runs successful or not. This could be done either by evaluating the coefficient of variation or by contrasting confidence intervals.

Our evaluation shows that the ZI can process a maximum of 1800 MB of attached files per minute. The 1800 MB are cumulated over all messages. Further research should address bottleneck detection. This could be done by adding TCP monitoring to obtain PostgreSQL latency and by adding systematic evaluation of I/O and network states.

Preparing Theodolite for automated benchmarks as part of the continuous integration process could be also addressed in future work. This could help to detect and repair performance issues and support DevOps [Waller et al. 2015]. System benchmarking as part of a continuous quality management process makes it possible to compare the performance of different software versions and ensure that new versions are at least as good as old versions [Grambow et al. 2019]. This could support the quality assurance of the PLS.

A completely different aspect of future work could be focussing on evolutionary algorithms. Evolutionary algorithms [Bäck and Schwefel 1993] use a process like a natural evolution to find fitting parameterization of computer systems. Frey et al. [2013] introduce an approach for a genetic optimization search to evaluate and configure cloud deployment options (CDOs) for the migration of enterprise systems to the cloud. A time-consuming part of this work was finding *suitable* parameterization for the number of PIs, JMeter instances, the request rate, the resource provisioning, or the size of messages. Adding these capabilities to Theodolite could be helpful in the future.

Appendix A

LogQL and PromQL Queries

Listing A.1. LogQL query to get the one minute rate of received messages on the ZI.

```
sum(
    rate(
        {container=‘pls-zi-bank‘} |= ‘Mitteilung vom PI‘[1m]
        )
    )
*
60
```

Listing A.2. LogQL query to get the one minute rate created messages.

```
sum(
    rate(
        {container=‘jmeter‘} |= ‘Mitteilung freigegeben: true‘[1m]
        )
    )
*
60
```

Listing A.3. P90 Latency Query in PromQL.

```
histogram_quantile(
    0.9,
    sum(
        irate(
            osm_request_duration_ms_bucket{source_name=~'deployment_app.*',
                destination_name='deployment_pls_zi_bank_app'}[1m]
            )
        )
    by (le)
    )
> 0
```

Listing A.4. PromQL query to count the number of restarts of the ZI container.

```
    sum(
            (
                sum(
                    kube_Pod_container_status_restarts_total{container='pls-zi-
                        bank'}
                    )
            )
            > 0
        or
        vector(0)
    )
```

Listing A.5. PromQL query to get the ratio metric.

```
(
    sum(
        rate(
            {container='pls-zi-bank'} |= 'Mitteilung vom PI'[1m])
        )
    /
    sum(
        rate(
            {container='jmeter'} |= 'Mitteilung freigegeben: true'[1m]
            )
        )
)
*
100
```

Listing A.6. PromQL query to calcluate the rate of failing requests between JMeter and the PIs.

```
sum(
    rate(
        osm_request_total{source_name=~'jmeter.*', destination_name=~'
            deployment_app.*', response_code!='302', response_code!='200'}[1m]
        )
    )
*
60
or
vector(0)"
```

Listing A.7. PromQL query to calcluate the rate failing requests between the PIs and the ZI.

```
sum(
    rate(
        osm_request_total{source_name=~'deployment_app.*', destination_name='
            deployment_pls_zi_bank_app', response_code!='200'}[1m]
        )
    )
*
60
or
vector(0)"
```

Listing A.8. PromQL query[1] to get relative CPU utilization in relation to specified limits.

```
round(
    100
    *
    sum(
        rate(
            container_cpu_usage_seconds_total{container='pls-zi-bank'}[1m]
            )
        )
    /
    sum(
        container_spec_cpu_quota{container='pls-zi-bank'}
        /
        container_spec_cpu_period{container='pls-zi-bank'}
        )
    )
```

[1] https://github.com/google/cadvisor/issues/2026#issuecomment-486134079

Bibliography

[Armbrust et al. 2010] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia. A view of cloud computing. *Commun. ACM* 53.4 (2010). DOI: 10.1145/1721654.1721672. (Cited on page 1)

[b+m Informatik AG 2021] b+m Informatik AG. *b+m PLS Benutzerhandbuch*. b+m Informatik AG. 2021. 2021. (Cited on pages 1, 6)

[Bäck and Schwefel 1993] T. Bäck and H.-P. Schwefel. An overview of evolutionary algorithms for parameter optimization. *Evolutionary Computation* 1.1 (1993). DOI: 10.1162/evco.1993.1.1.1. (Cited on page 82)

[Bensien 2021] J. R. Bensien. Scalability benchmarking of stream processing engines with Apache Beam. Bachelor's Thesis. Kiel University, 2021. (Cited on page 13)

[Bermbach et al. 2017] D. Bermbach, E. Wittern, and S. Tai. *Cloud service benchmarking*. Springer, 2017. DOI: 10.1007/978-3-319-55483-9. (Cited on pages 2, 7–10, 27, 28, 33, 43, 48, 51, 53)

[Beyer 2016] B. Beyer. *Site reliability engineering: how Google runs production systems*. O'Reilly Media, 2016. (Cited on pages 9–11, 14, 49)

[Biernat 2020] N. A. Biernat. Scalability benchmarking of Apache Flink. Bachelor's Thesis. Kiel University, 2020. (Cited on page 13)

[Bilzhause et al. 2016] A. Bilzhause, M. Huber, H. C. Pöhls, and K. Samelin. Cryptographically enforced four-eyes principle. In: *2016 11th International Conference on Availability, Reliability and Security (ARES)*. 2016. DOI: 10.1109/ARES.2016.28. (Cited on page 28)

[Boden et al. 2018] C. Boden, A. Alexandrov, A. Kunft, T. Rabl, and V. Markl. PEEL: a framework for benchmarking distributed systems and algorithms. In: *Performance Evaluation and Benchmarking for the Analytics Era*. Springer, 2018. (Cited on pages 48, 78)

[Boyd 2012] R. Boyd. *Getting started with OAuth 2.0*. O'Reilly Media, 2012. (Cited on page 19)

[Brataas et al. 2017] G. Brataas, N. Herbst, S. Ivansek, and J. Polutnik. Scalability analysis of cloud software services. In: *2017 IEEE International Conference on Autonomic Computing (ICAC)*. 2017. DOI: 10.1109/ICAC.2017.34. (Cited on pages 9, 10)

[Brataas and Fægri 2017] G. Brataas and T. E. Fægri. Agile scalability requirements. In: *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering*. 2017. (Cited on page 9)

[Burns et al. 2016] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM* 59 (2016). DOI: 10.1145/2890784. (Cited on page 11)

[Carbone et al. 2020] P. Carbone, M. Fragkoulis, V. Kalavri, and A. Katsifodimos. Beyond analytics: the evolution of stream processing systems. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, 2020. DOI: 10.1145/3318464.3383131. (Cited on page 13)

[Cloud Native Computing Foundation 2018] Cloud Native Computing Foundation. *CNCF cloud native definition v1.0*. https://github.com/cncf/toc/blob/main/DEFINITION.md. 2018. (Cited on page 11)

[Dragoni et al. 2017] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina. Microservices: yesterday, today, and tomorrow. In: *Present and Ulterior Software Engineering*. Springer, 2017. DOI: 10.1007/978-3-319-67425-4_12. (Cited on pages 11, 78)

[Ehmer and Khan 2012] M. Ehmer and F. Khan. A comparative study of white box, black box and grey box testing techniques. *International Journal of Advanced Computer Science and Applications* 3 (2012). DOI: 10.14569/IJACSA.2012.030603. (Cited on pages 1, 9)

[Ehrenstein 2022] S. B. N. F. A. Ehrenstein. Scalability evaluation of ExplorViz with the Universal Scalability Law. Master's thesis. Kiel University, 2022. (Cited on pages 13, 79)

[Lyft 2022] Lyft. *Envoy proxy*. https://www.envoyproxy.io/docs. 2022. (Cited on page 22)

[Erinle 2017] B. Erinle. *Performance Testing with JMeter 3*. Packt Publishing, 2017. (Cited on page 21)

[Few 2006] S. Few. *Information dashboard design: the effective visual communication of data*. O'Reilly Media, Inc., 2006. (Cited on page 23)

[Fittkau et al. 2017] F. Fittkau, A. Krause, and W. Hasselbring. Software landscape and application visualization for system comprehension with ExplorViz. *Information and Software Technology* 87 (2017). DOI: https://doi.org/10.1016/j.infsof.2016.07.004. (Cited on page 79)

[Frey et al. 2013] S. Frey, F. Fittkau, and W. Hasselbring. Search-based genetic optimization for deployment and reconfiguration of software in the cloud. In: *2013 35th International Conference on Software Engineering (ICSE)*. 2013. DOI: 10.1109/ICSE.2013.6606597. (Cited on page 82)

[Frykholm and Mara 2021] E. Frykholm and E. Mara. *Evaluating service mesh as a network monitoring solution*. Student Paper. 2021. (Cited on page 77)

[Gannon et al. 2017] D. Gannon, R. Barga, and N. Sundaresan. Cloud-native applications. *IEEE Cloud Computing* 4.5 (2017). DOI: 10.1109/MCC.2017.4250939. (Cited on page 1)

[Goniwada 2021] S. R. Goniwada. *Cloud native architecture and design: a handbook for modern day architecture and design with enterprise-grade examples*. Apress, 2021. (Cited on pages 1, 23)

[Grambow et al. 2019] M. Grambow, F. Lehmann, and D. Bermbach. Continuous benchmarking: using system benchmarking in build pipelines. In: *2019 IEEE International Conference on Cloud Engineering (IC2E)*. 2019. DOI: 10.1109/IC2E.2019.00039. (Cited on page 82)

[Gregg 2020] B. Gregg. *Systems performance: enterprise and the cloud*. Pearson, 2020. (Cited on pages 28, 31)

[Gunther 1993] N. J. Gunther. A simple capacity model of massively parallel transaction systems. In: *Int. CMG Conference*. 1993. (Cited on page 79)

[Hardt 2012] D. Hardt. *The OAuth 2.0 authorization framework*. Technical report. 2012. (Cited on page 19)

[Hashemian et al. 2012] R. Hashemian, D. Krishnamurthy, and M. Arlitt. Web workload generation challenges - an empirical investigation. *Software: Practice and Experience* 42.5 (2012). DOI: https://doi.org/10.1002/spe.1093. (Cited on page 35)

[Hasselbring 2021] W. Hasselbring. Benchmarking as empirical standard in software engineering research. In: *Evaluation and Assessment in Software Engineering*. ACM, 2021. DOI: 10.1145/3463274.3463361. (Cited on pages 2, 7, 27)

[Hasselbring et al. 2020] W. Hasselbring, A. Krause, and C. Zirkelbach. ExplorViz: research on software visualization, comprehension and collaboration. *Software Impacts* 6 (2020). DOI: https://doi.org/10.1016/j.simpa.2020.100034. (Cited on page 79)

[Henning and Hasselbring 2021a] S. Henning and W. Hasselbring. How to measure scalability of distributed stream processing engines? In: *Companion of the ACM/SPEC International Conference on Performance Engineering*. ACM, 2021. (Cited on pages 13, 14)

[Henning and Hasselbring 2021b] S. Henning and W. Hasselbring. Theodolite: scalability benchmarking of distributed stream processing engines in microservice architectures. *Big Data Research* 25 (2021). DOI: 10.1016/j.bdr.2021.100209. (Cited on page 13)

[Henning and Hasselbring 2022a] S. Henning and W. Hasselbring. A configurable method for benchmarking scalability of cloud-native applications. *Empirical Software Engineering* (2022). In press. DOI: 10.1007/s10664-022-10162-1. (Cited on pages 1, 7, 13, 14, 27, 30, 32, 33)

[Henning and Hasselbring 2022b] S. Henning and W. Hasselbring. Demo paper: benchmarking scalability of cloud-native applications with Theodolite. In: *2022 IEEE International Conference on Cloud Engineering (IC2E 2022)*. 2022. (Cited on page 34)

[Henning et al. 2021] S. Henning, B. Wetzel, and W. Hasselbring. Reproducible benchmarking of cloud-native applications with the Kubernetes operator pattern. In: *Symposium on Software Performance 2021*. 2021. (Cited on pages 13, 14, 16, 18)

[Herbst et al. 2013] N. R. Herbst, S. Kounev, and R. Reussner. Elasticity in cloud computing: what it is, and what it is not. In: *10th International Conference on Autonomic Computing (ICAC 13)*. USENIX Association, 2013. (Cited on pages 1, 13)

[Huppler 2009] K. Huppler. The art of building a good benchmark. In: Springer, 2009. DOI: https://doi.org/10.1007/978-3-642-10424-4_3. (Cited on page 9)

[Ibryam and Huß 2019] B. Ibryam and R. Huß. *Kubernetes patterns: reusable elements for designing cloud-native applications*. O'Reilly Media, 2019. (Cited on page 12)

[Jake 2009] B. Jake. *Speed matters for Google web search*. Google Study. 2009. (Cited on page 31)

[Jamshidi et al. 2013] P. Jamshidi, A. Ahmad, and C. Pahl. Cloud migration research: a systematic review. *IEEE Transactions on Cloud Computing* 1.2 (2013). DOI: 10.1109/TCC.2013.10. (Cited on page 1)

[Oracle 2022] Oracle. *Java documentation*. https://docs.oracle.com/en/java/. 2022. (Cited on page 55)

[Apache Software Foundation 2021] Apache Software Foundation. *Apache JMeter*. http://jmeter.apache.org. 2021. (Cited on pages 22, 35, 38, 43)

[Kounev et al. 2021] S. Kounev, K.-D. Lange, and J. von Kistowski. *Systems benchmarking*. Springer, 2021. (Cited on pages 7–10, 51, 66)

[Kratzke and Quint 2017] N. Kratzke and P.-C. Quint. Understanding cloud-native applications after 10 years of cloud computing - a systematic mapping study. *Journal of Systems and Software* 126 (2017). DOI: https://doi.org/10.1016/j.jss.2017.01.001. (Cited on page 1)

[Cloud Native Computing Foundation 2022a] Cloud Native Computing Foundation. *Kubernetes documentation*. https://kubernetes.io. 2022. (Cited on pages 11, 12, 22, 24, 25, 31, 34, 47, 56)

[Lehrig et al. 2015] S. Lehrig, H. Eikerling, and S. Becker. Scalability, elasticity, and efficiency in cloud computing: a systematic literature review of definitions and metrics. In: *Proceedings of the 11th International ACM SIGSOFT Conference on Quality of Software Architectures*. ACM, 2015. DOI: 10.1145/2737182.2737185. (Cited on page 13)

[Li et al. 2019] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han. Service mesh: challenges, state of the art, and future research opportunities. In: *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 2019. DOI: 10.1109/SOSE.2019.00026. (Cited on page 22)

[Matam and Jain 2017] S. Matam and J. Jain. Distributed testing. In: *Pro Apache JMeter: Web Application Performance Testing*. Apress, 2017. DOI: 10.1007/978-1-4842-2961-3_6. (Cited on pages 22, 35, 43)

[Maxwell et al. 2010] E. K. Maxwell, G. Back, and N. Ramakrishnan. Diagnosing memory leaks using graph mining on heap dumps. In: *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2010. DOI: 10.1145/1835804.1835822. (Cited on pages 42, 55)

[Merkel 2014] D. Merkel. Docker: lightweight linux containers for consistent development and deployment. *Linux J.* 2014.239 (2014). (Cited on page 11)

[Cloud Native Computing Foundation 2021a] Cloud Native Computing Foundation. *Benchmarking Linkerd and Istio*. https://www.cncf.io/blog/2021/12/17/benchmarking-linkerd-and-istio-2021-redux/. 2021. (Cited on page 73)

[Morgan 2019] W. Morgan. *Linkerd benchmarks*. https://linkerd.io/2019/05/18/linkerd-benchmarks/. 2019. (Cited on page 74)

[Cloud Native Computing Foundation 2022b] Cloud Native Computing Foundation. *Open Service Mesh benchmarks*. https://release-v1-1.docs.openservicemesh.io/docs/guides/ha_scale/perf-scale/#latency. 2022. (Cited on page 74)

[Morgenthaler 2009] S. Morgenthaler. Exploratory data analysis. *WIREs Computational Statistics* 1.1 (2009). DOI: https://doi.org/10.1002/wics.2. (Cited on page 51)

[Murawski et al. 2014] G. Murawski, P. Keck, and S. Schnaible. Evaluation of load testing tools (2014). DOI: 10.18419/opus-3417. (Cited on page 21)

[N. and Cholli 2020] D. N. and N. Cholli. A review on identity and access management server (Keycloak). *International Journal of Security and Privacy in Pervasive Computing* 12 (2020). DOI: 10.4018/IJSPPC.2020070104. (Cited on page 18)

[Nikolaidis et al. 2021] F. Nikolaidis, A. Chazapis, M. Marazakis, and A. Bilas. *Frisbee: automated testing of cloud-native applications in Kubernetes*. 2021. DOI: 10.48550/ARXIV.2109.10727. (Cited on page 78)

[Nongbri et al. 2018] I. Nongbri, P. Hadem, and S. Chettri. A survey on single sign-on. 6 (2018). DOI: 10.5281/zenodo.5763157. (Cited on page 18)

[OpenID Foundation 2022] OpenID Foundation. *OpenID specification*. https://openid.net/connect/. 2022. (Cited on page 19)

[Cloud Native Computing Foundation 2021b] Cloud Native Computing Foundation. *Open Service Mesh*. https://release-v0-9.docs.openservicemesh.io/. 2021. (Cited on page 22)

[Pahl et al. 2018] C. Pahl, P. Jamshidi, and O. Zimmermann. Architectural principles for cloud software. *ACM Trans. Internet Technol.* 18.2 (2018). DOI: 10.1145/3104028. (Cited on page 1)

[Papadopoulos et al. 2021] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. v. Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, and A. Iosup. Methodological principles for reproducible performance evaluation in cloud computing. *IEEE Transactions on Software Engineering* 47.8 (2021). DOI: 10.1109/TSE.2019.2927908. (Cited on pages 27, 43, 56)

[Pohlmann 2019] N. Pohlmann. Cyber-sicherheit: das lehrbuch für konzepte, prinzipien, mechanismen, architekturen und eigenschaften von cyber-sicherheitssystemen in der digitalisierung (2019). DOI: 10.1007/978-3-658-25398-1. (Cited on page 19)

[Pongratz and Vogelgesang 2019] P. Pongratz and M. Vogelgesang. Unternehmensfinanzierung und -förderung: grundlagen für die praxis. *Wirtschaftsförderung in Lehre und Praxis* (2019). DOI: 10.1007/978-3-658-25969-3. (Cited on page 5)

[Ralph 2021] P. Ralph. ACM SIGSOFT empirical standards released. *SIGSOFT Softw. Eng. Notes* 46.1 (2021). DOI: 10.1145/3437479.3437483. (Cited on pages 6, 7, 27)

[Ralph et al. 2021] P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, B. B. N. de França, C. A. Furia, G. Gay, N. Gold, D. Graziotin, P. He, R. Hoda, N. Juristo, B. Kitchenham, V. Lenarduzzi, J. Martínez, J. Melegati, D. Mendez, T. Menzies, J. Molleri, D. Pfahl, R. Robbes, D. Russo, N. Saarimäki, F. Sarro, D. Taibi, J. Siegmund, D. Spinellis, M. Staron, K. Stol, M.-A. Storey, D. Taibi, D. Tamburri, M. Torchiano, C. Treude, B. Turhan, X. Wang, and S. Vegas. *Empirical standards for software engineering research*. 2021. (Cited on page 7)

[Schroeder et al. 2006] B. Schroeder, A. Wierman, and M. Harchol-Balter. Open versus closed: a cautionary tale. In: USENIX. 2006. (Cited on page 10)

[Sim et al. 2003] S. Sim, S. Easterbrook, and R. Holt. Using benchmarking to advance research: a challenge to software engineering. In: *25th International Conference on Software Engineering, 2003. Proceedings.* 2003. DOI: 10.1109/ICSE.2003.1201189. (Cited on pages 1, 7, 8)

[Standard Performance Evaluation Corporation 2009] Standard Performance Evaluation Corporation. *Specweb 2009 benchmark*. http://www.spec.org/web2009/. 2009. (Cited on page 77)

[Stian Thorgersen 2021] P. I. S. Stian Thorgersen. *Keycloak - identity and access management for modern applications: harness the power of Keycloak, OpenID Connect, and OAuth 2.0 protocols to secure applications*. Packt Publishing, 2021. (Cited on pages 20, 21)

[Tukey 1977] J. W. Tukey. *Exploratory data analysis*. Addison-Wesley, 1977. (Cited on page 51)

[V. Kistowski et al. 2015] J. v. Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao. How to build a benchmark. In: *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*. ACM, 2015. DOI: 10.1145/2668930.2688819. (Cited on pages 2, 7, 9, 27)

[Van Dongen and Van den Poel 2020] G. van Dongen and D. Van den Poel. Evaluation of stream processing frameworks. *IEEE Transactions on Parallel and Distributed Systems* 31.8 (2020). DOI: 10.1109/TPDS.2020.2978480. (Cited on page 79)

[Vasar et al. 2012] M. Vasar, S. N. Srirama, and M. Dumas. Framework for monitoring and testing web application scalability on the cloud. In: *Proceedings of the WICSA/ECSA 2012 Companion Volume*. ACM, 2012. DOI: 10.1145/2361999.2362008. (Cited on pages 77, 78)

[Veeramachaneni 2018] G. Veeramachaneni. *Loki: Prometheus-inspired, open source logging for cloud natives*. https://grafana.com/blog/2018/12/12/loki-prometheus-inspired-open-source-logging-for-cloud-natives/. 2018. (Cited on page 24)

[Vonheiden 2021] B. Vonheiden. Empirical scalability evaluation of window aggregation methods in distributed stream processing. Master's thesis. Kiel University, 2021. (Cited on pages 13, 78, 79)

[Waller et al. 2015] J. Waller, N. C. Ehmke, and W. Hasselbring. Including performance benchmarks into continuous integration to enable DevOps. *SIGSOFT Softw. Eng. Notes* 40.2 (2015). DOI: 10.1145/2735399.2735416. (Cited on page 82)

[Weber et al. 2014] A. Weber, N. Herbst, H. Groenda, and S. Kounev. Towards a resource elasticity benchmark for cloud environments. In: *Proceedings of the 2nd International Workshop on Hot Topics in Cloud Service Scalability*. ACM, 2014. DOI: 10.1145/2649563.2649571. (Cited on page 13)

[Zhang et al. 2011] L. Zhang, Y. Chen, F. Tang, and X. Ao. Design and implementation of cloud-based performance testing system for web services. In: *2011 6th International ICST Conference on Communications and Networking in China (CHINACOM)*. 2011. DOI: 10.1109/ChinaCom.2011.6158278. (Cited on page 78)

[Zukowski 2006] J. Zukowski. Scripting and JSR 223. In: *Java™ 6 Platform Revealed*. Apress, 2006. DOI: 10.1007/978-1-4302-0187-8_9. (Cited on page 38)